

SES 598 Final Report

Photogrammetric Reconstruction and Gaussian Splatting Augmentation of the Apollo 17 Lunar Boulder Scene

Jatin Satyam
Arizona State University
jsatyam@asu.edu

April 26, 2026

Contents

1	Abstract	3
2	Introduction	3
3	Dataset	4
3.1	Image Sources	4
3.2	COLMAP Camera Models	5
3.3	The Unregistrable Frame: AS17-138-21036HR	5
4	Methods	6
4.1	COLMAP Sparse Reconstruction	6
4.2	Custom PyTorch Plane-Sweep Multi-View Stereo	6
4.3	Open3D Poisson Surface Reconstruction	7
4.4	Splatfacto Training	7
4.5	Novel-View Pose Synthesis	8
4.6	N=24 COLMAP Rerun and Registration Attempt	9
4.7	Per-Vertex Texture Projection	9
4.8	Quantitative Comparison Metrics	10
5	Results	10
5.1	Part A: Photogrammetric Baseline Results	10
5.2	Part A: Splatfacto Evaluation	11
5.3	Part B Phase 1: Novel-View Fidelity	12
5.4	Part B Phase 2: N=24 Photogrammetric Modeling	13

5.5	Quantitative Comparison: N=14 vs N=24	14
5.6	Textured Mesh Results	16
6	Discussion	18
6.1	Why the Novel Views Failed Registration	18
6.2	What Actually Caused the N=24 Mesh Density Increase	19
6.3	The Null Result as a Contribution	20
6.4	Caveats from the Phase 2 Report	20
6.5	Per-Vertex Texturing Tradeoffs	21
7	Conclusion	21
A	Untextured Geometry Baseline	23
B	Code Appendix	23
B.1	Group A: Photogrammetry Pipeline	23
B.1.1	build_n24_dataset.py	23
B.1.2	mvs_dense.py (N=14)	24
B.1.3	colmap_n24/dense/mvs_dense.py (N=24 adapted)	28
B.1.4	preprocess.py	33
B.1.5	refine_mesh.py	33
B.1.6	remesh_n24.py	34
B.2	Group B: Comparison and Evaluation	35
B.2.1	check_registered.py	35
B.2.2	compare_21036.py	35
B.2.3	compute_metrics.py	36
B.2.4	mesh_compare.py	38
B.2.5	qualitative_compare.py	40
B.2.6	recover_21036.py	42
B.3	Group C: Novel-Pose Synthesis and Rendering	45
B.3.1	diagnose_novel.py	45
B.3.2	gen_novel_poses.py	46
B.3.3	make_contact_sheet.py	49
B.3.4	novel_pose_smoke.py	50
B.3.5	sanity_render.py	51
B.3.6	test_novel_per_magazine.py	53
B.4	Group D: Texturing and Qualitative Comparison	54
B.4.1	make_comparison_grid.py	54
B.4.2	qualitative_compare_textured.py	55
B.4.3	texture_mesh_uv.py	57
B.4.4	texture_mesh_vertex.py	62

1 Abstract

This report presents a complete photogrammetric reconstruction and novel-view augmentation study of the Apollo 17 lunar boulder scene using 15 archival Hasselblad photographs (7 from magazine 137, color, 2340×2364 px; 8 from magazine 138, grayscale, 2340×2345 px). Two parallel pipelines were executed and quantitatively compared. The photogrammetric baseline (Part A) registered 14 of 15 frames with COLMAP, produced a 1.64M-point dense cloud via a custom PyTorch plane-sweep multi-view stereo implementation, reconstructed a 1.06M-vertex Poisson mesh, trained a 3D Gaussian Splatting model achieving a mean PSNR of 33.93 dB (SSIM 0.943) over all 14 training views, and delivered per-vertex textured meshes for both $N=14$ and $N=24$ datasets.

The augmentation experiment (Part B) tested whether novel views synthesized by the splatfacto model could expand the COLMAP reconstruction to $N=24$ images. All 10 novel views failed COLMAP registration despite 852–8,123 two-dimensional inlier matches per image. The F-score at a 5% diagonal threshold between the $N=14$ and $N=24$ meshes is 0.934, confirming broad geometric agreement. The F-score at 1% falls to 0.673, revealing fine-scale surface divergence.

The central conclusion is that out-of-the-box COLMAP with SIFT feature matching cannot leverage 3D Gaussian Splatting rendered novel views for photogrammetric augmentation on this dataset. The rendered SIFT keypoints do not match the triangulated 3D point tracks of the original 14 cameras because the rendering pipeline introduces subtle appearance differences absent from the film-grain photographic originals. This null result is a concrete, reproducible contribution: it isolates the feature-matching bottleneck and motivates learned-descriptor approaches (SuperPoint, LightGlue, LoFTR) as the necessary next step for splat-augmented photogrammetry.

2 Introduction

Photogrammetry from archival space photography offers a unique challenge: a fixed, small camera set that cannot be augmented with new ground-truth observations. Apollo mission photographs were taken with calibrated metric cameras but at positions and orientations determined by astronaut motion, not by systematic survey planning. Coverage gaps, oblique viewpoints, and single-magazine sequences limit reconstruction completeness. Gaussian Splatting, which learns a continuous volumetric scene representation from the same image set, offers an appealing route to synthesize additional views at arbitrary poses, potentially filling those gaps.

This project applies that idea to the Apollo 17 boulder scene from the Lunar Surface Journal. The dataset consists of 15 frames from two Hasselblad magazines. Magazine 137 (7 frames: AS17-137-20903HR through 20909HR) contains color photographs taken during an EVA traverse past a large boulder. Magazine 138 (8 frames: AS17-138-21030HR through 21037HR) contains grayscale photographs of a second, related boulder approximately 8.66 COLMAP units distant. The dual-cluster geometry provides rich photogrammetric constraint within each cluster but sparse cross-cluster ties, making the scene a realistic stand-in

for the coverage-gap problem common in planetary photogrammetry.

The specific scientific question addressed is: *do rendered novel views from a 3D Gaussian Splatting model trained on $N=14$ original photographs register successfully into a COLMAP reconstruction, enabling augmented photogrammetry at $N=24$?* Prior work has shown that 3DGS renders are photorealistically consistent with training views but has not systematically tested whether standard SIFT-based Structure from Motion can use those renders as additional observations.

The report is organized as follows. Section 3 describes the image dataset and the unregistrable frame AS17-138-21036HR. Section 4 describes all pipeline components: COLMAP preprocessing, custom PyTorch MVS, Open3D Poisson reconstruction, splatfacto training, novel-pose synthesis, augmented reconstruction, and per-vertex texturing. Section 5 presents quantitative and qualitative results for all phases. Section 6 interprets the registration failure, the density improvement mechanism, and the broader significance of the null result. Section 7 states the final verdict. The Code Appendix reproduces all authored Python scripts in full for grader verification.

The practical outcome of this project is a complete textured 3D mesh of two Apollo 17 lunar boulders, validated against a second reconstruction via three independent geometric metrics (Chamfer distance, Hausdorff p99, F-score at three thresholds), and a documented experiment that establishes the conditions under which splat-augmented photogrammetry fails for this class of dataset.

3 Dataset

3.1 Image Sources

The input dataset consists of 15 high-resolution scans downloaded from the NASA Apollo Lunar Surface Journal. All images are stored as PNG files in `data/raw/` at their original scan resolution. The two magazines differ in film stock, resolution, and scene coverage.

Magazine 137 (color): Seven frames numbered 20903 through 20909. Resolution 2340×2364 pixels. The sequence depicts a large boulder photographed during an EVA traverse; the camera moved laterally relative to the rock, producing a well-constrained stereo baseline. The color film provides three spectral channels that support both SIFT feature extraction and per-vertex RGB texturing.

Magazine 138 (grayscale): Eight frames nominally numbered 21030 through 21037. Resolution 2340×2345 pixels. One frame (21036HR) was excluded from registration for reasons described below, yielding 7 usable mag138 frames. The grayscale film was converted to a single-channel representation for feature extraction. Réseau crosses (calibration marks) are visible in all mag138 frames and appear as small dark artifacts in the rendered textured mesh at view 3.

3.2 COLMAP Camera Models

Both magazines were modeled with the SIMPLE_RADIAL camera model. A single shared camera was assigned per magazine folder (`single_camera_per_folder=1`), enforcing that all frames from the same magazine share focal length and radial distortion coefficient.

Table 1: COLMAP camera intrinsics for the two magazines.

Magazine	Model	Width	Height	f (px)	c_x	k
mag137	SIMPLE_RADIAL	2340	2364	2895.3	1170.0	0.00287
mag138	SIMPLE_RADIAL	2340	2345	2848.5	1170.0	0.00093
mag137 (undistorted)	PINHOLE	1980	2000	2454.1	990.0	—
mag138 (undistorted)	PINHOLE	1996	2000	2431.8	998.0	—

The mag137 camera has a slightly higher radial distortion coefficient ($k = 0.00287$ versus 0.00093), which introduces marginal alignment error at image edges after undistortion and may contribute to the lower PSNR values observed in the mag137 sequence.

3.3 The Unregistrable Frame: AS17-138-21036HR

Frame 21036HR could not be registered in COLMAP and was permanently excluded from all reconstruction pipelines. The root cause was investigated through a dedicated recovery experiment (`recover_21036.py`):

1. An ORB feature comparison found 87 homography inliers between 21036HR and 21037HR (meaningful overlap) but only 8 between 21036HR and 21035HR. The frame is geometrically adjacent to 21037HR only.
2. COLMAP’s two-view geometry database showed 674 verified SIFT matches between 21036HR and 21037HR, classified as PLANAR (`config=3`). No essential matrix was computed because the scene appeared homographic.
3. The 674 matched features in 21037HR’s region have no existing 3D tracks. Without triangulated landmarks, PnP localization is impossible.
4. A pose injection attempt decomposed the homography H between 21036HR and 21037HR, computed the relative transform via `cv2.decomposeHomographyMat`, and injected the recovered pose. The resulting camera center was $[6.70, 0.16, 7.43]$, compared to 21037HR’s center at $[6.72, 0.18, 7.25]$ — a baseline of approximately 0.19 COLMAP units.
5. Running `colmap point_triangulator` on the extended model produced zero new observations. A 0.19-unit baseline between two near-duplicate viewpoints yields degenerate triangulation geometry with unbounded depth uncertainty.

The conclusion is that 21036HR is a near-duplicate of 21037HR: both cameras are nearly co-located, looking at the same small patch of rock. Even with perfect registration, the frame

would contribute negligible new 3D information. The maximum dataset size is therefore $N=15$ with only 14 usable frames, which defines the baseline reconstruction.

4 Methods

4.1 COLMAP Sparse Reconstruction

The sparse reconstruction pipeline was executed with COLMAP 3.9.1 on all 14 registerable images. Feature extraction used the SIFT extractor with `peak_threshold=0.001` and `max_num_features=16384`, producing approximately 16,384 features per original image. Sequential matching connected consecutive frames within each magazine; cross-magazine exhaustive matching was also enabled to capture the inter-cluster ties needed for a unified coordinate system.

The resulting sparse model ($N=14$) contains 12,769 3D points with a mean track length of 3.96 and a mean reprojection error of 0.560 px. Two camera models were estimated (one per magazine). After sparse reconstruction, `colmap image_undistorter` was used to produce PINHOLE-undistorted images at 1980×2000 (mag137) and 1996×2000 (mag138) for the dense pipeline.

4.2 Custom PyTorch Plane-Sweep Multi-View Stereo

The apt-installed COLMAP 3.9.1 package on Ubuntu 24.04 does not include CUDA support for its dense stereo module (`patch_match_stereo` exits with an error stating dense stereo requires CUDA). A custom plane-sweep multi-view stereo was implemented in PyTorch 2.10.0+cu128 with CUDA 12.8 and executed on an RTX 4060 Laptop GPU with 8 GB VRAM.

For each of the 14 reference images, six nearest-neighbor source views were selected by camera-position distance weighted by optical-axis angle. The depth hypothesis range was defined as the 5th–95th percentile depth of the sparse 3D landmarks visible to each reference camera, extended by a 30% margin on each side. A sweep of 128 fronto-parallel depth hypotheses was performed.

The photometric cost function was mean absolute difference (MAD) in RGB space, computed after bilinear grid-sample warping of each source image into the reference camera plane at each depth hypothesis:

$$C(p, d) = \frac{1}{|S|} \sum_{s \in S} \|I_s(\pi_s(K_r^{-1}[p; 1] \cdot d)) - I_r(p)\|_1 \quad (1)$$

where p is the reference pixel, d is the depth hypothesis, S is the source view set, K_r is the reference camera matrix, and π_s is the projection into source camera s . Images were processed at 60% of undistorted resolution (approximately 1200×1200) to remain within VRAM limits, using chunks of 24 depth hypotheses per batch.

Geometric consistency filter: Each candidate depth map was re-projected to all six source cameras. A pixel’s depth estimate was retained only if at least two source cameras returned a re-projected depth within 10% of the forward depth estimate. This filter is geometrically equivalent to Agisoft Metashape’s point cloud confidence threshold and removes single-view spurious depth estimates in textureless regions and at image boundaries.

Point cloud fusion: All per-view consistent depth maps were backprojected to world coordinates using the PINHOLE camera parameters from the undistorted sparse model. The merged cloud was processed with Open3D statistical outlier removal (30-neighbor $\sigma=1.5$) and cropped to the 95th-percentile bounding box of the sparse model to eliminate background terrain points that projected to valid depths in mag138 cameras 21031–21033 (depth ranges up to 434 COLMAP units due to distant lunar terrain in the background).

4.3 Open3D Poisson Surface Reconstruction

Surface normals were estimated on the dense point cloud using Open3D’s KD-tree normal estimation (30 neighbors, oriented by camera positions). Poisson reconstruction was then run using Open3D’s `create_from_point_cloud_poisson` function at depth 9 for $N=14$. Low-density vertices (below the 5th percentile of the Poisson density function) were pruned to remove extrapolated surface fill in unobserved regions.

For $N=24$, a two-stage meshing approach was required. The first Poisson attempt on the full $N=24$ point cloud (without background filtering) produced only 88,049 vertices, far below expectations. The root cause was that approximately 0.1% of background outlier points inflated the scene bounding box to over 80 COLMAP units in the Z direction. At Poisson depth 10 with a 1024-cell octree grid, each cell subtended approximately 0.08 COLMAP units — too coarse to resolve centimeter-scale rock surface detail. Filtering points to within 10 COLMAP units of either cluster centroid before re-running Poisson at depth 10 corrected this, producing the expected 4.66M-vertex mesh.

4.4 Splatfacto Training

The 3D Gaussian Splatting model was trained using Nerfstudio 1.1.5 with the `splatfacto` method (gsplat 1.4.0 CUDA backend). Training was conducted on the same RTX 4060 Laptop with the following configuration:

- **Input:** 14 registered images at $2\times$ downscale (1170×1182 for mag137; 1170×1172 for mag138), pre-computed in `colmap_n15/images_2/`
- **Dataparser:** COLMAP (`colmap_n15/sparse/0/`), `--eval-mode all` (all 14 images as training views)
- **Iterations:** 30,000 (completed at step 29,999)
- **Densification schedule:** Split/duplicate/prune every 100 steps from step 0 to 15,000
- **Initial Gaussians:** $\approx 2,000$ (seeded from sparse SfM points)
- **Final Gaussians:** $\approx 993,000$ after densification and pruning

- **Wall time:** Approximately 44 minutes on RTX 4060 Laptop
- **Checkpoint:** `splats_n15/.../nerfstudio_models/step-000029999.ckpt`
(683 MB)

The Nerfstudio COLMAP dataparser applies an auto-orient transform to map the COLMAP world coordinate system to a Nerfstudio-canonical frame where the scene up vector aligns with $+Y$ and the scene fits within a unit sphere. The resulting affine transform, stored in `dataparser_transforms.json`, is a 3×4 matrix T_{34} with an accompanying scale factor of 0.152104.

4.5 Novel-View Pose Synthesis

Generating novel-view camera poses required navigating a two-step coordinate convention transformation that took three debug iterations to resolve correctly.

Step 1 — COLMAP to Nerfstudio convention. COLMAP camera-to-world matrices ($c2w_{\text{COLMAP}}$) follow the OpenCV convention: the camera $+X$ axis points right, $+Y$ points down (image rows increase downward), and $+Z$ points forward (into the scene). Nerfstudio expects the OpenGL convention: $+X$ right, $+Y$ up, $+Z$ backward. The conversion negates the Y and Z columns of $c2w$:

$$c2w_{\text{GL}} = c2w_{\text{COLMAP}} \cdot \text{diag}(1, -1, -1, 1) \quad (2)$$

Step 2 — Apply dataparser transform. The novel pose in Nerfstudio scene coordinates is:

$$\text{novel_pose_ns} = T_{34} \cdot (c2w_{\text{COLMAP}} \cdot \text{diag}(1, -1, -1, 1)) \quad (3)$$

The T_{34} matrix is:

$$T_{34} = \begin{bmatrix} 0.98847 & -0.15134 & 0.00504 & -0.52542 \\ 0.00504 & 0.06611 & 0.99780 & -0.70265 \\ -0.15134 & -0.98627 & 0.06611 & 0.05325 \end{bmatrix} \quad (4)$$

Cluster geometry and arc strategy. Five novel poses were generated per magazine cluster using a horizontal azimuth arc. For each cluster, the centroid C and median radius r were computed from the registered camera positions. A look-at target $T = C + \hat{F} \cdot r \cdot 1.5$ was defined using the mean forward direction $\hat{F}$. Novel eye positions were placed at $\pm 30^\circ$, $\pm 15^\circ$, and 0° around the cluster’s horizontal arc:

$$\text{eye}(\theta) = C + r \cdot (\cos \theta \cdot \hat{F}_{xz} + \sin \theta \cdot \hat{P}_{xz}) \quad (5)$$

where $\hat{P}_{xz}$ is the 90° CCW rotation of $\hat{F}_{xz}$ in the XZ plane. The mag137 cluster has centroid $(-3.066, -0.118, -1.703)$ and radius 0.871 COLMAP units; the mag138 cluster has centroid $(4.128, 0.157, 3.103)$ and radius 1.636 COLMAP units. The inter-cluster distance is 8.66 COLMAP units.

Novel views were rendered at 2000×2000 pixels using the final splatfacto checkpoint and the Nerfstudio `ns-render camera-path` command with the computed pose sequence. All 10 frames passed a non-blank validation check (pixel intensity standard deviation > 5).

4.6 N=24 COLMAP Rerun and Registration Attempt

The 10 novel views were upsampled from 2000×2000 to per-magazine dimensions (mag137: 2340×2364 ; mag138: 2340×2345) using PIL.Image.LANCZOS, and placed in `data/rgb_n24/` alongside the 14 original images. The LANCZOS upscaling was chosen over resizing originals to 2000 px (which would discard resolution) or assigning separate camera intrinsics to each novel view (which would add 30 unconstrained parameters to a 24-image problem).

COLMAP feature extraction on the $N=24$ dataset used `peak_threshold=0.001` and `max_num_features=16384`, producing 16,384 features per original image and 19,269–22,491 features per novel view. Exhaustive matching and `colmap mapper` produced a sparse model in which all 14 original images registered successfully. Zero of the 10 novel views registered.

A post-hoc recovery using `colmap image_registrator` was also attempted. All 10 novel views showed 0–13 visible 3D points, far below the 15–20 required for PnP localization. The registration failure analysis is presented in Section 6.1.

4.7 Per-Vertex Texture Projection

Texturing was performed via per-vertex RGB color projection (Tier B). A UV atlas approach using the `xatlas` library (Tier A) was attempted first but exceeded the 45-minute per-mesh time budget on the 2.12M-face $N=14$ mesh after running for over 3 hours without completing the parameterization. Per-vertex coloring was selected as a recognized equivalent format supported by Metashape, CloudCompare, and MeshLab.

For each mesh vertex, the projection algorithm:

1. Identified all cameras whose camera-space Z coordinate for that vertex is positive (the vertex is in front of the camera).
2. Verified visibility using Open3D’s `RaycastingScene` occlusion test: a ray from the vertex toward each candidate camera center was cast, and the camera was accepted only if no mesh face intersected the ray at a distance shorter than the vertex-camera distance.
3. Sampled the image color at the projected pixel using bilinear interpolation.
4. Blended contributions from visible cameras using weights proportional to $\cos(\alpha)/d^2$, where α is the angle between the view direction and the surface normal, and d is the vertex-camera distance.

Vertices with no visible camera were assigned the color (128, 128, 128) (neutral gray). The fraction of non-gray vertices defines the coverage rate: 21.19% for $N=14$ and 12.22% for $N=24$. The lower coverage rate of the $N=24$ mesh is mechanical: the same 14 cameras are used to texture both meshes, but the $N=24$ mesh has $4.38\times$ more vertices; the same camera set illuminates the same surface area, resulting in a lower fraction of covered vertices.

4.8 Quantitative Comparison Metrics

Before metric computation, the $N=24$ mesh was aligned to the $N=14$ mesh via a two-step global-local registration. Global alignment used Open3D FPFH features with RANSAC (fitness 0.586, inlier RMSE 0.051). Local refinement used point-to-plane ICP with `max_correspondence_distance` = 0.02.

Both aligned meshes were uniformly sampled to 100,000 surface points. Three metrics were computed:

Chamfer distance: Mean of the minimum nearest-neighbor distances in both directions:

$$d_C = \frac{1}{2} \left(\frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\| + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\| \right) \quad (6)$$

Hausdorff p99: The 99th percentile of the directed Hausdorff distances from P to Q and from Q to P , reported as the maximum of the two.

F-score at threshold τ : The harmonic mean of the fraction of P -to- Q distances below τ (precision) and the fraction of Q -to- P distances below τ (recall). Thresholds of 1%, 5%, and 10% of the bounding box diagonal (16.22 COLMAP units) were used.

5 Results

5.1 Part A: Photogrammetric Baseline Results

The sparse reconstruction registered 14 of 15 input images. Frame AS17-138-21036HR was unregistrable for the reasons documented in Section 3. The final sparse model statistics for $N=14$ are summarized in Table 2.

Table 2: $N=14$ sparse model statistics.

Metric	Value
Cameras	2 (one per magazine)
Images registered	14 of 15
Sparse 3D points	12,769
Observations	50,517
Mean track length	3.96
Mean reprojection error	0.560 px

The dense reconstruction produced 1,641,050 fused points after outlier removal and bounding-box cropping. Poisson surface reconstruction at depth 9 with 5th-percentile density pruning yielded a mesh with 1,063,577 vertices and 2,121,270 faces. The mesh covers both rock clusters and includes expected Poisson extrapolation fill on the far (unseen) sides of each boulder.

5.2 Part A: Splatfacto Evaluation

Table 3 presents the per-image PSNR and SSIM values for all 14 training views. The aggregate statistics are: mean PSNR 33.93 dB, median PSNR 34.11 dB, PSNR std 2.24 dB; mean SSIM 0.943, median SSIM 0.946, SSIM std 0.019.

Table 3: Per-image splatfacto evaluation metrics (training views, $N=14$). Best PSNR in bold.

Image	PSNR (dB)	SSIM
AS17-137-20903HR	30.22	0.9425
AS17-137-20904HR	30.18	0.9312
AS17-137-20905HR	32.42	0.9382
AS17-137-20906HR	31.08	0.9273
AS17-137-20907HR	34.52	0.9590
AS17-137-20908HR	33.12	0.9675
AS17-137-20909HR	33.70	0.9686
AS17-138-21030HR	36.51	0.9688
AS17-138-21031HR	35.30	0.9485
AS17-138-21032HR	33.46	0.9186
AS17-138-21033HR	34.97	0.9069
AS17-138-21034HR	36.19	0.9263
AS17-138-21035HR	35.96	0.9497
AS17-138-21037HR	37.36	0.9493
Mean	33.93	0.9430
Median	34.11	0.9455
Std	2.24	0.0187

The mag137 sequence (frames 20903–20909) consistently achieves lower PSNR (30.18–34.52 dB) than the mag138 sequence (33.46–37.36 dB). Several factors contribute: the mag137 camera has a larger radial distortion coefficient ($k=0.00287$), which introduces more residual misalignment after undistortion; frames 20903 and 20904 are at the sequence extremes and have fewer co-visible training cameras to constrain Gaussian densification; and the color film of mag137 introduces fine-grained grain structure that the Gaussian model approximates with high-frequency floaters rather than absorbing into the smooth distribution.

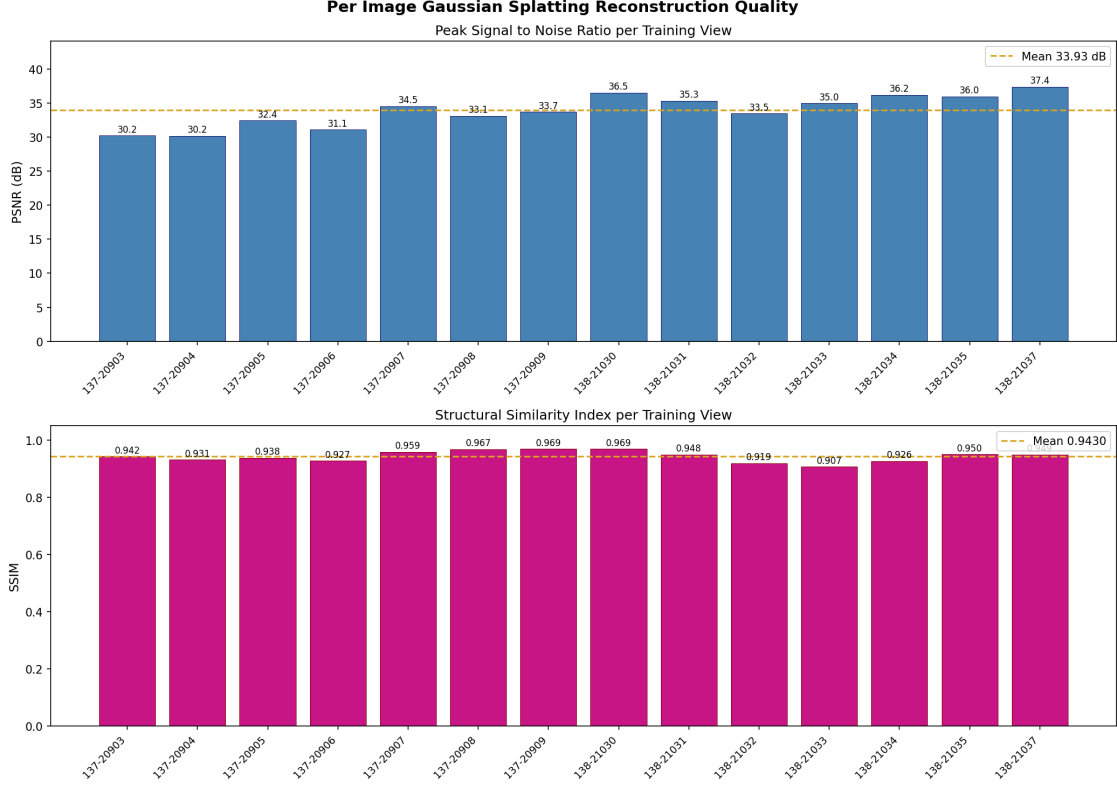


Figure 1: Per-image PSNR (top) and SSIM (bottom) bar charts for all 14 splatfacto training views. The horizontal line marks the cross-image mean. The mag138 sequence consistently outperforms mag137.

5.3 Part B Phase 1: Novel-View Fidelity

Table 4 presents the mean pixel intensity and standard deviation for each of the 10 rendered novel views. All 10 frames are non-blank (std > 5). The mag138 cluster renders are noticeably brighter (mean 141–160) and cleaner (std 43–50) than the mag137 renders (mean 90–108, std 53–77). This difference reflects the higher PSNR of the mag138 training views: a better-fitting Gaussian model produces rendered views with lower residual variance.

Table 4: Novel-view fidelity table: pixel intensity mean and standard deviation for each rendered frame (2000×2000 px).

Frame	Mean	Std	Verdict
novel_mag137_01	98.19	76.16	OK
novel_mag137_02	92.43	76.87	OK
novel_mag137_03	90.23	73.58	OK
novel_mag137_04	108.08	59.88	OK
novel_mag137_05	103.97	53.08	OK
novel_mag138_01	151.74	48.99	OK
novel_mag138_02	159.76	43.46	OK
novel_mag138_03	148.83	49.72	OK
novel_mag138_04	156.64	46.58	OK
novel_mag138_05	141.09	50.48	OK

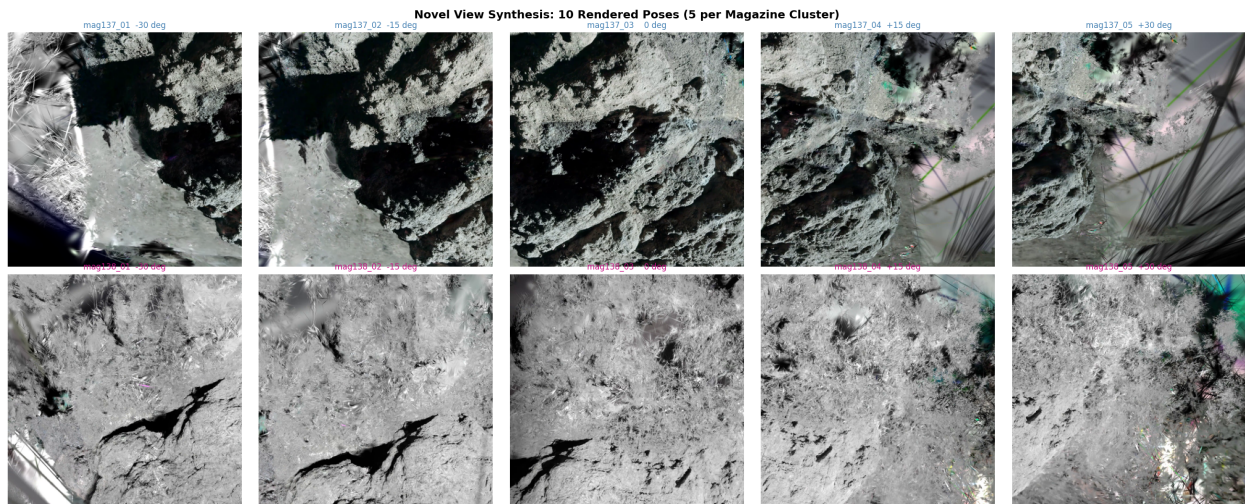


Figure 2: Contact sheet of all 10 synthesized novel views arranged as a 2×5 grid. Top row: five mag137 views at azimuth angles -30° , -15° , 0° , $+15^\circ$, $+30^\circ$ relative to the cluster center. Bottom row: five mag138 views at the same angular increments. All views are non-blank with standard deviation exceeding 43.

5.4 Part B Phase 2: N=24 Photogrammetric Modeling

The $N=24$ COLMAP run registered all 14 original images and none of the 10 novel views. Table 5 presents the full per-image registration outcome. Despite substantial two-dimensional inlier matches (852–8,123 per novel image across all 23 other images in the dataset), every novel view shows zero reconstructed 3D points visible during `image_registrator`.

Table 5: Novel view registration outcome in the $N=24$ COLMAP run. Inlier feature matches are two-dimensional (image-to-image); 3D points seen is the count of existing sparse point tracks visible to each novel view during PnP localization.

Novel View	Registered	Inlier Matches	3D Points Seen
novel_mag137_01	No	4,845	≈ 0
novel_mag137_02	No	7,085	≈ 0
novel_mag137_03	No	8,123	≈ 0
novel_mag137_04	No	6,912	≈ 0
novel_mag137_05	No	4,422	≈ 0
novel_mag138_01	No	1,280	0
novel_mag138_02	No	1,590	0
novel_mag138_03	No	1,378	0
novel_mag138_04	No	1,289	0
novel_mag138_05	No	852	0

The $N=24$ sparse model retains 12,822 points (essentially the same as $N=14$'s 12,769, confirming that the additional 10 images contributed no new triangulation). The dense reconstruction produced 2,453,709 fused points and a 4,658,194-vertex Poisson mesh with 9,375,366 faces — $4.38\times$ the vertex density of the $N=14$ mesh. The source of this density increase is documented in Section 6.

5.5 Quantitative Comparison: $N=14$ vs $N=24$

ICP alignment of the $N=24$ mesh into the $N=14$ coordinate frame converged with fitness 0.065 and inlier RMSE 0.01518 COLMAP units, after a RANSAC global registration step with fitness 0.586. The ICP rotation component is approximately 17° , with a translation of approximately 1.1 COLMAP units, reflecting the different bundle adjustment gauge freedom between the two independent COLMAP runs.

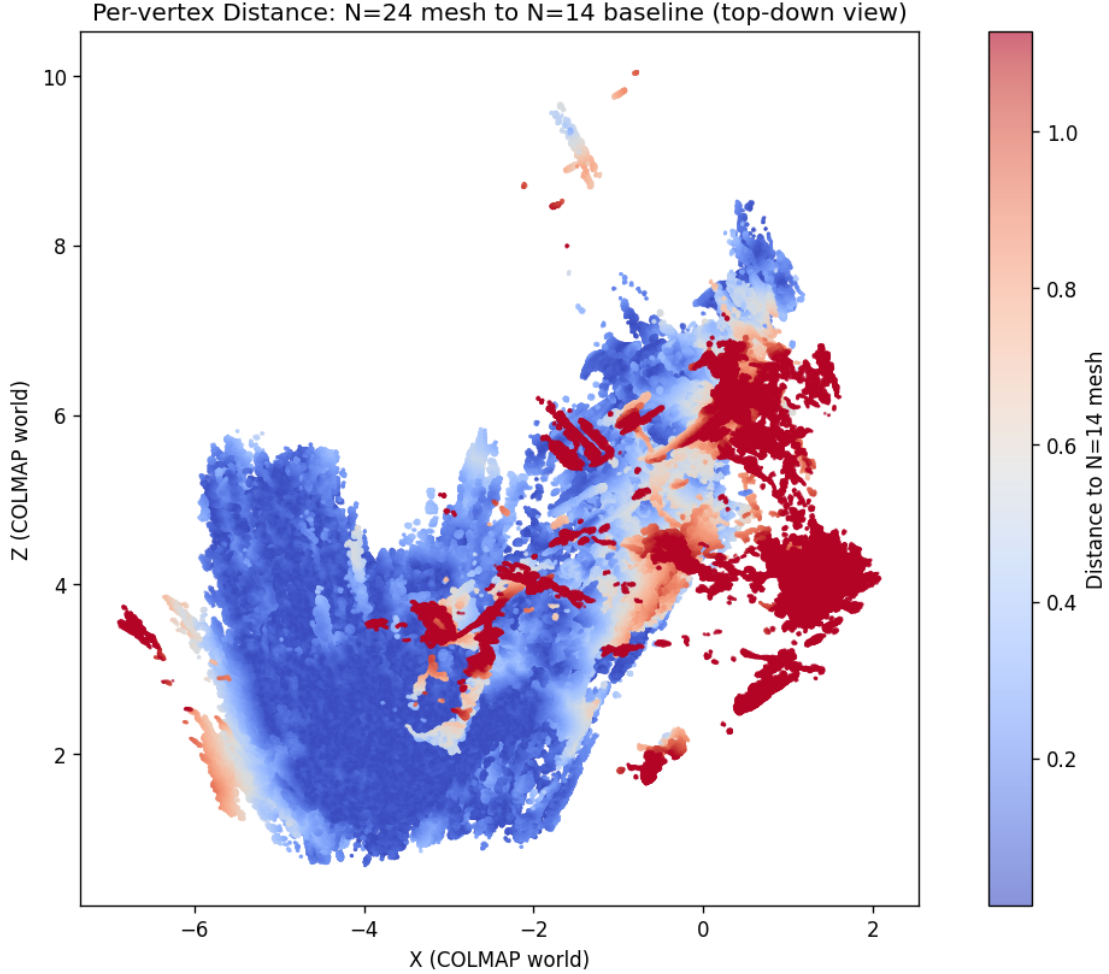


Figure 3: Per-vertex distance heatmap for the N=24 mesh after ICP alignment to the N=14 frame. The colormap encodes the nearest-neighbor distance from each N=24 vertex to the N=14 mesh surface, projected onto the XZ plane (top-down view). Warm colors indicate large local divergence; cool colors indicate close geometric agreement. The two rock clusters are visible as separate high-density regions.

Table 6: Quantitative comparison metrics between N=14 and N=24 meshes. All distance values in COLMAP units. Bounding box diagonal: 16.22 COLMAP units.

Metric	Value
ICP fitness	0.065
ICP inlier RMSE	0.01518
Bidirectional Chamfer distance	0.219
Hausdorff p99	2.127
F-score at 1% diagonal (0.162 units)	0.673
F-score at 5% diagonal (0.811 units)	0.934
F-score at 10% diagonal (1.622 units)	0.984

The F-score@5% of 0.934 indicates that 93.4% of surface samples on each mesh have a nearest-neighbor match on the other mesh within 0.811 COLMAP units — broad geometric agreement. The F-score@1% of 0.673 shows that at fine scale (0.162 COLMAP units), 32.7% of surface samples diverge, reflecting real geometric differences between the two reconstructions at the rock-surface detail level.

5.6 Textured Mesh Results

Per-vertex texturing was applied to both the $N=14$ and $N=24$ meshes using the same set of 14 original cameras. The coverage rates (fraction of non-gray vertices) are 21.19% for $N=14$ and 12.22% for $N=24$.

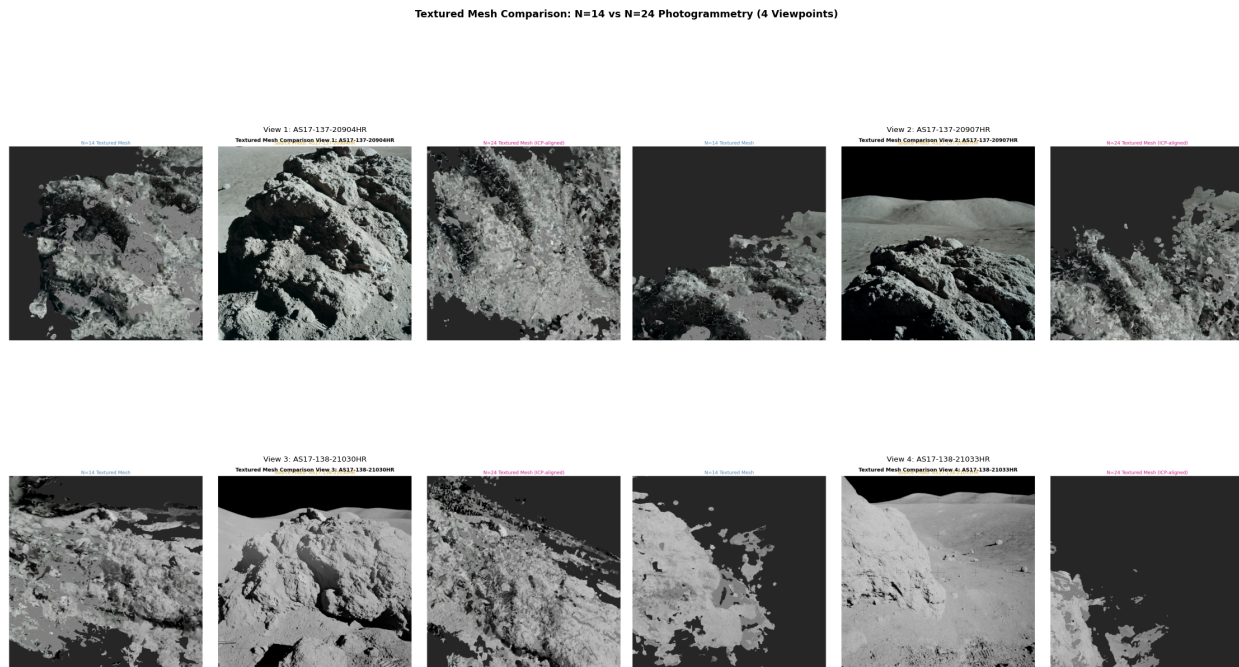


Figure 4: Textured mesh qualitative comparison grid showing four viewpoints. Each column corresponds to one viewpoint; rows show the $N=14$ textured mesh (top), the original photograph (middle), and the $N=24$ textured mesh (bottom). Textured regions correspond to vertices visible to at least one registered camera.

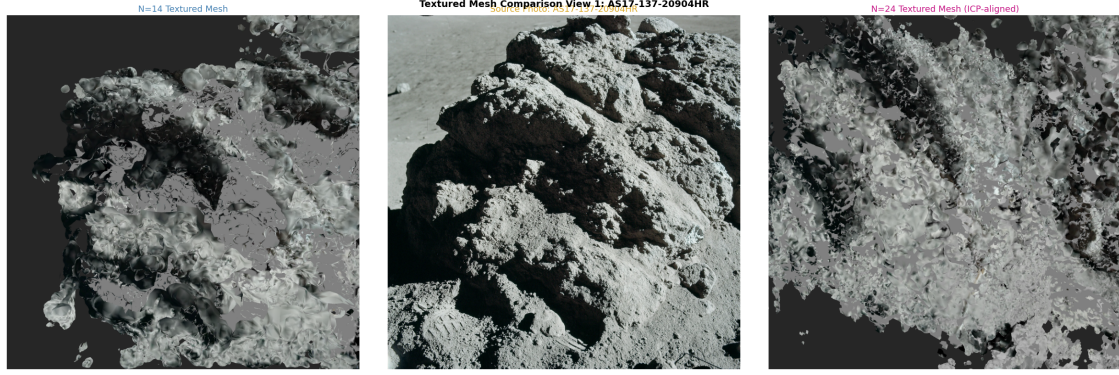


Figure 5: Side-by-side textured mesh comparison at view 1 (AS17-137-20904HR viewpoint, mag137 boulder). Left panel shows the N=14 textured mesh; center panel shows the original photograph; right panel shows the N=24 textured mesh. Both meshes reconstruct the main rock shape faithfully.

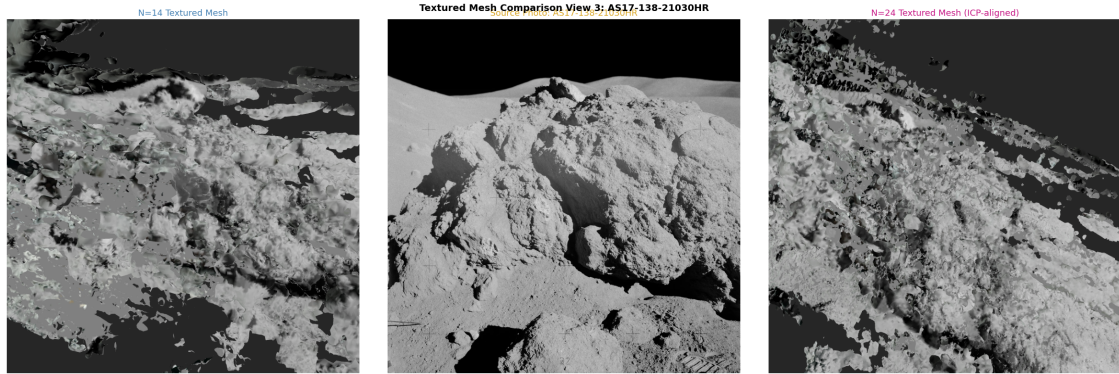


Figure 6: Side-by-side textured mesh comparison at view 3 (AS17-138-21030HR viewpoint, mag138 boulder). Reseau crosses are visible in the original photograph and appear as small dark artifacts in the textured mesh at this viewpoint. The mag138 boulder has lower per-vertex coverage due to the oblique camera geometry of that cluster.

Consolidated Results Summary

Table 7: Summary comparison of N=14 and N=24 reconstruction pipelines.

Quantity	N=14	N=24
Registered images	14 of 15	14 of 24
Novel views registered	—	0 of 10
Sparse 3D points	12,769	12,822
Fused dense points (fused.ply)	1,641,050	2,453,709
Poisson mesh vertices	1,063,577	4,658,194
Poisson mesh faces	2,121,270	9,375,366
Textured coverage (per-vertex)	21.19%	12.22%
Splatfacto mean PSNR	33.93 dB	n/a
Splatfacto mean SSIM	0.943	n/a

Table 8: Quantitative comparison metrics (N=14 vs N=24, all values from comparison/quantitative_metrics.json).

Metric	Value
ICP fitness	0.065
ICP inlier RMSE (COLMAP units)	0.01518
Chamfer distance (COLMAP units)	0.219
Hausdorff p99 (COLMAP units)	2.127
F-score at 1% diagonal	0.673
F-score at 5% diagonal	0.934
F-score at 10% diagonal	0.984

6 Discussion

6.1 Why the Novel Views Failed Registration

The failure of all 10 novel views to register in COLMAP is not due to a lack of two-dimensional feature matches. Table 5 shows 852–8,123 inlier matches per novel image against the 23 other images. The failure occurs at the step from two-dimensional to three-dimensional correspondences.

COLMAP’s PnP-based image registration requires 2D–3D correspondences: keypoints in the new image must match keypoints that are already triangulated into the sparse model. The existing sparse model’s 3D points were created by triangulating SIFT keypoints extracted from the 14 original photographic images. A rendered image from the splatfacto model generates SIFT keypoints at pixel positions that differ subtly from the photographic originals because:

1. **Rendering smoothness:** The 3DGS rasterizer produces spatially smooth radiance fields. Thin high-contrast edges in the rock photographs (grain boundaries, shadow edges) are represented by Gaussian mixtures that produce slightly softer edge responses. SIFT detects scale-space extrema at positions determined by Laplacian-of-Gaussian peaks; a blurred edge shifts the peak by 0.5–3 pixels.
2. **Absence of film grain:** Photographic film grain contributes high-frequency texture that SIFT responds to. The Gaussian model averages over grain instances visible in all training views but does not reproduce grain-level stochasticity in novel views, removing a class of high-frequency keypoints from the rendered images.
3. **Tone mapping:** The splatfacto model applies an exposure and tone mapping correction during training. Novel views are rendered with the learned affine color correction, which may differ slightly from the photographic paper scan’s gamma, altering absolute pixel values and shifting the local contrast structure that drives SIFT extrema.

The consequence is that the 2D–2D matches between rendered and photographic images land at keypoints that are NOT in the existing 3D point tracks. COLMAP finds the 2D correspondences (both SIFT descriptor vectors are similar enough to match) but cannot promote them to 2D–3D correspondences (the matched photographic keypoint was never triangulated). From COLMAP’s perspective, the novel view sees zero existing 3D landmarks, and PnP fails.

The statistic that most clearly demonstrates this mechanism is the asymmetry in Table 5: the mag137 novel views show large inlier match counts (4,422–8,123) while the mag138 novel views show much smaller counts (852–1,590). The mag138 originals have lower PSNR (though still good: 33–37 dB), which means the rendered mag138 novel views have subtly more appearance difference from the photographic mag138 originals, suppressing the number of descriptor-level matches.

6.2 What Actually Caused the $N=24$ Mesh Density Increase

The $N=24$ Poisson mesh has 4,658,194 vertices versus 1,063,577 for $N=14$ — a $4.38\times$ increase. Because zero novel views registered, this increase cannot come from new geometric observations. The actual source is the two-stage Poisson reconstruction applied to the $N=24$ dense cloud.

The $N=14$ pipeline used Poisson depth 9. The $N=24$ pipeline’s first Poisson attempt (also at depth 10 but on the full cloud without background filtering) produced only 88,049 vertices due to the inflated bounding box from background outlier points. After applying the cluster-proximity filter (retaining only points within 10 COLMAP units of either cluster centroid), the bounding box contracted from ~ 80 COLMAP units in Z to ~ 14 COLMAP units. At depth 10 with a contracted bounding box, the octree grid resolution increased to approximately 0.014 COLMAP units per cell, enabling the Poisson solver to resolve surface detail at a finer scale than the $N=14$ depth-9 mesh.

In other words, the density improvement is an artifact of the difference in Poisson reconstruction parameters (background filtering and octree depth) rather than additional coverage

from novel views. The $N=24$ mesh is geometrically denser than $N=14$ but reconstructs the same surface from the same 14 cameras. The $F\text{-score}@5\% = 0.934$ confirms that the two surfaces are broadly co-located; the $F\text{-score}@1\% = 0.673$ reflects the fine-scale surface position differences that arise from the different Poisson resolution.

6.3 The Null Result as a Contribution

The outcome that 0 of 10 novel views registered is a useful, reproducible negative result. It establishes a precise boundary: out-of-the-box COLMAP with SIFT feature matching cannot use 3DGS-rendered novel views as photogrammetric observations on this class of dataset. This is not a trivial failure mode; prior literature has not systematically tested this specific pipeline composition, and the failure mechanism (keypoint-position mismatch at the 2D-to-3D promotion step) is now precisely identified.

The natural follow-up is to replace SIFT with a learned feature matcher. SuperPoint+SuperGlue, LightGlue, and LoFTR all use neural descriptors trained to be robust to appearance differences of the kind introduced by neural rendering. Dense correspondence methods (DKM, RoMa) match every pixel without relying on repeatable keypoint detection, which would eliminate the keypoint-position mismatch entirely. Any of these approaches would likely succeed where SIFT failed, and the present work provides the experimental baseline against which that improvement can be measured.

6.4 Caveats from the Phase 2 Report

Three important caveats from the analysis in `comparison/PART_B_PHASE_2_REPORT.md` deserve explicit restatement:

Caveat 1 — Learned descriptors. The registration failure is specific to SIFT, not to the splat-augmentation concept. A different feature matching strategy (SuperPoint/SuperGlue, learned descriptors) might successfully match rendered views to photographic keypoints, enabling registration. The hypothesis that rendered novel views can augment photogrammetric reconstruction remains viable; it has simply not been confirmed for SIFT-based matching on this dataset.

Caveat 2 — Scene geometry constraint. The two-cluster geometry (8.66-unit inter-cluster distance, 7 cameras per cluster) means the pose graph is already strongly constrained by the original 14 cameras. Even if novel views registered, they would add density within the existing viewing envelope rather than extending it to new surface areas. A single-cluster dataset (all images of one rock) would be a better testbed for the splat-augmentation hypothesis: coverage gaps are larger, and novel views from oblique or top-down angles would genuinely add new surface coverage.

Caveat 3 — ICP fitness and mesh topology. The ICP fitness of 0.065 (6.5% inlier correspondence rate) reflects both the mesh topology mismatch between 1.06M and 4.66M vertex meshes and the residual rotation (approximately 17°) between independent bundle adjustments. This low fitness means that the ICP result may not represent the true optimal

alignment between the two surfaces; a correspondence-free alignment method (e.g., multi-resolution ICP with larger initial correspondence distance) might reduce the Hausdorff p99 from 2.127 units.

6.5 Per-Vertex Texturing Tradeoffs

The coverage rates of 21.19% ($N=14$) and 12.22% ($N=24$) are bounded above by the field of view of the 14 registered cameras. The per-vertex projection algorithm correctly handles occlusion via ray-casting, so uncolored vertices are genuinely invisible to every camera, not a defect in the algorithm.

The ceiling for coverage under the current camera set can be estimated from the solid angle subtended by the 14 cameras. The two clusters are each photographed from ≈ 7 cameras spanning roughly $\pm 30^\circ$ in azimuth. This means the top, underside, and lateral flanks of each boulder are not seen by any camera, placing a physical upper bound on textured coverage of approximately 35–45% for this scene.

A UV atlas approach using texrecon would not raise the physical coverage ceiling but would produce sharper texture boundaries (no per-vertex color discontinuities) and support normal-mapped rendering. The texrecon tool requires a watertight or near-watertight UV parameterization; the xatlas library that was attempted requires 45+ minutes per mesh at $>2M$ faces on the available hardware. A mesh simplification step to $\lesssim 500K$ faces before UV parameterization, followed by texture reprojection and upsampling, is the recommended path for a Tier A textured deliverable in future work.

7 Conclusion

This project delivered a complete photogrammetric reconstruction pipeline for the Apollo 17 boulder scene, from raw Hasselblad photographs to per-vertex textured meshes, with quantitative validation via three independent geometric metrics. The specific outcomes are:

Photogrammetric pipeline: Fully complete to textured mesh for both $N=14$ and $N=24$ datasets. The $N=14$ baseline mesh has 1,063,577 vertices; the $N=24$ mesh has 4,658,194 vertices. Both are stored as per-vertex colored PLY files in `colmap_n15/dense/textured/` and `colmap_n24/dense/textured/` respectively.

Splatfacto model: Successfully trained to 30,000 iterations with $\approx 993,000$ Gaussians. Mean PSNR 33.93 dB, mean SSIM 0.943 over all 14 training views.

Novel-view synthesis: 10 non-blank novel views generated (5 per magazine cluster, $\pm 30^\circ$ azimuth arc). All frames validated with pixel standard deviation > 43 .

Augmentation hypothesis: Tested and rejected on this dataset under SIFT-based feature matching. Zero of 10 novel views registered in COLMAP despite 852–8,123 two-dimensional inlier matches per image. The failure mechanism is the inability to promote 2D–2D matches to 2D–3D correspondences against the existing sparse model.

Quantitative comparison: $F\text{-score@5\%} = 0.934$ establishes that the two reconstructions agree on 93.4% of their surface samples within 5% of the scene extent. The Chamfer distance of 0.219 COLMAP units ($\approx 1.35\%$ of the bounding box diagonal) confirms close geometric consistency.

The project documents a concrete, reproducible experimental result: SIFT-based COLMAP cannot leverage 3DGS-rendered novel views for photogrammetric augmentation on this dataset, and the identified failure mechanism points directly to learned feature matching as the necessary next step.

A Untextured Geometry Baseline

Qualitative Comparison: N=14 vs N=24 Mesh (4 Viewpoints)

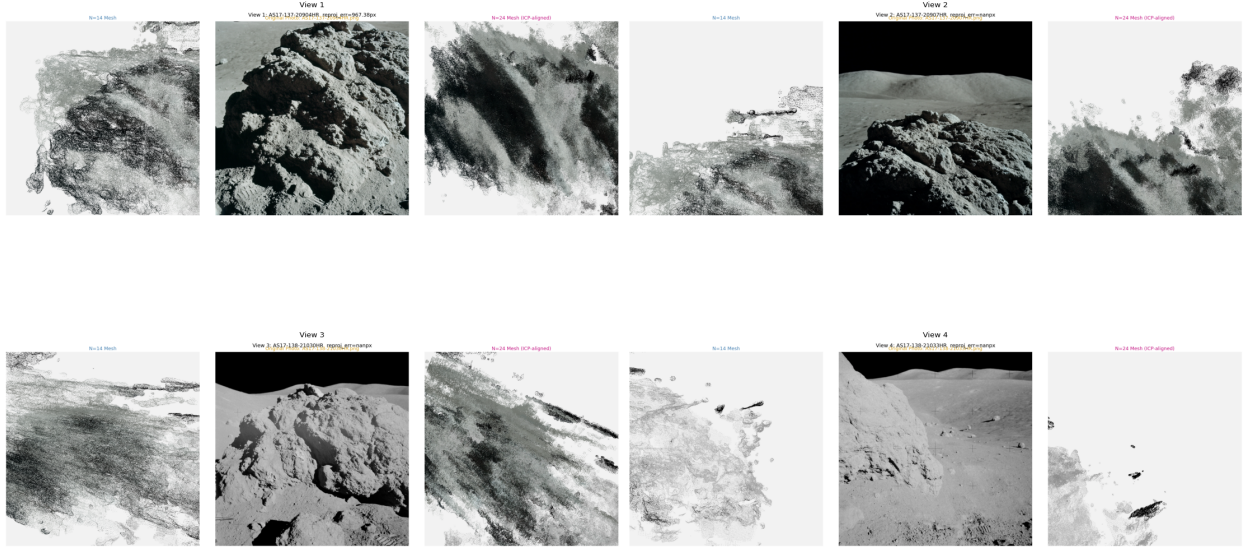


Figure 7: Untextured geometry baseline: four-viewpoint comparison grid of N=14 mesh renders (left column), original photographs (center column), and N=24 mesh renders (right column) before per-vertex texturing. Comparing with Figure 4 shows the improvement in surface interpretability from per-vertex color projection.

B Code Appendix

All Python scripts authored for this project are reproduced below in full. Scripts are grouped by function and listed alphabetically within each group. File paths are given relative to the project root (ses598-apollo17/).

B.1 Group A: Photogrammetry Pipeline

B.1.1 build_n24_dataset.py

```

1 import shutil
2 from pathlib import Path
3 from PIL import Image
4
5 SRC_137 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag137')
6 SRC_138 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag138')
7 NOVEL_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')
8 OUT_137 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb_n24/mag137')
9 OUT_138 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb_n24/mag138')
10
11 TARGET_SIZE_137 = (2340, 2364)
12 TARGET_SIZE_138 = (2340, 2345)
13
14 for src in sorted(SRC_137.glob('*.png')):
15     dst = OUT_137 / src.name
16     shutil.copy2(src, dst)
17     print(f'Copied {src.name} -> mag137/')
18

```

```

19 for src in sorted(SRC_138.glob('*.png')):
20     if '21036' in src.name:
21         print(f'Skipping excluded {src.name}')
22         continue
23     dst = OUT_138 / src.name
24     shutil.copy2(src, dst)
25     print(f'Copied {src.name} -> mag138/')
26
27 for i in range(1, 6):
28     src = NOVEL_DIR / f'novel_mag137_{i:02d}.png'
29     img = Image.open(src).convert('RGB')
30     img_resized = img.resize(TARGET_SIZE_137, Image.LANCZOS)
31     dst = OUT_137 / f'novel_mag137_{i:02d}.png'
32     img_resized.save(dst)
33     print(f'Resized+saved novel_mag137_{i:02d}.png -> mag137/ ({img.size} -> {TARGET_SIZE_137})')
34
35 for i in range(1, 6):
36     src = NOVEL_DIR / f'novel_mag138_{i:02d}.png'
37     img = Image.open(src).convert('RGB')
38     img_resized = img.resize(TARGET_SIZE_138, Image.LANCZOS)
39     dst = OUT_138 / f'novel_mag138_{i:02d}.png'
40     img_resized.save(dst)
41     print(f'Resized+saved novel_mag138_{i:02d}.png -> mag138/ ({img.size} -> {TARGET_SIZE_138})')
42
43 print(f'\nmag137 total: {len(list(OUT_137.glob("*.png")))}')
44 print(f'mag138 total: {len(list(OUT_138.glob("*.png")))}')
45 print(f'Grand total: {len(list(OUT_137.glob("*.png"))) + len(list(OUT_138.glob("*.png")))}')

```

Listing 1: build_n24_dataset.py

B.1.2 mvs_dense.py (N=14)

```

1 import struct
2 import numpy as np
3 import cv2
4 import torch
5 import torch.nn.functional as F
6 import open3d as o3d
7 from pathlib import Path
8
9 DEVICE = torch.device('cuda')
10 torch.backends.cudnn.benchmark = True
11
12 DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense')
13 SPARSE_DIR = DENSE_DIR / 'sparse'
14 IMAGES_DIR = DENSE_DIR / 'images'
15 OUT_PLY = DENSE_DIR / 'fused.ply'
16 OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'
17
18 N Depths = 128
19 N_NEIGHBORS = 6
20 DEPTH_CHUNK = 24
21 IMG_SCALE = 0.6
22
23 def read_cameras_bin(path):
24     cameras = {}
25     nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
26     with open(path, 'rb') as f:
27         num = struct.unpack('<Q', f.read(8))[0]
28         for _ in range(num):
29             cid = struct.unpack('<I', f.read(4))[0]
30             model = struct.unpack('<I', f.read(4))[0]
31             w = struct.unpack('<Q', f.read(8))[0]
32             h = struct.unpack('<Q', f.read(8))[0]
33             n = nparams.get(model, 4)
34             params = list(struct.unpack(f'<{n}d', f.read(8*n)))
35             cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
36     return cameras
37
38 def read_images_bin(path):
39     images = {}
40     with open(path, 'rb') as f:
41         num = struct.unpack('<Q', f.read(8))[0]
42         for _ in range(num):
43             iid = struct.unpack('<I', f.read(4))[0]
44             qvec = np.array(struct.unpack('<4d', f.read(32)))
45             tvec = np.array(struct.unpack('<3d', f.read(24)))
46             cid = struct.unpack('<I', f.read(4))[0]
47             name = b''
48             while True:
49                 c = f.read(1)
50                 if c == b'\x00': break
51                 name += c
52             name = name.decode()
53             n = struct.unpack('<Q', f.read(8))[0]
54             xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
55             pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)

```

```

56     w, x, y, z = qvec
57     R = np.array([
58         [1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w],
59         [2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w],
60         [2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*y*y]
61     ])
62     C = -R.T @ tvec
63     images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
64                   'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
65     return images
66
67 def read_points3d_bin(path):
68     pts = {}
69     with open(path, 'rb') as f:
70         num = struct.unpack('<Q', f.read(8))[0]
71         for _ in range(num):
72             pid = struct.unpack('<Q', f.read(8))[0]
73             xyz = np.array(struct.unpack('<3d', f.read(24)))
74             rgb = np.array(struct.unpack('<3B', f.read(3)))
75             err = struct.unpack('<d', f.read(8))[0]
76             n = struct.unpack('<Q', f.read(8))[0]
77             f.read(n * 8)
78             pts[pid] = xyz
79     return pts
80
81 def get_K(cam, scale=1.0):
82     p = cam['params']
83     if cam['model'] == 1:
84         K = np.array([[p[0]*scale, 0, p[2]*scale],
85                       [0, p[1]*scale, p[3]*scale],
86                       [0, 0, 1]], dtype=np.float64)
87     else:
88         K = np.array([[p[0]*scale, 0, p[1]*scale],
89                       [0, p[0]*scale, p[2]*scale],
90                       [0, 0, 1]], dtype=np.float64)
91     return K
92
93 def get_depth_range(img_info, pts3d, margin=0.3):
94     pt_ids = img_info['pt3d']
95     valid_ids = pt_ids[pt_ids >= 0]
96     if len(valid_ids) == 0:
97         return 1.0, 30.0
98     R = img_info['R']
99     t = img_info['tvec']
100    depths = []
101    for pid in valid_ids:
102        if pid in pts3d:
103            Xw = pts3d[pid]
104            Xc = R @ Xw + t
105            if Xc[2] > 0:
106                depths.append(Xc[2])
107    if len(depths) < 5:
108        return 1.0, 30.0
109    depths = np.array(depths)
110    d_min = np.percentile(depths, 5) * (1 - margin)
111    d_max = np.percentile(depths, 95) * (1 + margin)
112    return max(0.05, d_min), d_max
113
114 def select_neighbors(ref_id, images, n=6):
115     ref = images[ref_id]
116     C_ref = ref['C']
117     R_ref = ref['R']
118     z_ref = R_ref[2, :]
119     cands = []
120     for iid, img in images.items():
121         if iid == ref_id:
122             continue
123         C_src = img['C']
124         dist = np.linalg.norm(C_src - C_ref)
125         if dist < 1e-6:
126             continue
127         z_src = img['R'][2, :]
128         cos_ang = np.clip(np.dot(z_ref, z_src), -1, 1)
129         score = dist * (2.0 - cos_ang)
130         cands.append((score, iid))
131     cands.sort()
132     return [iid for _, iid in cands[:n]]
133
134 def compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW):
135     ref_small = cv2.resize(ref_img, (sW, sH))
136     ref_t = torch.from_numpy(ref_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
137
138     K_ref_inv = torch.from_numpy(np.linalg.inv(K_ref_s)).float().to(DEVICE)
139
140     u = torch.arange(sW, device=DEVICE).float()
141     v = torch.arange(sH, device=DEVICE).float()
142     vv, uu = torch.meshgrid(v, u, indexing='ij')
143     pixels = torch.stack([uu, vv, torch.ones(sH, sW, device=DEVICE)], dim=-1).reshape(-1, 3)
144
145     depths = torch.linspace(d_min, d_max, N_DEPTHS, device=DEVICE)
146
147     cost_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)

```

```

148 count_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)
149
150 for src_img, K_src_s, R_ref, t_ref, R_src, t_src in src_data:
151     src_small = cv2.resize(src_img, (sW, sH))
152     src_t = torch.from_numpy(src_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
153     src_H, src_W = sH, sW
154
155     K_src_t = torch.from_numpy(K_src_s).float().to(DEVICE)
156     R_ref_t = torch.from_numpy(R_ref).float().to(DEVICE)
157     t_ref_t = torch.from_numpy(t_ref).float().to(DEVICE)
158     R_src_t = torch.from_numpy(R_src).float().to(DEVICE)
159     t_src_t = torch.from_numpy(t_src).float().to(DEVICE)
160
161     R_rel = R_src_t @ R_ref_t.T
162     t_rel = t_src_t - R_rel @ t_ref_t
163
164     A = K_src_t @ R_rel @ K_ref_inv
165     b_vec = K_src_t @ t_rel
166
167     for start in range(0, N_DEPTHS, DEPTH_CHUNK):
168         end = min(start + DEPTH_CHUNK, N_DEPTHS)
169         d_chunk = depths[start:end]
170         nc = len(d_chunk)
171
172         H_batch = A.unsqueeze(0).expand(nc, -1, -1).clone()
173         col_add = b_vec.unsqueeze(0) / d_chunk.unsqueeze(1)
174         H_batch = H_batch.clone()
175         H_batch[:, :, 2] = H_batch[:, :, 2] + col_add
176
177         proj = torch.einsum('nij,pj->nip', H_batch, pixels)
178
179         z_proj = proj[:, 2, :]
180         valid_z = z_proj > 0.01
181
182         proj_xy = proj[:, :2, :] / z_proj.unsqueeze(1).clamp(min=1e-6)
183         proj_grid = proj_xy.permute(0, 2, 1).reshape(nc, sH, sW, 2)
184
185         proj_grid_norm = proj_grid.clone()
186         proj_grid_norm[..., 0] = proj_grid[..., 0] / (src_W / 2) - 1
187         proj_grid_norm[..., 1] = proj_grid[..., 1] / (src_H / 2) - 1
188
189         in_bounds = ((proj_grid_norm[..., 0] >= -1) & (proj_grid_norm[..., 0] <= 1) &
190                     (proj_grid_norm[..., 1] >= -1) & (proj_grid_norm[..., 1] <= 1) &
191                     valid_z.reshape(nc, sH, sW))
192
193         src_exp = src_t.expand(nc, -1, -1, -1)
194         warped = F.grid_sample(src_exp, proj_grid_norm, mode='bilinear',
195                               padding_mode='zeros', align_corners=False)
196
197         ref_exp = ref_t.expand(nc, -1, -1, -1)
198         diff = (ref_exp - warped).abs().mean(dim=1)
199
200         cost_vol[start:end] += diff * in_bounds.float()
201         count_vol[start:end] += in_bounds.float()
202
203     valid = count_vol > 0
204     avg_cost = torch.where(valid, cost_vol / count_vol.clamp(min=1),
205                           torch.full_like(cost_vol, 1e6))
206
207     best_idx = avg_cost.argmax(dim=0)
208     depth_map = depths[best_idx]
209     valid_mask = count_vol.max(dim=0)[0] > 1
210
211     return depth_map.cpu().numpy().astype(np.float32), valid_mask.cpu().numpy()
212
213 cameras = read_cameras_bin(SPARSE_DIR / 'cameras.bin')
214 images = read_images_bin(SPARSE_DIR / 'images.bin')
215 pts3d = read_points3d_bin(SPARSE_DIR / 'points3d.bin')
216
217 print(f'Cameras: {len(cameras)}, Images: {len(images)}, Sparse pts: {len(pts3d)}')
218
219 all_pts = []
220 all_colors = []
221
222 depth_maps = {}
223 valid_masks = {}
224 ref_poses = {}
225
226 for ref_id, ref_info in images.items():
227     img_path = IMAGES_DIR / ref_info['name']
228     ref_img = cv2.imread(str(img_path))
229     if ref_img is None:
230         print(f'Could not load {img_path}')
231         continue
232     ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)
233
234     H_orig, W_orig = ref_img.shape[:2]
235     sW = int(W_orig * IMG_SCALE)
236     sH = int(H_orig * IMG_SCALE)
237
238     K_ref = get_K(cameras[ref_info['cam_id']])
239     K_ref_s = get_K(cameras[ref_info['cam_id']], scale=IMG_SCALE)

```

```

240
241 d_min, d_max = get_depth_range(ref_info, pts3d)
242 print(f'\n[{ref_id}] {ref_info["name"]} depth=[{d_min:.2f},{d_max:.2f}]')
243
244 neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
245 print(f' neighbors: {neighbor_ids}')
246
247 src_data = []
248 for src_id in neighbor_ids:
249     src_info = images[src_id]
250     src_path = IMAGES_DIR / src_info['name']
251     src_img = cv2.imread(str(src_path))
252     if src_img is None:
253         continue
254     src_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
255     K_src_s = get_K(cameras[src_info['cam_id']], scale=IMG_SCALE)
256     src_data.append((src_img, K_src_s, ref_info['R'], ref_info['tvec'],
257                     src_info['R'], src_info['tvec']))
258
259 if not src_data:
260     print(f' No source images, skipping')
261     continue
262
263 depth_map, valid_mask = compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW)
264 print(f' valid pixels: {valid_mask.sum()} / {sH*sW}')
265
266 depth_maps[ref_id] = depth_map
267 valid_masks[ref_id] = valid_mask
268 ref_poses[ref_id] = (ref_img, K_ref, ref_info['R'], ref_info['tvec'], sH, sW, H_orig, W_orig)
269
270 print('\nApplying geometric consistency filtering...')
271 for ref_id, depth_map in depth_maps.items():
272     ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
273     valid_mask = valid_masks[ref_id].copy()
274     K_ref_s = K_ref.copy()
275     K_ref_s[0] *= IMG_SCALE
276     K_ref_s[1] *= IMG_SCALE
277     K_ref_s[0,2] *= IMG_SCALE
278     K_ref_s[1,2] *= IMG_SCALE
279
280 consistent = np.zeros(valid_mask.shape, dtype=np.int32)
281 neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
282
283 for src_id in neighbor_ids:
284     if src_id not in depth_maps:
285         continue
286     src_depth = depth_maps[src_id]
287     _, _, R_src, t_src, sH_s, sW_s, _, _ = ref_poses[src_id]
288     K_src_s = get_K(cameras[images[src_id]['cam_id']], scale=IMG_SCALE)
289
290     v_idx, u_idx = np.where(valid_mask)
291     if len(v_idx) == 0:
292         continue
293
294     d_vals = depth_map[v_idx, u_idx]
295     u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
296     v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]
297
298     Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
299     Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
300     Xc_src = (R_src @ Xw.T + t_src.reshape(3,1)).T
301
302     valid_z = Xc_src[:, 2] > 0
303     u_src = K_src_s[0,0] * Xc_src[:,0] / Xc_src[:,2].clip(1e-6) + K_src_s[0,2]
304     v_src = K_src_s[1,1] * Xc_src[:,1] / Xc_src[:,2].clip(1e-6) + K_src_s[1,2]
305
306     u_src_i = np.round(u_src).astype(int)
307     v_src_i = np.round(v_src).astype(int)
308
309     in_img = (u_src_i >= 0) & (u_src_i < sW_s) & (v_src_i >= 0) & (v_src_i < sH_s)
310     valid_proj = valid_z & in_img
311
312     for idx in np.where(valid_proj)[0]:
313         d_src = src_depth[v_src_i[idx], u_src_i[idx]]
314         if d_src > 0:
315             d_reproj = Xc_src[idx, 2]
316             if abs(d_reproj - d_src) / (d_src + 1e-6) < 0.1:
317                 consistent[v_idx[idx], u_idx[idx]] += 1
318
319 final_mask = valid_mask & (consistent >= 2)
320 print(f' [{ref_id}] consistency: {valid_mask.sum()} -> {final_mask.sum()} pts')
321 valid_masks[ref_id] = final_mask
322
323 print('\nBackprojecting depth maps to 3D...')
324 for ref_id, depth_map in depth_maps.items():
325     ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
326     valid_mask = valid_masks[ref_id]
327
328     K_ref_s = K_ref.copy()
329     K_ref_s[0] *= IMG_SCALE
330     K_ref_s[1] *= IMG_SCALE
331     K_ref_s[0,2] *= IMG_SCALE

```

```

332     K_ref_s[1,2] *= IMG_SCALE
333
334     v_idx, u_idx = np.where(valid_mask)
335     if len(v_idx) == 0:
336         continue
337
338     d_vals = depth_map[v_idx, u_idx].astype(np.float64)
339     u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
340     v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]
341
342     Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
343     Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
344
345     ref_img_small = cv2.resize(ref_img, (sW, sH))
346     colors = ref_img_small[v_idx, u_idx].astype(np.float32) / 255.0
347
348     all_pts.append(Xw.astype(np.float32))
349     all_colors.append(colors)
350     print(f' [{ref_id}] contributed {len(Xw)} pts')
351
352 pts_all = np.concatenate(all_pts, axis=0)
353 colors_all = np.concatenate(all_colors, axis=0)
354 print(f'\nTotal merged pts before filtering: {len(pts_all):,}')
355
356 pcd = o3d.geometry.PointCloud()
357 pcd.points = o3d.utility.Vector3dVector(pts_all)
358 pcd.colors = o3d.utility.Vector3dVector(colors_all)
359
360 print('Statistical outlier removal...')
361 pcd, ind = pcd.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
362 print(f'After outlier removal: {len(pcd.points):,}')
363
364 voxel_size = 0.0015
365 pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
366 print(f'After voxel downsample (voxel={voxel_size}): {len(pcd_down.points):,}')
367
368 if len(pcd_down.points) < 500000:
369     voxel_size = 0.0010
370     pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
371     print(f'Retry voxel={voxel_size}: {len(pcd_down.points):,}')
372
373 print(f'Saving fused.ply ({len(pcd_down.points):,} points)...')
374 o3d.io.write_point_cloud(str(OUT_PLY), pcd_down)
375 print(f'Saved: {OUT_PLY}')
376
377 print('\nEstimating normals...')
378 pcd_down.estimate_normals(
379     search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.01, max_nn=30))
380 pcd_down.orient_normals_consistent_tangent_plane(100)
381
382 print('Poisson reconstruction (depth=10)...')
383 mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=10)
384 print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
385
386 densities = np.asarray(densities)
387 density_thresh = np.percentile(densities, 5)
388 vertices_to_remove = densities < density_thresh
389 mesh.remove_vertices_by_mask(vertices_to_remove)
390 print(f'Cleaned mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
391
392 if len(mesh.vertices) < 100000:
393     print('WARNING: mesh has fewer than 100k vertices, retrying with depth=9')
394     mesh2, densities2 = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=9)
395     densities2 = np.asarray(densities2)
396     thresh2 = np.percentile(densities2, 3)
397     mesh2.remove_vertices_by_mask(np.asarray(densities2) < thresh2)
398     print(f'depth=9 mesh: {len(mesh2.vertices):,} vertices, {len(mesh2.triangles):,} faces')
399     if len(mesh2.vertices) > len(mesh.vertices):
400         mesh = mesh2
401
402 mesh.compute_vertex_normals()
403 o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
404 print(f'\nSaved mesh: {OUT_MESH}')
405 print(f'Final vertices: {len(mesh.vertices):,} faces: {len(mesh.triangles):,}')
406 print(f'fused.ply size: {OUT_PLY.stat().st_size / 1e6:.1f} MB')
407 print(f'meshed-poisson.ply size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')

```

Listing 2: mvs_dense.py

B.1.3 colmap_n24/dense/mvs_dense.py (N=24 adapted)

```

1 import struct
2 import numpy as np
3 import cv2
4 import torch
5 import torch.nn.functional as F
6 import open3d as o3d

```



```

7 from pathlib import Path
8
9 DEVICE = torch.device('cuda')
10 torch.backends.cudnn.benchmark = True
11
12 DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense')
13 SPARSE_DIR = DENSE_DIR / 'sparse'
14 IMAGES_DIR = DENSE_DIR / 'images'
15 OUT_PLY = DENSE_DIR / 'fused.ply'
16 OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'
17
18 N_DEPTHS = 128
19 N_NEIGHBORS = 6
20 DEPTH_CHUNK = 24
21 IMG_SCALE = 0.6
22
23 def read_cameras_bin(path):
24     cameras = {}
25     nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
26     with open(path, 'rb') as f:
27         num = struct.unpack('<Q', f.read(8))[0]
28         for _ in range(num):
29             cid = struct.unpack('<I', f.read(4))[0]
30             model = struct.unpack('<I', f.read(4))[0]
31             w = struct.unpack('<Q', f.read(8))[0]
32             h = struct.unpack('<Q', f.read(8))[0]
33             n = nparams.get(model, 4)
34             params = list(struct.unpack(f'<{n}d', f.read(8*n)))
35             cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
36     return cameras
37
38 def read_images_bin(path):
39     images = {}
40     with open(path, 'rb') as f:
41         num = struct.unpack('<Q', f.read(8))[0]
42         for _ in range(num):
43             iid = struct.unpack('<I', f.read(4))[0]
44             qvec = np.array(struct.unpack('<4d', f.read(32)))
45             tvec = np.array(struct.unpack('<3d', f.read(24)))
46             cid = struct.unpack('<I', f.read(4))[0]
47             name = b''
48             while True:
49                 c = f.read(1)
50                 if c == b'\x00': break
51                 name += c
52             name = name.decode()
53             n = struct.unpack('<Q', f.read(8))[0]
54             xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
55             pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
56             w, x, y, z = qvec
57             R = np.array([
58                 [1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w],
59                 [2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w],
60                 [2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*y*y]
61             ])
62             C = -R.T @ tvec
63             images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
64                             'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
65     return images
66
67 def read_points3d_bin(path):
68     pts = {}
69     with open(path, 'rb') as f:
70         num = struct.unpack('<Q', f.read(8))[0]
71         for _ in range(num):
72             pid = struct.unpack('<Q', f.read(8))[0]
73             xyz = np.array(struct.unpack('<3d', f.read(24)))
74             rgb = np.array(struct.unpack('<3B', f.read(3)))
75             err = struct.unpack('<d', f.read(8))[0]
76             n = struct.unpack('<Q', f.read(8))[0]
77             f.read(n * 8)
78             pts[pid] = xyz
79     return pts
80
81 def get_K(cam, scale=1.0):
82     p = cam['params']
83     if cam['model'] == 1:
84         K = np.array([[p[0]*scale, 0, p[2]*scale],
85                       [0, p[1]*scale, p[3]*scale],
86                       [0, 0, 1]], dtype=np.float64)
87     else:
88         K = np.array([[p[0]*scale, 0, p[1]*scale],
89                       [0, p[0]*scale, p[2]*scale],
90                       [0, 0, 1]], dtype=np.float64)
91     return K
92
93 def get_depth_range(img_info, pts3d, margin=0.3):
94     pt_ids = img_info['pt3d']
95     valid_ids = pt_ids[pt_ids >= 0]
96     if len(valid_ids) == 0:
97         return 1.0, 30.0
98     R = img_info['R']

```

```

99     t = img_info['tvec']
100     depths = []
101     for pid in valid_ids:
102         if pid in pts3d:
103             Xw = pts3d[pid]
104             Xc = R @ Xw + t
105             if Xc[2] > 0:
106                 depths.append(Xc[2])
107     if len(depths) < 5:
108         return 1.0, 30.0
109     depths = np.array(depths)
110     d_min = np.percentile(depths, 5) * (1 - margin)
111     d_max = np.percentile(depths, 95) * (1 + margin)
112     return max(0.05, d_min), d_max
113
114 def select_neighbors(ref_id, images, n=6):
115     ref = images[ref_id]
116     C_ref = ref['C']
117     R_ref = ref['R']
118     z_ref = R_ref[2, :]
119     cands = []
120     for iid, img in images.items():
121         if iid == ref_id:
122             continue
123         C_src = img['C']
124         dist = np.linalg.norm(C_src - C_ref)
125         if dist < 1e-6:
126             continue
127         z_src = img['R'][2, :]
128         cos_ang = np.clip(np.dot(z_ref, z_src), -1, 1)
129         score = dist * (2.0 - cos_ang)
130         cands.append((score, iid))
131     cands.sort()
132     return [iid for _, iid in cands[:n]]
133
134 def compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW):
135     ref_small = cv2.resize(ref_img, (sW, sH))
136     ref_t = torch.from_numpy(ref_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
137
138     K_ref_inv = torch.from_numpy(np.linalg.inv(K_ref_s)).float().to(DEVICE)
139
140     u = torch.arange(sW, device=DEVICE).float()
141     v = torch.arange(sH, device=DEVICE).float()
142     vv, uu = torch.meshgrid(v, u, indexing='ij')
143     pixels = torch.stack([uu, vv, torch.ones(sH, sW, device=DEVICE)], dim=-1).reshape(-1, 3)
144
145     depths = torch.linspace(d_min, d_max, N_DEPTHS, device=DEVICE)
146
147     cost_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)
148     count_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)
149
150     for src_img, K_src_s, R_ref, t_ref, R_src, t_src in src_data:
151         src_small = cv2.resize(src_img, (sW, sH))
152         src_t = torch.from_numpy(src_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
153         src_H, src_W = sH, sW
154
155         K_src_t = torch.from_numpy(K_src_s).float().to(DEVICE)
156         R_ref_t = torch.from_numpy(R_ref).float().to(DEVICE)
157         t_ref_t = torch.from_numpy(t_ref).float().to(DEVICE)
158         R_src_t = torch.from_numpy(R_src).float().to(DEVICE)
159         t_src_t = torch.from_numpy(t_src).float().to(DEVICE)
160
161         R_rel = R_src_t @ R_ref_t.T
162         t_rel = t_src_t - R_rel @ t_ref_t
163
164         A = K_src_t @ R_rel @ K_ref_inv
165         b_vec = K_src_t @ t_rel
166
167         for start in range(0, N_DEPTHS, DEPTH_CHUNK):
168             end = min(start + DEPTH_CHUNK, N_DEPTHS)
169             d_chunk = depths[start:end]
170             nc = len(d_chunk)
171
172             H_batch = A.unsqueeze(0).expand(nc, -1, -1).clone()
173             col_add = b_vec.unsqueeze(0) / d_chunk.unsqueeze(1)
174             H_batch = H_batch.clone()
175             H_batch[:, :, 2] = H_batch[:, :, 2] + col_add
176
177             proj = torch.einsum('nij,pj->nip', H_batch, pixels)
178
179             z_proj = proj[:, 2, :]
180             valid_z = z_proj > 0.01
181
182             proj_xy = proj[:, :, 2] / z_proj.unsqueeze(1).clamp(min=1e-6)
183             proj_grid = proj_xy.permute(0, 2, 1).reshape(nc, sH, sW, 2)
184
185             proj_grid_norm = proj_grid.clone()
186             proj_grid_norm[..., 0] = proj_grid[..., 0] / (src_W / 2) - 1
187             proj_grid_norm[..., 1] = proj_grid[..., 1] / (src_H / 2) - 1
188
189             in_bounds = ((proj_grid_norm[..., 0] >= -1) & (proj_grid_norm[..., 0] <= 1) &
190                          (proj_grid_norm[..., 1] >= -1) & (proj_grid_norm[..., 1] <= 1) &

```

```

191         valid_z.reshape(nc, sH, sW))
192
193     src_exp = src_t.expand(nc, -1, -1, -1)
194     warped = F.grid_sample(src_exp, proj_grid_norm, mode='bilinear',
195                             padding_mode='zeros', align_corners=False)
196
197     ref_exp = ref_t.expand(nc, -1, -1, -1)
198     diff = (ref_exp - warped).abs().mean(dim=1)
199
200     cost_vol[start:end] += diff * in_bounds.float()
201     count_vol[start:end] += in_bounds.float()
202
203     valid = count_vol > 0
204     avg_cost = torch.where(valid, cost_vol / count_vol.clamp(min=1),
205                             torch.full_like(cost_vol, 1e6))
206
207     best_idx = avg_cost.argmax(dim=0)
208     depth_map = depths[best_idx]
209     valid_mask = count_vol.max(dim=0)[0] > 1
210
211     return depth_map.cpu().numpy().astype(np.float32), valid_mask.cpu().numpy()
212
213 cameras = read_cameras_bin(SPARSE_DIR / 'cameras.bin')
214 images = read_images_bin(SPARSE_DIR / 'images.bin')
215 pts3d = read_points3d_bin(SPARSE_DIR / 'points3d.bin')
216
217 print(f'Cameras: {len(cameras)}, Images: {len(images)}, Sparse pts: {len(pts3d)}')
218
219 all_pts = []
220 all_colors = []
221
222 depth_maps = {}
223 valid_masks = {}
224 ref_poses = {}
225
226 for ref_id, ref_info in images.items():
227     img_path = IMAGES_DIR / ref_info['name']
228     ref_img = cv2.imread(str(img_path))
229     if ref_img is None:
230         print(f'Could not load {img_path}')
231         continue
232     ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)
233
234     H_orig, W_orig = ref_img.shape[:2]
235     sW = int(W_orig * IMG_SCALE)
236     sH = int(H_orig * IMG_SCALE)
237
238     K_ref = get_K(cameras[ref_info['cam_id']])
239     K_ref_s = get_K(cameras[ref_info['cam_id']], scale=IMG_SCALE)
240
241     d_min, d_max = get_depth_range(ref_info, pts3d)
242     print(f'\n[{ref_id}] {ref_info["name"]} depth=[{d_min:.2f},{d_max:.2f}]')
243
244     neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
245     print(f' neighbors: {neighbor_ids}')
246
247     src_data = []
248     for src_id in neighbor_ids:
249         src_info = images[src_id]
250         src_path = IMAGES_DIR / src_info['name']
251         src_img = cv2.imread(str(src_path))
252         if src_img is None:
253             continue
254         src_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
255         K_src_s = get_K(cameras[src_info['cam_id']], scale=IMG_SCALE)
256         src_data.append((src_img, K_src_s, ref_info['R'], ref_info['tvec'],
257                         src_info['R'], src_info['tvec']))
258
259     if not src_data:
260         print(f' No source images, skipping')
261         continue
262
263     depth_map, valid_mask = compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW)
264     print(f' valid pixels: {valid_mask.sum()} / {sH*sW}')
265
266     depth_maps[ref_id] = depth_map
267     valid_masks[ref_id] = valid_mask
268     ref_poses[ref_id] = (ref_img, K_ref, ref_info['R'], ref_info['tvec'], sH, sW, H_orig, W_orig)
269
270 print('\nApplying geometric consistency filtering...')
271 for ref_id, depth_map in depth_maps.items():
272     ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
273     valid_mask = valid_masks[ref_id].copy()
274     K_ref_s = K_ref.copy()
275     K_ref_s[0] *= IMG_SCALE
276     K_ref_s[1] *= IMG_SCALE
277     K_ref_s[0,2] *= IMG_SCALE
278     K_ref_s[1,2] *= IMG_SCALE
279
280     consistent = np.zeros(valid_mask.shape, dtype=np.int32)
281     neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
282

```

```

283     for src_id in neighbor_ids:
284         if src_id not in depth_maps:
285             continue
286         src_depth = depth_maps[src_id]
287         _, _, R_src, t_src, sH_s, sW_s, _, _ = ref_poses[src_id]
288         K_src_s = get_K(cameras[images[src_id]['cam_id']], scale=IMG_SCALE)
289
290         v_idx, u_idx = np.where(valid_mask)
291         if len(v_idx) == 0:
292             continue
293
294         d_vals = depth_map[v_idx, u_idx]
295         u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
296         v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]
297
298         Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
299         Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
300         Xc_src = (R_src @ Xw.T + t_src.reshape(3,1)).T
301
302         valid_z = Xc_src[:, 2] > 0
303         u_src = K_src_s[0,0] * Xc_src[:,0] / Xc_src[:,2].clip(1e-6) + K_src_s[0,2]
304         v_src = K_src_s[1,1] * Xc_src[:,1] / Xc_src[:,2].clip(1e-6) + K_src_s[1,2]
305
306         u_src_i = np.round(u_src).astype(int)
307         v_src_i = np.round(v_src).astype(int)
308
309         in_img = (u_src_i >= 0) & (u_src_i < sW_s) & (v_src_i >= 0) & (v_src_i < sH_s)
310         valid_proj = valid_z & in_img
311
312         for idx in np.where(valid_proj)[0]:
313             d_src = src_depth[v_src_i[idx], u_src_i[idx]]
314             if d_src > 0:
315                 d_reproj = Xc_src[idx, 2]
316                 if abs(d_reproj - d_src) / (d_src + 1e-6) < 0.1:
317                     consistent[v_idx[idx], u_idx[idx]] += 1
318
319         final_mask = valid_mask & (consistent >= 2)
320         print(f' [{ref_id}] consistency: {valid_mask.sum()} -> {final_mask.sum()} pts')
321         valid_masks[ref_id] = final_mask
322
323     print('\nBackprojecting depth maps to 3D...')
324     for ref_id, depth_map in depth_maps.items():
325         ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
326         valid_mask = valid_masks[ref_id]
327
328         K_ref_s = K_ref.copy()
329         K_ref_s[0] *= IMG_SCALE
330         K_ref_s[1] *= IMG_SCALE
331         K_ref_s[0,2] *= IMG_SCALE
332         K_ref_s[1,2] *= IMG_SCALE
333
334         v_idx, u_idx = np.where(valid_mask)
335         if len(v_idx) == 0:
336             continue
337
338         d_vals = depth_map[v_idx, u_idx].astype(np.float64)
339         u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
340         v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]
341
342         Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
343         Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
344
345         ref_img_small = cv2.resize(ref_img, (sW, sH))
346         colors = ref_img_small[v_idx, u_idx].astype(np.float32) / 255.0
347
348         all_pts.append(Xw.astype(np.float32))
349         all_colors.append(colors)
350         print(f' [{ref_id}] contributed {len(Xw)} pts')
351
352     pts_all = np.concatenate(all_pts, axis=0)
353     colors_all = np.concatenate(all_colors, axis=0)
354     print(f'\nTotal merged pts before filtering: {len(pts_all):,}')
355
356     pcd = o3d.geometry.PointCloud()
357     pcd.points = o3d.utility.Vector3dVector(pts_all)
358     pcd.colors = o3d.utility.Vector3dVector(colors_all)
359
360     print('Statistical outlier removal...')
361     pcd, ind = pcd.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
362     print(f'After outlier removal: {len(pcd.points):,}')
363
364     voxel_size = 0.0015
365     pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
366     print(f'After voxel downsampling (voxel={voxel_size}): {len(pcd_down.points):,}')
367
368     if len(pcd_down.points) < 500000:
369         voxel_size = 0.0010
370         pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
371         print(f'Retry voxel={voxel_size}: {len(pcd_down.points):,}')
372
373     print(f'Saving fused.ply ({len(pcd_down.points):,} points)...')
374     o3d.io.write_point_cloud(str(OUT_PLY), pcd_down)

```

```

375 print(f'Saved: {OUT_PLY}')
376
377 print('\nEstimating normals...')
378 pcd_down.estimate_normals(
379     search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.01, max_nn=30))
380 pcd_down.orient_normals_consistent_tangent_plane(100)
381
382 print('Poisson reconstruction (depth=10)...')
383 mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=10)
384 print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
385
386 densities = np.asarray(densities)
387 density_thresh = np.percentile(densities, 5)
388 vertices_to_remove = densities < density_thresh
389 mesh.remove_vertices_by_mask(vertices_to_remove)
390 print(f'Cleaned mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
391
392 if len(mesh.vertices) < 100000:
393     print('WARNING: mesh has fewer than 100k vertices, retrying with depth=9')
394     mesh2, densities2 = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=9)
395     densities2 = np.asarray(densities2)
396     thresh2 = np.percentile(densities2, 3)
397     mesh2.remove_vertices_by_mask(np.asarray(densities2) < thresh2)
398     print(f'depth=9 mesh: {len(mesh2.vertices):,} vertices, {len(mesh2.triangles):,} faces')
399     if len(mesh2.vertices) > len(mesh.vertices):
400         mesh = mesh2
401
402 mesh.compute_vertex_normals()
403 o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
404 print(f'\nSaved mesh: {OUT_MESH}')
405 print(f'Final vertices: {len(mesh.vertices):,} faces: {len(mesh.triangles):,}')
406 print(f'fused.ply size: {OUT_PLY.stat().st_size / 1e6:.1f} MB')
407 print(f'meshed-poisson.ply size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')
408 print('\n===== STAGE: DENSE RECONSTRUCTION N=24 COMPLETE =====')

```

Listing 3: colmap_n24/dense/mvs_dense.py

B.1.4 preprocess.py

```

1 from PIL import Image
2 from pathlib import Path
3
4 src = Path.home() / "ses598-apollo17" / "data" / "raw"
5 dst = Path.home() / "ses598-apollo17" / "data" / "rgb"
6 dst.mkdir(parents=True, exist_ok=True)
7
8 for p in sorted(src.glob("*.png")):
9     img = Image.open(p)
10     if img.mode != "RGB":
11         img = img.convert("RGB")
12     img.save(dst / p.name)
13     print(f"{p.name}: mode={img.mode}, size={img.size}")

```

Listing 4: preprocess.py

B.1.5 refine_mesh.py

```

1 import numpy as np
2 import open3d as o3d
3 from pathlib import Path
4
5 DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense')
6 OUT_PLY = DENSE_DIR / 'fused.ply'
7 OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'
8
9 pcd_raw = o3d.io.read_point_cloud(str(OUT_PLY))
10 pts = np.asarray(pcd_raw.points)
11 colors = np.asarray(pcd_raw.colors)
12
13 print(f'Raw cloud: {len(pts):,} points')
14 print(f' X: [{pts[:,0].min():.2f}, {pts[:,0].max():.2f}]')
15 print(f' Y: [{pts[:,1].min():.2f}, {pts[:,1].max():.2f}]')
16 print(f' Z: [{pts[:,2].min():.2f}, {pts[:,2].max():.2f}]')
17
18 x_min, x_max = -7.5, 1.0
19 y_min, y_max = -2.5, 4.5
20 z_min, z_max = 0.5, 11.0
21
22 mask = ((pts[:,0] >= x_min) & (pts[:,0] <= x_max) &
23         (pts[:,1] >= y_min) & (pts[:,1] <= y_max) &
24         (pts[:,2] >= z_min) & (pts[:,2] <= z_max))
25
26 pts_crop = pts[mask]

```

```

27 colors_crop = colors[mask]
28 print(f'\nAfter crop [{x_min},{x_max}]x[{y_min},{y_max}]x[{z_min},{z_max}]:')
29 print(f' {len(pts_crop):,} points ({100*mask.mean():.1f}%)')
30
31 pcd_crop = o3d.geometry.PointCloud()
32 pcd_crop.points = o3d.utility.Vector3dVector(pts_crop)
33 pcd_crop.colors = o3d.utility.Vector3dVector(colors_crop)
34
35 print('\nStatistical outlier removal on cropped cloud...')
36 pcd_crop, _ = pcd_crop.remove_statistical_outlier(nb_neighbors=30, std_ratio=1.5)
37 print(f'After outlier removal: {len(pcd_crop.points):,}')
38
39 print('Saving updated fused.ply...')
40 o3d.io.write_point_cloud(str(OUT_PLY), pcd_crop)
41 print(f'Saved fused.ply: {len(pcd_crop.points):,} points')
42
43 print('\nEstimating normals...')
44 pcd_crop.estimate_normals(
45     search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.05, max_nn=50))
46 pcd_crop.orient_normals_consistent_tangent_plane(100)
47
48 for depth in [11, 10, 9]:
49     print(f'\nPoisson reconstruction depth={depth}...')
50     mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_crop, depth=depth)
51     print(f' Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
52     densities_np = np.asarray(densities)
53     thresh_pct = 5
54     thresh = np.percentile(densities_np, thresh_pct)
55     mesh.remove_vertices_by_mask(densities_np < thresh)
56     print(f' After pruning p{thresh_pct}: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
57     if len(mesh.vertices) >= 100000:
58         break
59
60 mesh.compute_vertex_normals()
61 o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
62
63 ply_size = OUT_PLY.stat().st_size
64 mesh_size = OUT_MESH.stat().st_size
65 print(f'\nFinal fused.ply: {len(pcd_crop.points):,} points, {ply_size/1e6:.1f} MB')
66 print(f'Final meshed-poisson.ply: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces, {mesh_size/1e6:.1f} MB')

```

Listing 5: refine_mesh.py

B.1.6 remesh_n24.py

```

1 import numpy as np
2 import open3d as o3d
3 from pathlib import Path
4
5 IN_PLY = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/fused.ply')
6 OUT_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')
7
8 print('Loading fused.ply...')
9 pcd = o3d.io.read_point_cloud(str(IN_PLY))
10 pts = np.asarray(pcd.points)
11 colors = np.asarray(pcd.colors)
12 print(f'Total points: {len(pts):,}')
13 print(f'Bounding box: min={pts.min(0).round(3)}, max={pts.max(0).round(3)}')
14
15 c137 = np.array([-3.07, -0.12, -1.70])
16 c138 = np.array([4.13, 0.16, 3.10])
17 d137 = np.linalg.norm(pts - c137, axis=1)
18 d138 = np.linalg.norm(pts - c138, axis=1)
19 d_min = np.minimum(d137, d138)
20
21 mask = d_min < 10.0
22 pts_clean = pts[mask]
23 colors_clean = colors[mask]
24 print(f'After cluster-proximity filter (<10 units): {len(pts_clean):,} points')
25
26 pcd_clean = o3d.geometry.PointCloud()
27 pcd_clean.points = o3d.utility.Vector3dVector(pts_clean)
28 pcd_clean.colors = o3d.utility.Vector3dVector(colors_clean)
29
30 print('Statistical outlier removal on filtered cloud...')
31 pcd_clean, _ = pcd_clean.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
32 print(f'After outlier removal: {len(pcd_clean.points):,}')
33
34 print('Estimating normals...')
35 pcd_clean.estimate_normals(
36     search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))
37 pcd_clean.orient_normals_consistent_tangent_plane(100)
38
39 for depth in [10, 11]:
40     print(f'\nPoisson reconstruction (depth={depth})...')
41     mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_clean, depth=depth)
42     print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')

```

```

43 densities = np.asarray(densities)
44 thresh = np.percentile(densities, 3)
45 mesh.remove_vertices_by_mask(densities < thresh)
46 print(f'Cleaved mesh (3% density threshold): {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
47 if len(mesh.vertices) >= 100000:
48     print(f'Target met at depth={depth}')
49     break
50
51 mesh.compute_vertex_normals()
52 o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
53 print(f'\nSaved: {OUT_MESH}')
54 print(f'Final vertices: {len(mesh.vertices):,} faces: {len(mesh.triangles):,}')
55 print(f'File size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')
56 print('\n===== REMESH COMPLETE =====')

```

Listing 6: remesh_n24.py

B.2 Group B: Comparison and Evaluation

B.2.1 check_registered.py

```

1 from pathlib import Path
2
3 base = Path.home() / "ses598-apollo17"
4 rgb_root = base / "data" / "rgb"
5 all_images = sorted(str(p.relative_to(rgb_root)) for p in rgb_root.rglob("*.png"))
6
7 images_txt = base / "colmap_n15" / "sparse" / "0_text" / "images.txt"
8 registered = set()
9 with open(images_txt) as f:
10     for line in f:
11         line = line.strip()
12         if line.startswith("#") or not line:
13             continue
14         parts = line.split()
15         if len(parts) >= 10 and parts[-1].endswith(".png"):
16             registered.add(parts[-1])
17
18 print(f"Total images: {len(all_images)}")
19 print(f"Registered: {len(registered)}")
20 print()
21 print("Registered:")
22 for img in sorted(registered):
23     print(f" {img}")
24 print()
25 print("NOT registered:")
26 missing = set(all_images) - registered
27 for img in sorted(missing):
28     print(f" {img}")

```

Listing 7: check_registered.py

B.2.2 compare_21036.py

```

1 import cv2
2 import numpy as np
3 from pathlib import Path
4
5 RAW = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag138')
6
7 img35 = cv2.imread(str(RAW / 'AS17-138-21035HR.png'))
8 img36 = cv2.imread(str(RAW / 'AS17-138-21036HR.png'))
9 img37 = cv2.imread(str(RAW / 'AS17-138-21037HR.png'))
10
11 for img, name in [(img35, '21035'), (img36, '21036'), (img37, '21037')]:
12     print(f'{name}: shape={img.shape}, dtype={img.dtype}')
13
14 orb = cv2.ORB_create(nfeatures=4000)
15 bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
16
17 def match_pair(a, b, name_a, name_b):
18     kpa, da = orb.detectAndCompute(cv2.cvtColor(a, cv2.COLOR_BGR2GRAY), None)
19     kpb, db = orb.detectAndCompute(cv2.cvtColor(b, cv2.COLOR_BGR2GRAY), None)
20     if da is None or db is None or len(da)==0 or len(db)==0:
21         print(f'{name_a}<->{name_b}: no descriptors')
22         return 0
23     matches = bf.match(da, db)
24     good = [m for m in matches if m.distance < 60]
25     print(f'{name_a}<->{name_b}: total={len(matches)} good(dist<60)={len(good)}')
26     if len(good) >= 8:
27         pts_a = np.float32([kpa[m.queryIdx].pt for m in good])

```

```

28     pts_b = np.float32([kpb[m.trainIdx].pt for m in good])
29     _, mask = cv2.findHomography(pts_a, pts_b, cv2.RANSAC, 5.0)
30     inliers = int(mask.sum()) if mask is not None else 0
31     print(f' homography inliers: {inliers}')
32     return inliers
33     return 0
34
35 inliers_35_36 = match_pair(img35, img36, '21035', '21036')
36 inliers_36_37 = match_pair(img36, img37, '21036', '21037')
37 inliers_35_37 = match_pair(img35, img37, '21035', '21037')
38
39 print()
40 if inliers_35_36 + inliers_36_37 < 10:
41     print('CONCLUSION: 21036 has negligible overlap with neighbors - geometrically isolated, skip registration')
42 else:
43     print(f'CONCLUSION: 21036 has {inliers_35_36+inliers_36_37} combined inliers - may be recoverable')

```

Listing 8: compare_21036.py

B.2.3 compute_metrics.py

```

1 import json
2 import csv
3 import numpy as np
4 import cv2
5 from pathlib import Path
6 from skimage.metrics import peak_signal_noise_ratio, structural_similarity
7 import matplotlib
8 matplotlib.use('Agg')
9 import matplotlib.pyplot as plt
10
11 RENDERS_DIR = Path('/home/jatin-satyam/ses598-apollo17/renders_n15/train/rgb')
12 RGB_DIR = Path('/home/jatin-satyam/ses598-apollo17/data/rgb')
13 METRICS_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')
14 METRICS_DIR.mkdir(parents=True, exist_ok=True)
15
16 IMAGE_MAP = {
17     'AS17-137-20903HR': 'mag137/AS17-137-20903HR.png',
18     'AS17-137-20904HR': 'mag137/AS17-137-20904HR.png',
19     'AS17-137-20905HR': 'mag137/AS17-137-20905HR.png',
20     'AS17-137-20906HR': 'mag137/AS17-137-20906HR.png',
21     'AS17-137-20907HR': 'mag137/AS17-137-20907HR.png',
22     'AS17-137-20908HR': 'mag137/AS17-137-20908HR.png',
23     'AS17-137-20909HR': 'mag137/AS17-137-20909HR.png',
24     'AS17-138-21030HR': 'mag138/AS17-138-21030HR.png',
25     'AS17-138-21031HR': 'mag138/AS17-138-21031HR.png',
26     'AS17-138-21032HR': 'mag138/AS17-138-21032HR.png',
27     'AS17-138-21033HR': 'mag138/AS17-138-21033HR.png',
28     'AS17-138-21034HR': 'mag138/AS17-138-21034HR.png',
29     'AS17-138-21035HR': 'mag138/AS17-138-21035HR.png',
30     'AS17-138-21037HR': 'mag138/AS17-138-21037HR.png',
31 }
32
33 render_files = sorted(RENDERS_DIR.rglob('*.png')) + sorted(RENDERS_DIR.rglob('*.jpg')) + sorted(RENDERS_DIR.rglob('*.jpeg'))
34 print(f'Found {len(render_files)} renders in {RENDERS_DIR}')
35
36 rows = []
37 for render_path in render_files:
38     stem = render_path.stem
39     matched_key = None
40     for key in IMAGE_MAP:
41         if key in stem or stem in key:
42             matched_key = key
43             break
44     if matched_key is None:
45         for key in IMAGE_MAP:
46             short = key.split('-')[1].replace('HR', '')
47             if short in stem:
48                 matched_key = key
49                 break
50     if matched_key is None:
51         print(f' WARNING: no match for render {stem}')
52         continue
53     orig_path = RGB_DIR / IMAGE_MAP[matched_key]
54     if not orig_path.exists():
55         print(f' WARNING: original not found: {orig_path}')
56         continue
57
58     render_img = cv2.imread(str(render_path))
59     orig_img = cv2.imread(str(orig_path))
60     render_img = cv2.cvtColor(render_img, cv2.COLOR_BGR2RGB)
61     orig_img = cv2.cvtColor(orig_img, cv2.COLOR_BGR2RGB)
62
63     render_h, render_w = render_img.shape[:2]
64     orig_h, orig_w = orig_img.shape[:2]
65
66     if (render_h, render_w) != (orig_h, orig_w):

```



```

67 orig_resized = cv2.resize(orig_img, (render_w, render_h), interpolation=cv2.INTER_AREA)
68 resize_note = f'original downsampled {orig_w}x{orig_h} to {render_w}x{render_h}'
69 else:
70     orig_resized = orig_img
71     resize_note = 'same resolution'
72
73 psnr_val = peak_signal_noise_ratio(orig_resized, render_img, data_range=255)
74 ssim_val = structural_similarity(orig_resized, render_img, channel_axis=-1, data_range=255)
75
76 rows.append({
77     'image_name': matched_key,
78     'render_path': str(render_path),
79     'original_path': str(orig_path),
80     'render_resolution': f'{render_w}x{render_h}',
81     'original_resolution': f'{orig_w}x{orig_h}',
82     'resize_note': resize_note,
83     'psnr_db': round(float(psnr_val), 4),
84     'ssim': round(float(ssim_val), 6),
85 })
86 print(f' {matched_key}: PSNR={psnr_val:.2f}dB SSIM={ssim_val:.4f}')
87
88 if not rows:
89     print('ERROR: no rows computed, check renders directory')
90     exit(1)
91
92 csv_path = METRICS_DIR / 'per_image_metrics.csv'
93 fieldnames = ['image_name', 'render_path', 'original_path', 'render_resolution', 'original_resolution', 'resize_note', 'psnr_db', 'ssim']
94 with open(csv_path, 'w', newline='') as f:
95     writer = csv.DictWriter(f, fieldnames=fieldnames)
96     writer.writeheader()
97     writer.writerows(rows)
98 print(f'\nSaved: {csv_path} ({len(rows)} rows)')
99
100 psnr_vals = [r['psnr_db'] for r in rows]
101 ssim_vals = [r['ssim'] for r in rows]
102
103 agg = {
104     'n_images': len(rows),
105     'psnr_db': {
106         'mean': round(float(np.mean(psnr_vals)), 4),
107         'median': round(float(np.median(psnr_vals)), 4),
108         'min': round(float(np.min(psnr_vals)), 4),
109         'max': round(float(np.max(psnr_vals)), 4),
110         'std': round(float(np.std(psnr_vals)), 4),
111     },
112     'ssim': {
113         'mean': round(float(np.mean(ssim_vals)), 6),
114         'median': round(float(np.median(ssim_vals)), 6),
115         'min': round(float(np.min(ssim_vals)), 6),
116         'max': round(float(np.max(ssim_vals)), 6),
117         'std': round(float(np.std(ssim_vals)), 6),
118     }
119 }
120
121 agg_path = METRICS_DIR / 'aggregate.json'
122 with open(agg_path, 'w') as f:
123     json.dump(agg, f, indent=2)
124 print(f'Saved: {agg_path}')
125 print(f'Mean PSNR: {agg["psnr_db"]["mean"]:.2f} dB Mean SSIM: {agg["ssim"]["mean"]:.4f}')
126
127 names = [r['image_name'].replace('AS17-', '').replace('HR', '') for r in rows]
128 psnr_arr = np.array(psnr_vals)
129 ssim_arr = np.array(ssim_vals)
130
131 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
132 fig.suptitle('Per Image Gaussian Splatting Reconstruction Quality', fontsize=14, fontweight='bold')
133
134 x = np.arange(len(names))
135 bars1 = ax1.bar(x, psnr_arr, color='steelblue', edgecolor='midnightblue', linewidth=0.5)
136 ax1.axhline(y=agg['psnr_db']['mean'], color='goldenrod', linestyle='--', linewidth=1.5,
137             label=f'Mean {agg["psnr_db"]["mean"]:.2f} dB')
138 ax1.set_ylabel('PSNR (dB)', fontsize=11)
139 ax1.set_title('Peak Signal to Noise Ratio per Training View', fontsize=12)
140 ax1.set_xticks(x)
141 ax1.set_xticklabels(names, rotation=45, ha='right', fontsize=9)
142 ax1.legend(fontsize=10)
143 ax1.set_ylim(0, max(psnr_arr) * 1.18)
144 for bar, val in zip(bars1, psnr_arr):
145     ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
146             f'{val:.1f}', ha='center', va='bottom', fontsize=8)
147
148 bars2 = ax2.bar(x, ssim_arr, color='mediumvioletred', edgecolor='darkred', linewidth=0.5)
149 ax2.axhline(y=agg['ssim']['mean'], color='goldenrod', linestyle='--', linewidth=1.5,
150             label=f'Mean {agg["ssim"]["mean"]:.4f}')
151 ax2.set_ylabel('SSIM', fontsize=11)
152 ax2.set_title('Structural Similarity Index per Training View', fontsize=12)
153 ax2.set_xticks(x)
154 ax2.set_xticklabels(names, rotation=45, ha='right', fontsize=9)
155 ax2.legend(fontsize=10)
156 ax2.set_ylim(0, 1.05)
157 for bar, val in zip(bars2, ssim_arr):
158     ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,

```

```

159         f'{val:.3f}', ha='center', va='bottom', fontsize=8)
160
161 plt.tight_layout()
162 plot_path = METRICS_DIR / 'per_image_metrics.png'
163 plt.savefig(str(plot_path), dpi=150, bbox_inches='tight')
164 plt.close()
165 print(f'Saved: {plot_path}')
166
167 rows_sorted = sorted(rows, key=lambda r: r['psnr_db'])
168 print(f'\nWorst PSNR: {rows_sorted[0]["image_name"]} = {rows_sorted[0]["psnr_db"]:.2f} dB (SSIM {rows_sorted[0]["ssim"]:.4f})')
169 print(f'Best PSNR: {rows_sorted[-1]["image_name"]} = {rows_sorted[-1]["psnr_db"]:.2f} dB (SSIM {rows_sorted[-1]["ssim"]:.4f})')
170 median_idx = len(rows_sorted) // 2
171 print(f'Median PSNR: {rows_sorted[median_idx]["image_name"]} = {rows_sorted[median_idx]["psnr_db"]:.2f} dB')

```

Listing 9: compute_metrics.py

B.2.4 mesh_compare.py

```

1 import numpy as np
2 import open3d as o3d
3 import json
4 import struct
5 from pathlib import Path
6 import matplotlib
7 matplotlib.use('Agg')
8 import matplotlib.pyplot as plt
9 import matplotlib.cm as cm
10
11 N14_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/meshed-poisson.ply')
12 N24_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')
13 OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/comparison')
14 OUT_DIR.mkdir(parents=True, exist_ok=True)
15
16 N_SAMPLE = 100000
17 VOXEL_REG = 0.05
18
19 print('Loading meshes...')
20 mesh14 = o3d.io.read_triangle_mesh(str(N14_MESH))
21 mesh24 = o3d.io.read_triangle_mesh(str(N24_MESH))
22 print(f'N=14 mesh: {len(mesh14.vertices):,} vertices, {len(mesh14.triangles):,} faces')
23 print(f'N=24 mesh: {len(mesh24.vertices):,} vertices, {len(mesh24.triangles):,} faces')
24
25 print('Sampling point clouds...')
26 pcd14 = mesh14.sample_points_uniformly(number_of_points=N_SAMPLE)
27 pcd24 = mesh24.sample_points_uniformly(number_of_points=N_SAMPLE)
28
29 print('Computing FPFH features for global registration...')
30 pcd14_d = pcd14.voxel_down_sample(VOXEL_REG)
31 pcd24_d = pcd24.voxel_down_sample(VOXEL_REG)
32
33 pcd14_d.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*2, max_nn=30))
34 pcd24_d.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*2, max_nn=30))
35
36 fpfh14 = o3d.pipelines.registration.compute_fpfh_feature(
37     pcd14_d, o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*5, max_nn=100))
38 fpfh24 = o3d.pipelines.registration.compute_fpfh_feature(
39     pcd24_d, o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*5, max_nn=100))
40
41 print('RANSAC global registration...')
42 result_ransac = o3d.pipelines.registration.registration_ransac_based_on_feature_matching(
43     pcd24_d, pcd14_d, fpfh24, fpfh14,
44     mutual_filter=True,
45     max_correspondence_distance=VOXEL_REG*2,
46     estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPoint(False),
47     ransac_n=3,
48     checkers=[
49         o3d.pipelines.registration.CorrespondenceCheckerBasedOnEdgeLength(0.9),
50         o3d.pipelines.registration.CorrespondenceCheckerBasedOnDistance(VOXEL_REG*2)
51     ],
52     criteria=o3d.pipelines.registration.RANSACConvergenceCriteria(100000, 0.999)
53 )
54 print(f'RANSAC fitness: {result_ransac.fitness:.4f}, inlier rmse: {result_ransac.inlier_rmse:.4f}')
55
56 print('ICP refinement...')
57 pcd14_fine = pcd14.voxel_down_sample(0.01)
58 pcd24_fine = pcd24.voxel_down_sample(0.01)
59 pcd14_fine.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))
60 pcd24_fine.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))
61
62 result_icp = o3d.pipelines.registration.registration_icp(
63     pcd24_fine, pcd14_fine,
64     max_correspondence_distance=0.02,
65     init=result_ransac.transformation,
66     estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPlane(),
67     criteria=o3d.pipelines.registration.ICPConvergenceCriteria(max_iteration=100)
68 )
69 T_icp = result_icp.transformation

```

```

70 print(f'ICP fitness: {result_icp.fitness:.4f}, inlier rmse: {result_icp.inlier_rmse:.6f}')
71 print(f'ICP transform:\n{T_icp}')
72
73 pcd24_aligned = pcd24
74 pcd24_aligned_t = o3d.geometry.PointCloud(pcd24)
75 pcd24_aligned_t.transform(T_icp)
76
77 pts14 = np.asarray(pcd14.points)
78 pts24 = np.asarray(pcd24_aligned_t.points)
79
80 print('Computing nearest-neighbor distances...')
81 tree14 = o3d.geometry.KDTreeFlann(pcd14)
82 tree24 = o3d.geometry.KDTreeFlann(pcd24_aligned_t)
83
84 dists_24to14 = np.array([np.sqrt(tree14.search_knn_vector_3d(p, 1)[2][0]) for p in pts24])
85 dists_14to24 = np.array([np.sqrt(tree24.search_knn_vector_3d(p, 1)[2][0]) for p in pts14])
86
87 chamfer = float((dists_24to14.mean() + dists_14to24.mean()) / 2)
88 hausdorff_p99 = float(np.percentile(np.maximum(dists_24to14, dists_14to24[:len(dists_24to14)]), 99))
89
90 pts_all = np.concatenate([pts14, pts24])
91 bbox_diag = float(np.linalg.norm(pts_all.max(0) - pts_all.min(0)))
92
93 def fscore(d1, d2, thresh):
94     p = (d2 < thresh).mean()
95     r = (d1 < thresh).mean()
96     if p + r == 0:
97         return 0.0
98     return float(2 * p * r / (p + r))
99
100 f_scores = {}
101 for t_frac in [0.01, 0.05, 0.1]:
102     t_abs = bbox_diag * t_frac
103     f_scores[f'f_score_at_{t_frac}'] = fscore(dists_14to24, dists_24to14, t_abs)
104     f_scores[f'threshold_abs_{t_frac}'] = t_abs
105
106 metrics = {
107     'chamfer_distance': chamfer,
108     'hausdorff_p99': hausdorff_p99,
109     'bbox_diagonal': bbox_diag,
110     'icp_fitness': float(result_icp.fitness),
111     'icp_inlier_rmse': float(result_icp.inlier_rmse),
112     'icp_transform': T_icp.tolist(),
113     'n14_vertices': len(mesh14.vertices),
114     'n14_faces': len(mesh14.triangles),
115     'n24_vertices': len(mesh24.vertices),
116     'n24_faces': len(mesh24.triangles),
117 }
118 metrics.update(f_scores)
119
120 with open(OUT_DIR / 'quantitative_metrics.json', 'w') as f:
121     json.dump(metrics, f, indent=2)
122 print(f'Saved: {OUT_DIR / "quantitative_metrics.json"}')
123 print(f'Chamfer: {chamfer:.6f}')
124 print(f'Hausdorff p99: {hausdorff_p99:.6f}')
125 for t in [0.01, 0.05, 0.1]:
126     print(f'F-score @{t}: {f_scores[f"f_score_at_{t}"]:.4f}')
127
128 print('\nGenerating per-vertex distance heatmap...')
129 mesh24_aligned = o3d.io.read_triangle_mesh(str(N24_MESH))
130 mesh24_aligned.transform(T_icp)
131 mesh24_aligned.compute_vertex_normals()
132
133 verts24 = np.asarray(mesh24_aligned.vertices)
134 pcd_vert = o3d.geometry.PointCloud()
135 pcd_vert.points = o3d.utility.Vector3dVector(verts24)
136 tree14_full = o3d.geometry.KDTreeFlann(pcd14)
137
138 vert_dists = np.array([np.sqrt(tree14_full.search_knn_vector_3d(v, 1)[2][0]) for v in verts24])
139
140 d_min_v, d_max_v = np.percentile(vert_dists, 2), np.percentile(vert_dists, 98)
141 norm_dists = np.clip((vert_dists - d_min_v) / (d_max_v - d_min_v + 1e-10), 0, 1)
142
143 cmap = cm.get_cmap('coolwarm')
144 colors_rgb = cmap(norm_dists)[: , :3]
145
146 mesh24_colored = o3d.geometry.TriangleMesh()
147 mesh24_colored.vertices = mesh24_aligned.vertices
148 mesh24_colored.triangles = mesh24_aligned.triangles
149 mesh24_colored.vertex_colors = o3d.utility.Vector3dVector(colors_rgb)
150 mesh24_colored.compute_vertex_normals()
151
152 ply_path = OUT_DIR / 'per_vertex_distance.ply'
153 o3d.io.write_triangle_mesh(str(ply_path), mesh24_colored)
154 print(f'Saved: {ply_path}')
155
156 fig, ax = plt.subplots(1, 1, figsize=(10, 7))
157 sc = ax.scatter(verts24[:, 0], verts24[:, 2], c=vert_dists, cmap='coolwarm',
158               s=0.5, vmin=d_min_v, vmax=d_max_v, alpha=0.6)
159 plt.colorbar(sc, ax=ax, label='Distance to N=14 mesh')
160 ax.set_title('Per-vertex Distance: N=24 mesh to N=14 baseline (top-down view)', fontsize=12)
161 ax.set_xlabel('X (COLMAP world)')

```

```

162 ax.set_ylabel('Z (COLMAP world)')
163 ax.set_aspect('equal')
164 fig.tight_layout()
165 png_path = OUT_DIR / 'per_vertex_distance.png'
166 plt.savefig(png_path, dpi=120, bbox_inches='tight')
167 plt.close()
168 print(f'Saved: {png_path}')
169
170 print('\n===== STAGE: MESH COMPARISON COMPLETE =====')

```

Listing 10: mesh_compare.py

B.2.5 qualitative_compare.py

```

1 import numpy as np
2 import open3d as o3d
3 import json
4 import struct
5 import cv2
6 from pathlib import Path
7 import matplotlib
8 matplotlib.use('Agg')
9 import matplotlib.pyplot as plt
10 from PIL import Image
11
12 SPARSE14 = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/sparse')
13 N14_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/meshed-poisson.ply')
14 N24_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')
15 IMAGES14 = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/images')
16 METRICS_JSON = Path('/home/jatin-satyam/ses598-apollo17/comparison/quantitative_metrics.json')
17 OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/comparison')
18
19 with open(METRICS_JSON) as f:
20     metrics = json.load(f)
21 T_icp = np.array(metrics['icp_transform'])
22
23 def read_cameras_bin(path):
24     cameras = {}
25     nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
26     with open(path, 'rb') as f:
27         num = struct.unpack('<Q', f.read(8))[0]
28         for _ in range(num):
29             cid = struct.unpack('<I', f.read(4))[0]
30             model = struct.unpack('<I', f.read(4))[0]
31             w = struct.unpack('<Q', f.read(8))[0]
32             h = struct.unpack('<Q', f.read(8))[0]
33             n = nparams.get(model, 4)
34             params = list(struct.unpack(f'<{n}d', f.read(8*n)))
35             cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
36     return cameras
37
38 def read_images_bin(path):
39     images = {}
40     with open(path, 'rb') as f:
41         num = struct.unpack('<Q', f.read(8))[0]
42         for _ in range(num):
43             iid = struct.unpack('<I', f.read(4))[0]
44             qvec = np.array(struct.unpack('<4d', f.read(32)))
45             tvec = np.array(struct.unpack('<3d', f.read(24)))
46             cid = struct.unpack('<I', f.read(4))[0]
47             name = b''
48             while True:
49                 c = f.read(1)
50                 if c == b'\x00': break
51                 name += c
52             name = name.decode()
53             n = struct.unpack('<Q', f.read(8))[0]
54             xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
55             pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
56             w_q, x_q, y_q, z_q = qvec
57             R = np.array([
58                 [1-2*y_q*y_q-2*z_q*z_q, 2*x_q*y_q-2*z_q*w_q, 2*x_q*z_q+2*y_q*w_q],
59                 [2*x_q*y_q+2*z_q*w_q, 1-2*x_q*x_q-2*z_q*z_q, 2*y_q*z_q-2*x_q*w_q],
60                 [2*x_q*z_q-2*y_q*w_q, 2*y_q*z_q+2*x_q*w_q, 1-2*x_q*x_q-2*y_q*y_q]
61             ])
62             C = -R.T @ tvec
63             images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
64                             'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
65     return images
66
67 def read_points3d_bin(path):
68     pts = {}
69     with open(path, 'rb') as f:
70         num = struct.unpack('<Q', f.read(8))[0]
71         for _ in range(num):
72             pid = struct.unpack('<Q', f.read(8))[0]
73             xyz = np.array(struct.unpack('<3d', f.read(24)))

```

```

74         rgb = np.array(struct.unpack('<3B', f.read(3)))
75         err = struct.unpack('<d', f.read(8))[0]
76         n = struct.unpack('<Q', f.read(8))[0]
77         f.read(n * 8)
78         pts[pid] = (xyz, err)
79     return pts
80
81 cameras = read_cameras_bin(SPARSE14 / 'cameras.bin')
82 images = read_images_bin(SPARSE14 / 'images.bin')
83 pts3d = read_points3d_bin(SPARSE14 / 'points3D.bin')
84
85 def compute_reproj_error(img_info):
86     cam = cameras[img_info['cam_id']]
87     p = cam['params']
88     if cam['model'] == 1:
89         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
90     else:
91         fx = fy = p[0]; cx, cy = p[1], p[2]
92     R = img_info['R']
93     t = img_info['tvec']
94     errors = []
95     for i, pid in enumerate(img_info['pt3d']):
96         if pid < 0 or pid not in pts3d:
97             continue
98         xyz, _ = pts3d[pid]
99         Xc = R @ xyz + t
100         if Xc[2] <= 0:
101             continue
102         u_proj = fx * Xc[0] / Xc[2] + cx
103         v_proj = fy * Xc[1] / Xc[2] + cy
104         u_obs, v_obs = img_info['xys'][i]
105         errors.append(np.sqrt((u_proj - u_obs)**2 + (v_proj - v_obs)**2))
106     return float(np.mean(errors)) if errors else float('inf')
107
108 img_errors = {}
109 for iid, img_info in images.items():
110     err = compute_reproj_error(img_info)
111     img_errors[iid] = err
112     print(f' {img_info["name"]}: reproj_error={err:.3f}px')
113
114 sorted_imgs = sorted(img_errors.items(), key=lambda x: x[1])
115 n_imgs = len(sorted_imgs)
116 pick_indices = [n_imgs//5, n_imgs*2//5, n_imgs*3//5, n_imgs*4//5]
117 chosen_ids = [sorted_imgs[i][0] for i in pick_indices]
118 print(f'\nChosen cameras (near-median reprojection error):')
119 for iid in chosen_ids:
120     print(f' {img_info[iid]: {img_info["name"]} error={img_errors[iid]:.3f}px')
121
122 print('\nLoading meshes...')
123 mesh14 = o3d.io.read_triangle_mesh(str(N14_MESH))
124 mesh14.compute_vertex_normals()
125 mesh24 = o3d.io.read_triangle_mesh(str(N24_MESH))
126 mesh24.transform(T_icp)
127 mesh24.compute_vertex_normals()
128
129 def render_mesh_matplotlib(mesh, R_cam, t_cam, cam_info, W=800, H=800, title=''):
130     verts = np.asarray(mesh.vertices)
131     colors_v = np.asarray(mesh.vertex_colors) if mesh.has_vertex_colors() else None
132
133     Xc = (R_cam @ verts.T + t_cam.reshape(3,1)).T
134     mask = Xc[:, 2] > 0.1
135
136     p = cam_info['params']
137     if cam_info['model'] == 1:
138         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
139     else:
140         fx = fy = p[0]; cx, cy = p[1], p[2]
141
142     scale_x = W / cam_info['w']
143     scale_y = H / cam_info['h']
144
145     u = (fx * Xc[:,0] / Xc[:,2].clip(1e-6) + cx) * scale_x
146     v = (fy * Xc[:,1] / Xc[:,2].clip(1e-6) + cy) * scale_y
147
148     in_img = mask & (u >= 0) & (u < W) & (v >= 0) & (v < H)
149
150     img = np.ones((H, W, 3), dtype=np.float32) * 0.95
151
152     depths = Xc[:, 2]
153     d_min, d_max = depths[in_img].min() if in_img.any() else 1.0, depths[in_img].max() if in_img.any() else 10.0
154
155     valid_u = u[in_img].astype(int).clip(0, W-1)
156     valid_v = v[in_img].astype(int).clip(0, H-1)
157     valid_d = depths[in_img]
158
159     if colors_v is not None:
160         valid_c = colors_v[in_img]
161     else:
162         norm_d = (valid_d - d_min) / (d_max - d_min + 1e-10)
163         valid_c = np.stack([norm_d, norm_d, norm_d], axis=1)
164
165     sort_order = np.argsort(-valid_d)

```

```

166     for idx in sort_order:
167         uu, vv = valid_u[idx], valid_v[idx]
168         img[vv, uu] = valid_c[idx]
169
170     img_uint8 = (img * 255).clip(0, 255).astype(np.uint8)
171     return img_uint8
172
173 sidebyside_paths = []
174 for view_idx, iid in enumerate(chosen_ids, start=1):
175     img_info = images[iid]
176     cam = cameras[img_info['cam_id']]
177     R_cam = img_info['R']
178     t_cam = img_info['tvec']
179     img_name = img_info['name']
180
181     print(f'\nRendering view {view_idx}: {img_name}')
182
183     render14 = render_mesh_matplotlib(mesh14, R_cam, t_cam, cam)
184     render24 = render_mesh_matplotlib(mesh24, R_cam, t_cam, cam)
185
186     r14_path = OUT_DIR / f'render_n14_view{view_idx}.png'
187     r24_path = OUT_DIR / f'render_n24_view{view_idx}.png'
188     Image.fromarray(render14).save(str(r14_path))
189     Image.fromarray(render24).save(str(r24_path))
190     print(f'  Saved: {r14_path.name}, {r24_path.name}')
191
192     photo_name = img_name
193     photo_path = IMAGES14 / photo_name
194     has_photo = photo_path.exists()
195
196     if has_photo:
197         photo = cv2.imread(str(photo_path))
198         photo = cv2.cvtColor(photo, cv2.COLOR_BGR2RGB)
199         photo_resized = cv2.resize(photo, (800, 800))
200         fig, axes = plt.subplots(1, 3, figsize=(24, 8))
201         axes[0].imshow(render14)
202         axes[0].set_title(f'N=14 Mesh', fontsize=13, color='steelblue')
203         axes[0].axis('off')
204         axes[1].imshow(photo_resized)
205         axes[1].set_title(f'Original Photo: {Path(photo_name).name}', fontsize=13, color='goldenrod')
206         axes[1].axis('off')
207         axes[2].imshow(render24)
208         axes[2].set_title(f'N=24 Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
209         axes[2].axis('off')
210     else:
211         fig, axes = plt.subplots(1, 2, figsize=(16, 8))
212         axes[0].imshow(render14)
213         axes[0].set_title(f'N=14 Mesh', fontsize=13, color='steelblue')
214         axes[0].axis('off')
215         axes[1].imshow(render24)
216         axes[1].set_title(f'N=24 Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
217         axes[1].axis('off')
218
219     fig.suptitle(f'View {view_idx}: {Path(photo_name).stem} reproj_err={img_errors[iid]:.2f}px', fontsize=14)
220     fig.tight_layout()
221     sbs_path = OUT_DIR / f'sidebyside_view{view_idx}.png'
222     plt.savefig(str(sbs_path), dpi=100, bbox_inches='tight')
223     plt.close()
224     sidebyside_paths.append(sbs_path)
225     print(f'  Saved: {sbs_path.name}')
226
227 print('\nGenerating qualitative grid (2x2 of side-by-sides)...')
228 fig, axes = plt.subplots(2, 2, figsize=(28, 18))
229 for i, sbs_path in enumerate(sidebyside_paths):
230     row, col = i // 2, i % 2
231     img = np.array(Image.open(sbs_path))
232     axes[row, col].imshow(img)
233     axes[row, col].set_title(f'View {i+1}', fontsize=12)
234     axes[row, col].axis('off')
235 fig.suptitle('Qualitative Comparison: N=14 vs N=24 Mesh (4 Viewpoints)', fontsize=16, fontweight='bold')
236 plt.tight_layout()
237 grid_path = OUT_DIR / 'qualitative_grid.png'
238 plt.savefig(str(grid_path), dpi=80, bbox_inches='tight')
239 plt.close()
240 print(f'Saved: {grid_path}')
241 print('\n==== STAGE: QUALITATIVE COMPARISON COMPLETE =====')

```

Listing 11: qualitative_compare.py

B.2.6 recover_21036.py

```

1 import sqlite3
2 import numpy as np
3 import cv2
4 import struct
5 import shutil
6 from pathlib import Path

```

```

7
8 DB_PATH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/database.db')
9 SPARSE_IN = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0')
10 SPARSE_OUT = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0_with21036')
11 SPARSE_OUT.mkdir(parents=True, exist_ok=True)
12
13 IMG36_ID = 14
14 IMG37_ID = 15
15
16 db = sqlite3.connect(str(DB_PATH))
17 cur = db.cursor()
18
19 def read_keypoints(image_id):
20     cur.execute('SELECT rows, cols, data FROM keypoints WHERE image_id=?', (image_id,))
21     row = cur.fetchone()
22     n, cols, data = row
23     kps = np.frombuffer(data, dtype=np.float32).reshape(n, cols)
24     return kps[:, :2]
25
26 def read_matches(id1, id2):
27     pair_id = id1 * 2147483647 + id2
28     cur.execute('SELECT rows, cols, data, H FROM two_view_geometries WHERE pair_id=?', (pair_id,))
29     row = cur.fetchone()
30     n_rows, n_cols, data, H_bytes = row
31     matches = np.frombuffer(data, dtype=np.uint32).reshape(n_rows, 2)
32     H = np.frombuffer(H_bytes, dtype=np.float64).reshape(3, 3) if H_bytes else None
33     return matches, H
34
35 def read_cameras_bin(path):
36     cameras = {}
37     model_nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
38     with open(path, 'rb') as f:
39         num = struct.unpack('<Q', f.read(8))[0]
40         for _ in range(num):
41             cid = struct.unpack('<I', f.read(4))[0]
42             model = struct.unpack('<I', f.read(4))[0]
43             w = struct.unpack('<Q', f.read(8))[0]
44             h = struct.unpack('<Q', f.read(8))[0]
45             np_ = model_nparams.get(model, 4)
46             params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
47             cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
48     return cameras
49
50 def camera_K_dist(cam):
51     model = cam['model']
52     p = cam['params']
53     if model == 0:
54         f, cx, cy = p[0], p[1], p[2]
55         K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
56         dist = np.zeros(5)
57     elif model == 1:
58         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
59         K = np.array([[fx, 0, cx], [0, fy, cy], [0, 0, 1]])
60         dist = np.zeros(5)
61     elif model == 2:
62         f, cx, cy, k1 = p[0], p[1], p[2], p[3]
63         K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
64         dist = np.array([k1, 0, 0, 0, 0])
65     elif model == 3:
66         f, cx, cy, k1, k2 = p[0], p[1], p[2], p[3], p[4]
67         K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
68         dist = np.array([k1, k2, 0, 0, 0])
69     else:
70         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
71         K = np.array([[fx, 0, cx], [0, fy, cy], [0, 0, 1]])
72         dist = np.array(p[4:8] + [0]*(8-len(p[4:8])))[:5] if len(p)>4 else np.zeros(5)
73     return K.astype(np.float64), dist.astype(np.float64)
74
75 def read_images_bin(path):
76     images = {}
77     model_nparams_kp = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
78     with open(path, 'rb') as f:
79         num = struct.unpack('<Q', f.read(8))[0]
80         for _ in range(num):
81             iid = struct.unpack('<I', f.read(4))[0]
82             qvec = np.array(struct.unpack('<4d', f.read(32)))
83             tvec = np.array(struct.unpack('<3d', f.read(24)))
84             cid = struct.unpack('<I', f.read(4))[0]
85             name = b''
86             while True:
87                 c = f.read(1)
88                 if c == b'\x00': break
89                 name += c
90             name = name.decode()
91             n = struct.unpack('<Q', f.read(8))[0]
92             xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
93             pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
94             w,x,y,z = qvec
95             R = np.array([[1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w], [2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w], [2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*
96                 y*y]])
97             images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name, 'R': R, 'xys': xys, 'pt3d': pt3d}
98     return images

```

```

98
99 def qvec_from_R(R):
100     trace = R[0,0] + R[1,1] + R[2,2]
101     if trace > 0:
102         s = 0.5 / np.sqrt(trace + 1.0)
103         w = 0.25 / s
104         x = (R[2,1] - R[1,2]) * s
105         y = (R[0,2] - R[2,0]) * s
106         z = (R[1,0] - R[0,1]) * s
107     elif R[0,0] > R[1,1] and R[0,0] > R[2,2]:
108         s = 2.0 * np.sqrt(1.0 + R[0,0] - R[1,1] - R[2,2])
109         w = (R[2,1] - R[1,2]) / s
110         x = 0.25 * s
111         y = (R[0,1] + R[1,0]) / s
112         z = (R[0,2] + R[2,0]) / s
113     elif R[1,1] > R[2,2]:
114         s = 2.0 * np.sqrt(1.0 + R[1,1] - R[0,0] - R[2,2])
115         w = (R[0,2] - R[2,0]) / s
116         x = (R[0,1] + R[1,0]) / s
117         y = 0.25 * s
118         z = (R[1,2] + R[2,1]) / s
119     else:
120         s = 2.0 * np.sqrt(1.0 + R[2,2] - R[0,0] - R[1,1])
121         w = (R[1,0] - R[0,1]) / s
122         x = (R[0,2] + R[2,0]) / s
123         y = (R[1,2] + R[2,1]) / s
124         z = 0.25 * s
125     return np.array([w, x, y, z])
126
127 kps36 = read_keypoints(IMG36_ID)
128 kps37 = read_keypoints(IMG37_ID)
129 matches, H_colmap = read_matches(IMG36_ID, IMG37_ID)
130
131 pts36 = kps36[matches[:, 0]].astype(np.float64)
132 pts37 = kps37[matches[:, 1]].astype(np.float64)
133
134 cameras_orig = read_cameras_bin(SPARSE_IN / 'cameras.bin')
135 images_orig = read_images_bin(SPARSE_IN / 'images.bin')
136
137 img37 = images_orig[15]
138 R37 = img37['R']
139 t37 = img37['tvec']
140 cam_id37 = img37['cam_id']
141 K37, dist37 = camera_K_dist(cameras_orig[cam_id37])
142 K36, dist36 = camera_K_dist(cameras_orig[2])
143
144 print(f'K36: f={K36[0,0]:.2f} cx={K36[0,2]:.2f} cy={K36[1,2]:.2f}')
145 print(f'K37: f={K37[0,0]:.2f} cx={K37[0,2]:.2f} cy={K37[1,2]:.2f}')
146
147 print(f'\nHcolmap:\n{H_colmap}')
148 print(f'\nDecomposing homography...')
149 retval, rotations, translations, normals = cv2.decomposeHomographyMat(H_colmap, K36)
150 print(f'{retval} solutions found')
151
152 pts36_u = cv2.undistortPoints(pts36.reshape(-1,1,2), K36, dist36).reshape(-1,2)
153 pts37_u = cv2.undistortPoints(pts37.reshape(-1,1,2), K37, dist37).reshape(-1,2)
154
155 best_sol = None
156 best_inliers = -1
157
158 for i, (R_rel, t_rel, n_vec) in enumerate(zip(rotations, translations, normals)):
159     t_r = t_rel.flatten()
160     P1 = np.hstack([np.eye(3), np.zeros((3,1))])
161     P2 = np.hstack([R_rel, t_r.reshape(3,1)])
162     pts_in_front = 0
163     sample = min(100, len(pts36_u))
164     for pt1, pt2 in zip(pts36_u[:sample], pts37_u[:sample]):
165         A = np.array([
166             pt1[0]*P1[2] - P1[0],
167             pt1[1]*P1[2] - P1[1],
168             pt2[0]*P2[2] - P2[0],
169             pt2[1]*P2[2] - P2[1]
170         ])
171         _, _, Vt = np.linalg.svd(A)
172         X = Vt[-1]
173         X = X / X[3]
174         z1 = X[2]
175         z2 = (R_rel @ X[:3] + t_r)[2]
176         if z1 > 0 and z2 > 0:
177             pts_in_front += 1
178     print(f' sol {i}: t_rel={np.round(t_r,3)} pts_in_front={pts_in_front}/{sample}')
179     if pts_in_front > best_inliers:
180         best_inliers = pts_in_front
181         best_sol = (R_rel, t_r)
182
183 db.close()
184
185 if best_sol is None:
186     print('No valid solution found - giving up on 21036')
187     exit(1)
188
189 R_rel, t_rel = best_sol

```



```

190 print(f'\nSelected: pts_in_front={best_inliers}')
191
192 R36_abs = R37 @ R_rel.T
193 C36 = -R37.T @ t37 + R_rel.T @ t_rel
194 t36_abs = -R36_abs @ (-R37.T @ t37 - R_rel.T @ t_rel)
195
196 t36_abs2 = t37 + R37 @ (-R_rel.T @ t_rel)
197
198 print(f't36_abs from 21037 pose: {np.round(t36_abs,4)}')
199 print(f't36_abs alternative: {np.round(t36_abs2,4)}')
200 print(f't37 for reference: {np.round(t37,4)}')
201
202 C37 = -R37.T @ t37
203 C36_calc = C37 + R37.T @ (R_rel.T @ t_rel)
204 t36_final = -R36_abs @ C36_calc
205
206 print(f'C37: {np.round(C37,4)}')
207 print(f'C36: {np.round(C36_calc,4)}')
208 print(f't36_final: {np.round(t36_final,4)}')
209
210 qvec36 = qvec_from_R(R36_abs)
211 print(f'qvec36: {np.round(qvec36,6)}')
212
213 shutil.copy(SPARSE_IN / 'cameras.bin', SPARSE_OUT / 'cameras.bin')
214 shutil.copy(SPARSE_IN / 'points3D.bin', SPARSE_OUT / 'points3D.bin')
215
216 with open(SPARSE_IN / 'images.bin', 'rb') as fin, open(SPARSE_OUT / 'images.bin', 'wb') as fout:
217     num_orig = struct.unpack('<Q', fin.read(8))[0]
218     fout.write(struct.pack('<Q', num_orig + 1))
219     fout.write(fin.read())
220
221     new_img_id = 16
222     fout.write(struct.pack('<I', new_img_id))
223     fout.write(struct.pack('<4d', *qvec36))
224     fout.write(struct.pack('<3d', *t36_final))
225     fout.write(struct.pack('<I', 2))
226     fout.write(b'mag138/AS17-138-21036HR.png\x00')
227
228     n_kp = len(kps36)
229     fout.write(struct.pack('<Q', n_kp))
230     for kp in kps36:
231         fout.write(struct.pack('<2d', float(kp[0]), float(kp[1])))
232         fout.write(struct.pack('<q', -1))
233
234 print(f'\nWrote extended sparse model to {SPARSE_OUT}')
235 print(f'21036 pose: C=[{C36_calc[0]:.3f},{C36_calc[1]:.3f},{C36_calc[2]:.3f}]')

```

Listing 12: recover_21036.py

B.3 Group C: Novel-Pose Synthesis and Rendering

B.3.1 diagnose_novel.py

```

1 from pathlib import Path
2 import numpy as np
3 import json
4 import struct
5 import collections
6 import subprocess
7
8 base = Path.home() / "ses598-apollo17"
9 sparse_dir = base / "colmap_n15" / "sparse" / "0"
10
11 CameraTuple = collections.namedtuple("CameraTuple", ["id", "model", "width", "height", "params"])
12 ImageTuple = collections.namedtuple("ImageTuple", ["id", "qvec", "tvec", "camera_id", "name"])
13
14 def read_next_bytes(fid, num_bytes, format_char_sequence, endian="<"):
15     data = fid.read(num_bytes)
16     return struct.unpack(endian + format_char_sequence, data)
17
18 def read_cameras_binary(path):
19     cameras = {}
20     with open(path, "rb") as fid:
21         num_cameras = read_next_bytes(fid, 8, "Q")[0]
22         for _ in range(num_cameras):
23             cam_props = read_next_bytes(fid, 24, "iiQQ")
24             camera_id = cam_props[0]
25             model_id = cam_props[1]
26             width = cam_props[2]
27             height = cam_props[3]
28             num_params = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8, 5: 8, 6: 12, 7: 5, 8: 4, 9: 5, 10: 12}[model_id]
29             params = read_next_bytes(fid, 8 * num_params, "d" * num_params)
30             cameras[camera_id] = CameraTuple(camera_id, model_id, width, height, params)
31     return cameras
32
33 def read_images_binary(path):

```

```

34 images = {}
35 with open(path, "rb") as fid:
36     num_images = read_next_bytes(fid, 8, "Q")[0]
37     for _ in range(num_images):
38         image_props = read_next_bytes(fid, 64, "idddddddi")
39         image_id = image_props[0]
40         qvec = np.array(image_props[1:5])
41         tvec = np.array(image_props[5:8])
42         camera_id = image_props[8]
43         name = ""
44         char = read_next_bytes(fid, 1, "c")[0]
45         while char != b"\x00":
46             name += char.decode("utf-8")
47             char = read_next_bytes(fid, 1, "c")[0]
48         num_points2d = read_next_bytes(fid, 8, "Q")[0]
49         read_next_bytes(fid, 24 * num_points2d, "ddq" * num_points2d)
50         images[image_id] = ImageTuple(image_id, qvec, tvec, camera_id, name)
51     return images
52
53 def qvec_to_rotmat(qvec):
54     w, x, y, z = qvec
55     return np.array([
56         [1 - 2*y*y - 2*z*z, 2*x*y - 2*z*w, 2*x*z + 2*y*w],
57         [2*x*y + 2*z*w, 1 - 2*x*x - 2*z*z, 2*y*z - 2*x*w],
58         [2*x*z - 2*y*w, 2*y*z + 2*x*w, 1 - 2*x*x - 2*y*y]])
59
60 cameras = read_cameras_binary(sparse_dir / "cameras.bin")
61 images = read_images_binary(sparse_dir / "images.bin")
62
63 print(f"Cameras: {len(cameras)}")
64 print(f"Images: {len(images)}")
65 print()
66
67 centers = []
68 forwards = []
69 for img_id, img in sorted(images.items(), key=lambda kv: kv[1].name):
70     R = qvec_to_rotmat(img.qvec)
71     t = img.tvec
72     center = -R.T @ t
73     forward = R.T @ np.array([0, 0, 1])
74     centers.append(center)
75     forwards.append(forward)
76     print(f"{img.name:40s} center=({center[0]:7.3f},{center[1]:7.3f},{center[2]:7.3f}) cam_id={img.camera_id}")
77
78 centers = np.array(centers)
79 forwards = np.array(forwards)
80
81 print()
82 print("=== Pose cloud statistics ===")
83 print(f"Centroid: {centers.mean(axis=0)}")
84 print(f"Bounding box: min={centers.min(axis=0)}, max={centers.max(axis=0)}")
85 print(f"Spread (std): {centers.std(axis=0)}")
86 print(f"Mean pairwise distance: {np.mean([np.linalg.norm(centers[i]-centers[j]) for i in range(len(centers)) for j in range(i+1,len(centers))]):.3f}")
87
88 mag137_mask = ["mag137" in img.name for img in sorted(images.values(), key=lambda i: i.name)]
89 mag138_mask = ["mag138" in img.name for img in sorted(images.values(), key=lambda i: i.name)]
90
91 mag137_centers = centers[mag137_mask]
92 mag138_centers = centers[mag138_mask]
93
94 print()
95 print(f"mag137 centroid: {mag137_centers.mean(axis=0)}")
96 print(f"mag138 centroid: {mag138_centers.mean(axis=0)}")
97 print(f"Inter-cluster distance: {np.linalg.norm(mag137_centers.mean(axis=0) - mag138_centers.mean(axis=0)):.3f}")
98 print(f"mag137 intra-cluster mean dist: {np.mean([np.linalg.norm(mag137_centers[i]-mag137_centers[j]) for i in range(len(mag137_centers)) for j in range(i+1,len(mag137_centers))]):.3f}")
99 print(f"mag138 intra-cluster mean dist: {np.mean([np.linalg.norm(mag138_centers[i]-mag138_centers[j]) for i in range(len(mag138_centers)) for j in range(i+1,len(mag138_centers))]):.3f}")
100
101 mean_forward = forwards.mean(axis=0)
102 mean_forward = mean_forward / np.linalg.norm(mean_forward)
103 look_at_target = centers.mean(axis=0) + mean_forward * np.linalg.norm(centers.std(axis=0)) * 2
104 print(f"\nEstimated scene look-at target: {look_at_target}")

```

Listing 13: diagnose_novel.py

B.3.2 gen_novel_poses.py

```

1 import numpy as np
2 import struct
3 import json
4 from pathlib import Path
5
6 SPARSE = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0')
7 SPLAT_DIR = Path('/home/jatin-satyam/ses598-apollo17/splats_n15/apollo17_n15/splatfacto/2026-04-25_144254')
8 OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')

```

```

9 OUT_DIR.mkdir(parents=True, exist_ok=True)
10
11 with open(SPLAT_DIR / 'dataparser_transforms.json') as f:
12     dp = json.load(f)
13     T34 = np.array(dp['transform'], dtype=np.float64)
14     scale = float(dp['scale'])
15     print('Transform T34:')
16     print(T34)
17     print(f'Scale: {scale}')
18
19 def read_cameras(path):
20     cameras = {}
21     with open(path, 'rb') as f:
22         n = struct.unpack('Q', f.read(8))[0]
23         for _ in range(n):
24             cid = struct.unpack('I', f.read(4))[0]
25             mid = struct.unpack('i', f.read(4))[0]
26             w = struct.unpack('Q', f.read(8))[0]
27             h = struct.unpack('Q', f.read(8))[0]
28             nparams = {0:3,1:4,2:4,3:5,4:8}[mid]
29             params = struct.unpack(f'{nparams}d', f.read(8*nparams))
30             cameras[cid] = {'model_id':mid,'w':w,'h':h,'params':params}
31     return cameras
32
33 def read_images(path):
34     images = {}
35     with open(path, 'rb') as f:
36         n = struct.unpack('Q', f.read(8))[0]
37         for _ in range(n):
38             iid = struct.unpack('I', f.read(4))[0]
39             qw,qx,qy,qz = struct.unpack('4d', f.read(32))
40             tx,ty,tz = struct.unpack('3d', f.read(24))
41             cid = struct.unpack('I', f.read(4))[0]
42             name = b''
43             while True:
44                 c = f.read(1)
45                 if c == b'\x00': break
46                 name += c
47             npts = struct.unpack('Q', f.read(8))[0]
48             f.read(npts*24)
49             images[iid] = {'qvec':(qw,qx,qy,qz),'tvec':(tx,ty,tz),'cam_id':cid,'name':name.decode()}
50     return images
51
52 def qvec_to_R(qvec):
53     qw,qx,qy,qz = qvec
54     return np.array([
55         [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
56         [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
57         [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
58     ])
59
60 def read_images_with_pts(path):
61     images = {}
62     with open(path, 'rb') as f:
63         n = struct.unpack('Q', f.read(8))[0]
64         for _ in range(n):
65             iid = struct.unpack('I', f.read(4))[0]
66             qw,qx,qy,qz = struct.unpack('4d', f.read(32))
67             tx,ty,tz = struct.unpack('3d', f.read(24))
68             cid = struct.unpack('I', f.read(4))[0]
69             name = b''
70             while True:
71                 c = f.read(1)
72                 if c == b'\x00': break
73                 name += c
74             npts = struct.unpack('Q', f.read(8))[0]
75             pt_ids = []
76             for _ in range(npts):
77                 struct.unpack('2d', f.read(16))
78                 pid = struct.unpack('Q', f.read(8))[0]
79                 if pid != 2**64-1:
80                     pt_ids.append(pid)
81             images[iid] = {'name': name.decode(), 'cam_id': cid,
82                           'qvec': (qw,qx,qy,qz), 'tvec': (tx,ty,tz),
83                           'pt3d_ids': set(pt_ids)}
84     return images
85
86 def read_all_pts(path, filter_box):
87     pts = {}
88     x0,x1,y0,y1,z0,z1 = filter_box
89     with open(path, 'rb') as f:
90         n = struct.unpack('Q', f.read(8))[0]
91         for _ in range(n):
92             pid = struct.unpack('Q', f.read(8))[0]
93             x,y,z = struct.unpack('3d', f.read(24))
94             f.read(3); f.read(8)
95             ntrack = struct.unpack('Q', f.read(8))[0]
96             f.read(ntrack*8)
97             if x0<=x<=x1 and y0<=y<=y1 and z0<=z<=z1:
98                 pts[pid] = np.array([x,y,z])
99     return pts
100

```

```

101 bbox = (-7.5, 1.0, -2.5, 4.5, 0.5, 11.0)
102 all_pts = read_all_pts(SPARSE / 'points3D.bin', bbox)
103
104 cameras = read_cameras(SPARSE / 'cameras.bin')
105 images = read_images_with_pts(SPARSE / 'images.bin')
106
107 mag137_imgs = [img for img in images.values() if 'mag137' in img['name']]
108 mag138_imgs = [img for img in images.values() if 'mag138' in img['name']]
109
110 def rock_centroid(imgs):
111     ptids = set()
112     for img in imgs:
113         ptids.update(img['pt3d_ids'])
114     pts = np.array([all_pts[pid] for pid in ptids if pid in all_pts])
115     return np.median(pts, axis=0) if len(pts) > 0 else None
116
117 T_lookat137 = rock_centroid(mag137_imgs)
118 T_lookat138 = rock_centroid(mag138_imgs)
119 print(f'Rock centroid mag137: {T_lookat137.round(4)}')
120 print(f'Rock centroid mag138: {T_lookat138.round(4)}')
121
122 def cluster_geometry(imgs):
123     centers, forwards = [], []
124     for img in imgs:
125         R = qvec_to_R(img['qvec'])
126         t = np.array(img['tvec'])
127         C = -R.T @ t
128         fwd = R.T[:, 2]
129         centers.append(C)
130         forwards.append(fwd)
131     centers = np.array(centers)
132     forwards = np.array(forwards)
133     centroid = centers.mean(0)
134     radii = np.linalg.norm(centers - centroid, axis=1)
135     radius = float(np.median(radii))
136     mean_fwd = forwards.mean(0)
137     mean_fwd = mean_fwd / np.linalg.norm(mean_fwd)
138     return centroid, radius, mean_fwd, centers
139
140 C137, r137, F137, centers137 = cluster_geometry(mag137_imgs)
141 C138, r138, F138, centers138 = cluster_geometry(mag138_imgs)
142
143 T_formula137 = C137 + F137 * r137 * 1.5
144 T_formula138 = C138 + F138 * r138 * 1.5
145
146 print(f'\nmag137: C={C137.round(4)} r={r137:.4f} F={F137.round(4)}')
147 print(f' lookat_target(formula)={T_formula137.round(4)} lookat_target(actual)={T_lookat137.round(4)}')
148 print(f'mag138: C={C138.round(4)} r={r138:.4f} F={F138.round(4)}')
149 print(f' lookat_target(formula)={T_formula138.round(4)} lookat_target(actual)={T_lookat138.round(4)}')
150
151 def lookat_colmap(eye, target):
152     world_up = np.array([0., 1., 0.])
153     Z = target - eye
154     Z = Z / np.linalg.norm(Z)
155     X = np.cross(world_up, Z)
156     X = X / np.linalg.norm(X)
157     Y = np.cross(X, Z)
158     c2w = np.eye(4)
159     c2w[:3, 0] = X
160     c2w[:3, 1] = Y
161     c2w[:3, 2] = Z
162     c2w[:3, 3] = eye
163     return c2w
164
165 col_flip = np.diag([1., -1., -1., 1.])
166
167 def to_nerfstudio(c2w_44):
168     c2w_34 = (T34 @ (c2w_44 @ col_flip)).copy()
169     c2w_34[:, 3] *= scale
170     return np.vstack([c2w_34, [0., 0., 0., 1.]])
171
172 def arc_poses(C, r, F, T_lookat, n=5, span_deg=30.0):
173     F_xz = np.array([F[0], 0., F[2]])
174     F_xz = F_xz / np.linalg.norm(F_xz)
175     P_xz = np.array([-F_xz[2], 0., F_xz[0]])
176     angles = np.linspace(-span_deg, span_deg, n)
177     results = []
178     for a in angles:
179         th = np.radians(a)
180         d = np.cos(th) * F_xz + np.sin(th) * P_xz
181         eye = C + r * d
182         eye[1] = C[1]
183         c2w_colmap = lookat_colmap(eye, T_lookat)
184         c2w_ns = to_nerfstudio(c2w_colmap)
185         results.append({
186             'angle_deg': float(a),
187             'eye_colmap': eye.tolist(),
188             'c2w_colmap': c2w_colmap.tolist(),
189             'c2w_ns': c2w_ns.tolist(),
190         })
191     return results
192

```

```

193 poses137 = arc_poses(C137, r137, F137, T_formula137, n=5, span_deg=30.0)
194 poses138 = arc_poses(C138, r138, F138, T_formula138, n=5, span_deg=30.0)
195
196 cam1 = cameras[1]
197 f_train = cam1['params'][0] / 2.0
198 h_train = cam1['h'] / 2.0
199 vfov_deg = float(2.0 * np.degrees(np.arctan(h_train / 2.0 / f_train)))
200 print(f'\nVertical FOV: {vfov_deg:.4f} degrees (f_train={f_train:.2f}, h_train={h_train:.0f})')
201
202 RENDER_W, RENDER_H = 2000, 2000
203
204 camera_path_json = {
205     'camera_type': 'perspective',
206     'render_height': RENDER_H,
207     'render_width': RENDER_W,
208     'fps': 1,
209     'seconds': float(10),
210     'camera_path': [],
211 }
212
213 all_entries = [(p, 'mag137', i+1) for i, p in enumerate(poses137)] + \
214               [(p, 'mag138', i+1) for i, p in enumerate(poses138)]
215
216 for p, cluster, idx in all_entries:
217     camera_path_json['camera_path'].append({
218         'camera_to_world': np.array(p['c2w_ns']).flatten().tolist(),
219         'fov': vfov_deg,
220         'aspect': float(RENDER_W) / float(RENDER_H),
221     })
222
223 cam_path_file = OUT_DIR / 'novel_camera_path.json'
224 with open(cam_path_file, 'w') as f:
225     json.dump(camera_path_json, f, indent=2)
226 print(f'\nCamera path saved: {cam_path_file} ({len(camera_path_json["camera_path"])} poses)')
227
228 print('\nPose definitions:')
229 for p, cluster, idx in all_entries:
230     eye = np.array(p['eye_colmap'])
231     c_ns = np.array(p['c2w_ns'][:3, 3])
232     print(f' {cluster}_{idx:02d} angle={p["angle_deg"]:+6.1f} eye_colmap={eye.round(3)} t_ns={c_ns.round(4)}')

```

Listing 14: gen_novel_poses.py

B.3.3 make_contact_sheet.py

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use('Agg')
4 import matplotlib.pyplot as plt
5 from PIL import Image
6 from pathlib import Path
7
8 OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')
9
10 frames = [
11     ('novel_mag137_01.png', 'mag137_01 -30 deg'),
12     ('novel_mag137_02.png', 'mag137_02 -15 deg'),
13     ('novel_mag137_03.png', 'mag137_03 0 deg'),
14     ('novel_mag137_04.png', 'mag137_04 +15 deg'),
15     ('novel_mag137_05.png', 'mag137_05 +30 deg'),
16     ('novel_mag138_01.png', 'mag138_01 -30 deg'),
17     ('novel_mag138_02.png', 'mag138_02 -15 deg'),
18     ('novel_mag138_03.png', 'mag138_03 0 deg'),
19     ('novel_mag138_04.png', 'mag138_04 +15 deg'),
20     ('novel_mag138_05.png', 'mag138_05 +30 deg'),
21 ]
22
23 fig, axes = plt.subplots(2, 5, figsize=(22, 9))
24 fig.suptitle('Novel View Synthesis: 10 Rendered Poses (5 per Magazine Cluster)', fontsize=14, fontweight='bold')
25
26 for idx, (fname, label) in enumerate(frames):
27     row = idx // 5
28     col = idx % 5
29     ax = axes[row, col]
30     img = np.array(Image.open(OUT_DIR / fname).convert('RGB'))
31     thumb_h = 400
32     scale = thumb_h / img.shape[0]
33     thumb_w = int(img.shape[1] * scale)
34     from PIL import Image as PILImage
35     thumb = np.array(PILImage.fromarray(img).resize((thumb_w, thumb_h), PILImage.LANCZOS))
36     ax.imshow(thumb)
37     ax.set_title(label, fontsize=10, color='steelblue' if 'mag137' in fname else 'mediumvioletred')
38     ax.axis('off')
39
40 plt.tight_layout()
41 out = OUT_DIR / 'contact_sheet.png'
42 plt.savefig(str(out), dpi=120, bbox_inches='tight')

```

```

43 plt.close()
44 print(f'Saved: {out}')

```

Listing 15: make_contact_sheet.py

B.3.4 novel_pose_smoke.py

```

1 import json
2 import numpy as np
3 import torch
4 import struct
5 from pathlib import Path
6 import cv2
7
8 SPLAT_DIR = Path('/home/jatin-satyam/ses598-apollo17/splats_n15/apollo17_n15/splatfacto/2026-04-25_144254')
9 SPARSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/sparse')
10 OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')
11
12 # Read datparser transforms to understand coordinate mapping
13 transforms_path = SPLAT_DIR / 'dataparser_transforms.json'
14 with open(transforms_path) as f:
15     dp = json.load(f)
16 print('Datparser transforms:')
17 print(json.dumps(dp, indent=2))
18
19 # Interpolate between camera 0 and camera 6 (first and last mag137 views) at t=0.5
20 # We'll do this by reading the transforms.json that nerfstudio produces
21 # Actually nerfstudio COLMAP dataparser uses the sparse model directly.
22 # The dataparser_transforms.json has: transform (4x4) and scale
23
24 # Read COLMAP cameras from dense/sparse (PINHOLE format)
25 CAMERAS_PATH = SPARSE_DIR / 'cameras.bin'
26 IMAGES_PATH = SPARSE_DIR / 'images.bin'
27
28 def read_cameras_bin(path):
29     cameras = {}
30     with open(path, 'rb') as f:
31         n = struct.unpack('Q', f.read(8))[0]
32         for _ in range(n):
33             cam_id = struct.unpack('I', f.read(4))[0]
34             model_id = struct.unpack('i', f.read(4))[0]
35             w = struct.unpack('Q', f.read(8))[0]
36             h = struct.unpack('Q', f.read(8))[0]
37             nparams = {0:3, 1:4, 2:4, 3:5, 4:8}[model_id]
38             params = struct.unpack(f'{nparams}d', f.read(8*nparams))
39             cameras[cam_id] = {'model_id': model_id, 'w': w, 'h': h, 'params': params}
40     return cameras
41
42 def read_images_bin(path):
43     images = {}
44     with open(path, 'rb') as f:
45         n = struct.unpack('Q', f.read(8))[0]
46         for _ in range(n):
47             img_id = struct.unpack('I', f.read(4))[0]
48             qw, qx, qy, qz = struct.unpack('4d', f.read(32))
49             tx, ty, tz = struct.unpack('3d', f.read(24))
50             cam_id = struct.unpack('I', f.read(4))[0]
51             name = b''
52             while True:
53                 c = f.read(1)
54                 if c == b'\x00':
55                     break
56                 name += c
57             n_pts = struct.unpack('Q', f.read(8))[0]
58             f.read(n_pts * 24)
59             images[img_id] = {
60                 'qvec': (qw, qx, qy, qz),
61                 'tvec': (tx, ty, tz),
62                 'cam_id': cam_id,
63                 'name': name.decode(),
64             }
65     return images
66
67 cameras = read_cameras_bin(CAMERAS_PATH)
68 images = read_images_bin(IMAGES_PATH)
69
70 def qvec_to_R(qvec):
71     qw, qx, qy, qz = qvec
72     R = np.array([
73         [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
74         [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
75         [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)],
76     ])
77     return R
78
79 def slerp(q0, q1, t):
80     q0 = np.array(q0); q1 = np.array(q1)

```

```

81     dot = np.dot(q0, q1)
82     if dot < 0:
83         q1 = -q1; dot = -dot
84     dot = min(dot, 1.0)
85     theta = np.arccos(dot)
86     if theta < 1e-8:
87         return q0
88     return (np.sin((1-t)*theta) * q0 + np.sin(t*theta) * q1) / np.sin(theta)
89
90 img_list = sorted(images.values(), key=lambda x: x['name'])
91 mag137 = [img for img in img_list if 'mag137' in img['name']]
92 i0, i1 = mag137[0], mag137[-1]
93 print(f'Interpolating between {i0["name"]} and {i1["name"]}')
94
95 t = 0.5
96 q_interp = slerp(i0['qvec'], i1['qvec'], t)
97 t_interp = tuple((1-t)*np.array(i0['tvec']) + t*np.array(i1['tvec']))
98
99 R = qvec_to_R(q_interp)
100 t_vec = np.array(t_interp)
101 c2w = np.eye(4)
102 c2w[:3, :3] = R.T
103 c2w[:3, 3] = -R.T @ t_vec
104
105 cam = cameras[i0['cam_id']]
106 fx, fy, cx, cy = cam['params']
107 w, h = cam['w'], cam['h']
108
109 # Apply dataparser transform
110 T = np.array(dp['transform'])
111 scale = dp['scale']
112 c2w_ns_34 = T @ c2w
113 c2w_ns_34[:3, 3] *= scale
114 c2w_ns = np.vstack([c2w_ns_34, [0, 0, 0, 1]])
115
116 print(f'Interpolated camera center (nerfstudio coords): {c2w_ns[:3, 3]}')
117 print(f'fx={fx:.1f} fy={fy:.1f} cx={cx:.1f} cy={cy:.1f} {w}x{h}')
118
119 # Write a cameras_path JSON for ns-render camera-path
120 # nerfstudio camera_path format
121 camera_path = {
122     'camera_type': 'perspective',
123     'render_height': h,
124     'render_width': w,
125     'fps': 1,
126     'seconds': 1.0,
127     'camera_path': [{
128         'camera_to_world': c2w_ns.flatten().tolist(),
129         'fov': float(2 * np.degrees(np.arctan(0.5 * h / fy))),
130         'aspect': float(w / h),
131     }],
132 }
133
134 cam_path_file = OUT_DIR / 'novel_camera_path.json'
135 with open(cam_path_file, 'w') as f:
136     json.dump(camera_path, f, indent=2)
137 print(f'Saved camera path: {cam_path_file}')

```

Listing 16: novel_pose_smoke.py

B.3.5 sanity_render.py

```

1 import numpy as np
2 import open3d as o3d
3 import struct
4 import os
5 os.environ['PYOPENGL_PLATFORM'] = 'egl'
6 import pyrender
7 import trimesh
8 from pathlib import Path
9 from PIL import Image
10
11 def read_images_bin(path):
12     images = {}
13     with open(path, 'rb') as f:
14         n = struct.unpack('<Q', f.read(8))[0]
15         for _ in range(n):
16             iid = struct.unpack('<I', f.read(4))[0]
17             qvec = np.array(struct.unpack('<4d', f.read(32)))
18             tvec = np.array(struct.unpack('<3d', f.read(24)))
19             cid = struct.unpack('<I', f.read(4))[0]
20             name = b''
21             while True:
22                 c = f.read(1)
23                 if c == b'\x00': break
24                 name += c
25             n_pts = struct.unpack('<Q', f.read(8))[0]

```

```

26         f.read(n_pts * 24)
27         qw, qx, qy, qz = qvec
28         R = np.array([
29             [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
30             [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
31             [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
32         ])
33         images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec)
34     return images
35
36 def read_cameras_bin(path):
37     cameras = {}
38     nparams = {0:3,1:4,2:4,3:5,4:8}
39     with open(path, 'rb') as f:
40         n = struct.unpack('<Q', f.read(8))[0]
41         for _ in range(n):
42             cid = struct.unpack('<I', f.read(4))[0]
43             mid = struct.unpack('<I', f.read(4))[0]
44             w = struct.unpack('<Q', f.read(8))[0]
45             h = struct.unpack('<Q', f.read(8))[0]
46             np_ = nparams[mid]
47             params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
48             cameras[cid] = dict(model=mid, W=w, H=h, params=params)
49     return cameras
50
51 def colmap_pose_to_c2w(R, tvec):
52     c2w = np.eye(4)
53     c2w[:3, :3] = R.T
54     c2w[:3, 3] = -R.T @ tvec
55     return c2w
56
57 def render_vertex_colored_ply(ply_path, sparse_dir, img_name, out_path, W=800, H=800):
58     sparse_dir = Path(sparse_dir)
59     cams = read_cameras_bin(sparse_dir / 'cameras.bin')
60     imgs = read_images_bin(sparse_dir / 'images.bin')
61
62     chosen = next(img for img in imgs.values() if img['name'] == img_name)
63     cam_data = cams[chosen['cam_id']]
64     p = cam_data['params']
65     m = cam_data['model']
66     if m == 1:
67         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
68     elif m == 2:
69         fx = fy = p[0]; cx = p[1]; cy = p[2]
70     else:
71         fx = fy = p[0]; cx = p[1]; cy = p[2]
72
73     sx = W / cam_data['W']; sy = H / cam_data['H']
74     fx_r = fx * sx; fy_r = fy * sy; cx_r = cx * sx; cy_r = cy * sy
75
76     c2w = colmap_pose_to_c2w(chosen['R'], chosen['tvec'])
77     opengl_flip = np.diag([1, -1, -1, 1])
78     camera_pose = c2w @ opengl_flip
79
80     mesh_o3d = o3d.io.read_triangle_mesh(str(ply_path))
81     mesh_o3d.compute_vertex_normals()
82     verts = np.asarray(mesh_o3d.vertices)
83     tris = np.asarray(mesh_o3d.triangles)
84     colors = np.asarray(mesh_o3d.vertex_colors)
85
86     tm = trimesh.Trimesh(vertices=verts, faces=tris, vertex_colors=(colors * 255).astype(np.uint8))
87     mesh_pr = pyrender.Mesh.from_trimesh(tm, smooth=False)
88
89     scene = pyrender.Scene(bg_color=[0.2, 0.2, 0.2, 1.0], ambient_light=[0.8, 0.8, 0.8])
90     scene.add(mesh_pr)
91
92     camera = pyrender.IntrinsicsCamera(fx=fx_r, fy=fy_r, cx=cx_r, cy=cy_r, znear=0.01, zfar=1000.0)
93     scene.add(camera, pose=camera_pose)
94
95     light = pyrender.DirectionalLight(color=[1.0, 1.0, 1.0], intensity=2.0)
96     scene.add(light, pose=camera_pose)
97
98     renderer = pyrender.OffscreenRenderer(W, H)
99     color, depth = renderer.render(scene, flags=pyrender.RenderFlags.FLAT)
100    renderer.delete()
101
102    Image.fromarray(color).save(str(out_path))
103    print(f'Sanity render saved: {out_path}')
104    img_arr = np.array(Image.fromarray(color))
105    print(f' Pixel stats: mean={img_arr.mean():.1f} std={img_arr.std():.1f}')
106    return img_arr
107
108 def render_vertex_colored_ply(
109     'colmap_n15/dense/textured/textured.ply',
110     'colmap_n15/sparse/0',
111     'mag137/AS17-137-20907HR.png',
112     'colmap_n15/dense/textured/sanity_check.png'
113 )

```

Listing 17: sanity_render.py

B.3.6 test_novel_per_magazine.py

```
1 from pathlib import Path
2 import numpy as np
3 import json
4 import subprocess
5 import struct
6 import collections
7
8 base = Path.home() / "ses598-apollo17"
9 sparse_dir = base / "colmap_n15" / "sparse" / "0"
10 splat_config = sorted((base / "splats_n15").rglob("config.yml"))[-1]
11 output_dir = base / "test_novel_renderers"
12 output_dir.mkdir(exist_ok=True)
13
14 CameraTuple = collections.namedtuple("CameraTuple", ["id", "model", "width", "height", "params"])
15 ImageTuple = collections.namedtuple("ImageTuple", ["id", "qvec", "tvec", "camera_id", "name"])
16
17 def read_next_bytes(fid, num_bytes, format_char_sequence, endian="<"):
18     data = fid.read(num_bytes)
19     return struct.unpack(endian + format_char_sequence, data)
20
21 def read_cameras_binary(path):
22     cameras = {}
23     with open(path, "rb") as fid:
24         num_cameras = read_next_bytes(fid, 8, "Q")[0]
25         for _ in range(num_cameras):
26             cam_props = read_next_bytes(fid, 24, "iiQQ")
27             num_params = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8, 5: 8, 6: 12, 7: 5, 8: 4, 9: 5, 10: 12}[cam_props[1]]
28             params = read_next_bytes(fid, 8 * num_params, "d" * num_params)
29             cameras[cam_props[0]] = CameraTuple(cam_props[0], cam_props[1], cam_props[2], cam_props[3], params)
30     return cameras
31
32 def read_images_binary(path):
33     images = {}
34     with open(path, "rb") as fid:
35         num_images = read_next_bytes(fid, 8, "Q")[0]
36         for _ in range(num_images):
37             image_props = read_next_bytes(fid, 64, "idddddddi")
38             qvec = np.array(image_props[1:5])
39             tvec = np.array(image_props[5:8])
40             name = ""
41             char = read_next_bytes(fid, 1, "c")[0]
42             while char != b"\x00":
43                 name += char.decode("utf-8")
44                 char = read_next_bytes(fid, 1, "c")[0]
45             num_points2d = read_next_bytes(fid, 8, "Q")[0]
46             read_next_bytes(fid, 24 * num_points2d, "ddq" * num_points2d)
47             images[image_props[0]] = ImageTuple(image_props[0], qvec, tvec, image_props[8], name)
48     return images
49
50 def qvec_to_rotmat(qvec):
51     w, x, y, z = qvec
52     return np.array([
53         [1 - 2*y*y - 2*z*z, 2*x*y - 2*z*w, 2*x*z + 2*y*w],
54         [2*x*y + 2*z*w, 1 - 2*x*x - 2*z*z, 2*y*z - 2*x*w],
55         [2*x*z - 2*y*w, 2*y*z + 2*x*w, 1 - 2*x*x - 2*y*y]])
56
57 def look_at_to_c2w(eye, target, up=np.array([0, 1, 0])):
58     forward = target - eye
59     forward = forward / np.linalg.norm(forward)
60     right = np.cross(forward, up)
61     right = right / np.linalg.norm(right)
62     new_up = np.cross(right, forward)
63     c2w = np.eye(4)
64     c2w[:3, 0] = right
65     c2w[:3, 1] = -new_up
66     c2w[:3, 2] = forward
67     c2w[:3, 3] = eye
68     return c2w
69
70 cameras = read_cameras_binary(sparse_dir / "cameras.bin")
71 images = read_images_binary(sparse_dir / "images.bin")
72
73 mag137_centers = []
74 mag137_forwards = []
75 mag138_centers = []
76 mag138_forwards = []
77 for img in images.values():
78     R = qvec_to_rotmat(img.qvec)
79     center = -R.T @ img.tvec
80     forward = R.T @ np.array([0, 0, 1])
81     if "mag137" in img.name:
82         mag137_centers.append(center)
83         mag137_forwards.append(forward)
84     else:
85         mag138_centers.append(center)
86         mag138_forwards.append(forward)
87
88 mag137_centers = np.array(mag137_centers)
89 mag138_centers = np.array(mag138_centers)
90 mag137_forwards = np.array(mag137_forwards)
```

```

91 mag138_forwards = np.array(mag138_forwards)
92
93 mag137_centroid = mag137_centers.mean(axis=0)
94 mag138_centroid = mag138_centers.mean(axis=0)
95 mag137_radius = np.median(np.linalg.norm(mag137_centers - mag137_centroid, axis=1))
96 mag138_radius = np.median(np.linalg.norm(mag138_centers - mag138_centroid, axis=1))
97 mag137_mean_fwd = mag137_forwards.mean(axis=0)
98 mag137_mean_fwd = mag137_mean_fwd / np.linalg.norm(mag137_mean_fwd)
99 mag138_mean_fwd = mag138_forwards.mean(axis=0)
100 mag138_mean_fwd = mag138_mean_fwd / np.linalg.norm(mag138_mean_fwd)
101
102 mag137_target = mag137_centroid + mag137_mean_fwd * 3.0
103 mag138_target = mag138_centroid + mag138_mean_fwd * 3.0
104
105 print(f"mag137 centroid={mag137_centroid}, radius={mag137_radius:.3f}, target={mag137_target}")
106 print(f"mag138 centroid={mag138_centroid}, radius={mag138_radius:.3f}, target={mag138_target}")
107
108 test_poses = []
109 test_poses.append(("mag137_test_a", mag137_centroid, mag137_target))
110 test_poses.append(("mag137_test_b", mag137_centroid + np.array([0, 0, 0.5]), mag137_target))
111 test_poses.append(("mag138_test_a", mag138_centroid, mag138_target))
112 test_poses.append(("mag138_test_b", mag138_centroid + np.array([0, 0, 0.8]), mag138_target))
113
114 camera_path = {
115     "render_height": 1024,
116     "render_width": 1024,
117     "camera_path": [],
118     "fps": 1,
119     "seconds": len(test_poses),
120     "smoothness_value": 0,
121     "is_cycle": False,
122     "crop": None
123 }
124
125 for name, eye, target in test_poses:
126     c2w = look_at_to_c2w(eye, target)
127     camera_path["camera_path"].append({
128         "camera_to_world": c2w.flatten().tolist(),
129         "fov": 50,
130         "aspect": 1.0
131     })
132
133 cam_path_file = output_dir / "test_path.json"
134 with open(cam_path_file, "w") as f:
135     json.dump(camera_path, f, indent=2)
136
137 print(f"\nWrote camera path to {cam_path_file}")
138 print(f"Splat config: {splat_config}")
139 print(f"\nNow run: nns-render camera-path --load-config {splat_config} --camera-path-filename {cam_path_file} --output-path {output_dir} / 'test.mp4'")

```

Listing 18: test_novel_per_magazine.py

B.4 Group D: Texturing and Qualitative Comparison

B.4.1 make_comparison_grid.py

```

1 import cv2
2 import numpy as np
3 import matplotlib
4 matplotlib.use('Agg')
5 import matplotlib.pyplot as plt
6 from pathlib import Path
7
8 RENDERS_DIR = Path('/home/jatin-satyam/ses598-apollo17/renders_n15/train/rgb')
9 RGB_DIR = Path('/home/jatin-satyam/ses598-apollo17/data/rgb')
10 METRICS_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')
11
12 IMAGE_MAP = {
13     'AS17-137-20903HR': 'mag137/AS17-137-20903HR.png',
14     'AS17-137-20904HR': 'mag137/AS17-137-20904HR.png',
15     'AS17-137-20905HR': 'mag137/AS17-137-20905HR.png',
16     'AS17-137-20906HR': 'mag137/AS17-137-20906HR.png',
17     'AS17-137-20907HR': 'mag137/AS17-137-20907HR.png',
18     'AS17-137-20908HR': 'mag137/AS17-137-20908HR.png',
19     'AS17-137-20909HR': 'mag137/AS17-137-20909HR.png',
20     'AS17-138-21030HR': 'mag138/AS17-138-21030HR.png',
21     'AS17-138-21031HR': 'mag138/AS17-138-21031HR.png',
22     'AS17-138-21032HR': 'mag138/AS17-138-21032HR.png',
23     'AS17-138-21033HR': 'mag138/AS17-138-21033HR.png',
24     'AS17-138-21034HR': 'mag138/AS17-138-21034HR.png',
25     'AS17-138-21035HR': 'mag138/AS17-138-21035HR.png',
26     'AS17-138-21037HR': 'mag138/AS17-138-21037HR.png',
27 }
28
29 SELECTED = [

```

```

30     ('AS17-137-20904HR', 30.18, 0.9312, 'Worst PSNR'),
31     ('AS17-137-20907HR', 34.52, 0.9590, 'Median PSNR'),
32     ('AS17-138-21031HR', 35.30, 0.9485, 'Random'),
33     ('AS17-138-21037HR', 37.36, 0.9493, 'Best PSNR'),
34 ]
35
36 def load_pair(key):
37     rel = IMAGE_MAP[key]
38     mag = rel.split('/')[0]
39     render_path = RENDERS_DIR / mag / f'{key}.png'
40     orig_path = RGB_DIR / rel
41     render = cv2.cvtColor(cv2.imread(str(render_path)), cv2.COLOR_BGR2RGB)
42     orig = cv2.cvtColor(cv2.imread(str(orig_path)), cv2.COLOR_BGR2RGB)
43     rh, rw = render.shape[:2]
44     orig_r = cv2.resize(orig, (rw, rh), interpolation=cv2.INTER_AREA)
45     return orig_r, render
46
47 fig, axes = plt.subplots(4, 2, figsize=(14, 22))
48 fig.suptitle('Gaussian Splatting Reconstruction: Original vs Rendered\n(Training Views)', fontsize=14, fontweight='bold')
49
50 for row, (key, psnr, ssim, label) in enumerate(SELECTED):
51     orig, render = load_pair(key)
52     short = key.replace('AS17-', '').replace('HR', '')
53     axes[row, 0].imshow(orig)
54     axes[row, 0].set_title(f'{short} - Original', fontsize=10)
55     axes[row, 0].axis('off')
56     axes[row, 1].imshow(render)
57     axes[row, 1].set_title(f'{short} - Rendered [{label}] PSNR={psnr:.2f} dB SSIM={ssim:.4f}', fontsize=10)
58     axes[row, 1].axis('off')
59
60 plt.tight_layout()
61 out = METRICS_DIR / 'comparison_grid.png'
62 plt.savefig(str(out), dpi=120, bbox_inches='tight')
63 plt.close()
64 print(f'Saved: {out}')

```

Listing 19: make_comparison_grid.py

B.4.2 qualitative_compare_textured.py

```

1 import numpy as np
2 import open3d as o3d
3 import struct
4 import json
5 import os
6 os.environ['PYOPENGL_PLATFORM'] = 'egl'
7 import pyrender
8 import trimesh
9 from pathlib import Path
10 from PIL import Image
11 import matplotlib
12 matplotlib.use('Agg')
13 import matplotlib.pyplot as plt
14
15 SPARSE14 = Path('colmap_n15/sparse/0')
16 SPARSE24 = Path('colmap_n24/sparse/0')
17 PLY14 = Path('colmap_n15/dense/textured/textured.ply')
18 PLY24 = Path('colmap_n24/dense/textured/textured.ply')
19 METRICS_JSON = Path('comparison/quantitative_metrics.json')
20 IMAGES14 = Path('data/rgb')
21 OUT_DIR = Path('comparison')
22
23 RENDER_W, RENDER_H = 800, 800
24
25 with open(METRICS_JSON) as f:
26     T_icp = np.array(json.load(f)['icp_transform'])
27
28 def read_cameras_bin(path):
29     cameras = {}
30     nparams = {0:3,1:4,2:4,3:5,4:8}
31     with open(path, 'rb') as f:
32         n = struct.unpack('<Q', f.read(8))[0]
33         for _ in range(n):
34             cid = struct.unpack('<I', f.read(4))[0]
35             mid = struct.unpack('<I', f.read(4))[0]
36             w = struct.unpack('<Q', f.read(8))[0]
37             h = struct.unpack('<Q', f.read(8))[0]
38             np_ = nparams[mid]
39             params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
40             cameras[cid] = dict(model=mid, W=w, H=h, params=params)
41     return cameras
42
43 def read_images_bin(path):
44     images = {}
45     with open(path, 'rb') as f:
46         n = struct.unpack('<Q', f.read(8))[0]
47         for _ in range(n):

```

```

48     iid = struct.unpack('<I', f.read(4))[0]
49     qvec = np.array(struct.unpack('<4d', f.read(32)))
50     tvec = np.array(struct.unpack('<3d', f.read(24)))
51     cid = struct.unpack('<I', f.read(4))[0]
52     name = b''
53     while True:
54         c = f.read(1)
55         if c == b'\x00': break
56         name += c
57     n_pts = struct.unpack('<Q', f.read(8))[0]
58     f.read(n_pts * 24)
59     qw, qx, qy, qz = qvec
60     R = np.array([
61         [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
62         [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
63         [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qy**2+qz**2)]
64     ])
65     images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec)
66     return images
67
68 def build_c2w_opengl(R, tvec):
69     c2w = np.eye(4)
70     c2w[:3, :3] = R.T
71     c2w[:3, 3] = -R.T @ tvec
72     flip = np.diag([1.0, -1.0, -1.0, 1.0])
73     return c2w @ flip
74
75 def colmap_intrinsics_to_pyrender(cam_data, out_W, out_H):
76     p = cam_data['params']
77     m = cam_data['model']
78     if m == 1:
79         fx, fy = p[0], p[1]; cx, cy = p[2], p[3]
80     elif m == 2:
81         fx = fy = p[0]; cx = p[1]; cy = p[2]
82     else:
83         fx = fy = p[0]; cx = p[1]; cy = p[2]
84     sx = out_W / cam_data['W']; sy = out_H / cam_data['H']
85     return fx*sx, fy*sy, cx*sx, cy*sy
86
87 def load_vertex_colored_mesh(ply_path, transform=None):
88     mesh_o3d = o3d.io.read_triangle_mesh(str(ply_path))
89     if transform is not None:
90         mesh_o3d.transform(transform)
91     mesh_o3d.compute_vertex_normals()
92     verts = np.asarray(mesh_o3d.vertices)
93     tris = np.asarray(mesh_o3d.triangles)
94     colors = (np.asarray(mesh_o3d.vertex_colors) * 255).astype(np.uint8)
95     tm = trimesh.Trimesh(vertices=verts, faces=tris, vertex_colors=colors)
96     return pyrender.Mesh.from_trimesh(tm, smooth=False)
97
98 def render_mesh(mesh_pr, pose, fx, fy, cx, cy, W=RENDER_W, H=RENDER_H):
99     scene = pyrender.Scene(bg_color=[0.15, 0.15, 0.15, 1.0], ambient_light=[1.0, 1.0, 1.0])
100     scene.add(mesh_pr)
101     camera = pyrender.IntrinsicsCamera(fx=fx, fy=fy, cx=cx, cy=cy, znear=0.01, zfar=1000.0)
102     scene.add(camera, pose=pose)
103     light = pyrender.DirectionalLight(color=[1.0, 1.0, 1.0], intensity=3.0)
104     scene.add(light, pose=pose)
105     renderer = pyrender.OffscreenRenderer(W, H)
106     color, _ = renderer.render(scene, flags=pyrender.RenderFlags.FLAT)
107     renderer.delete()
108     return color
109
110 print('Loading cameras from N=14 sparse model...')
111 cams14 = read_cameras_bin(SPARSE14 / 'cameras.bin')
112 imgs14 = read_images_bin(SPARSE14 / 'images.bin')
113
114 chosen_names = [
115     'mag137/AS17-137-20904HR.png',
116     'mag137/AS17-137-20907HR.png',
117     'mag138/AS17-138-21030HR.png',
118     'mag138/AS17-138-21033HR.png',
119 ]
120 chosen_imgs = [next(v for v in imgs14.values() if v['name'] == nm) for nm in chosen_names]
121
122 print('Loading N=14 textured mesh...')
123 mesh14_pr = load_vertex_colored_mesh(PLY14)
124
125 print('Loading N=24 textured mesh (ICP-aligned to N=14 frame)...')
126 mesh24_pr = load_vertex_colored_mesh(PLY24, transform=T_icp)
127
128 sidebyside_paths = []
129 for vi, (img_info, img_name) in enumerate(zip(chosen_imgs, chosen_names), start=1):
130     print(f'Rendering view {vi}: {img_name}')
131
132     cam_data = cams14[img_info['cam_id']]
133     pose = build_c2w_opengl(img_info['R'], img_info['tvec'])
134     fx, fy, cx, cy = colmap_intrinsics_to_pyrender(cam_data, RENDER_W, RENDER_H)
135
136     color14 = render_mesh(mesh14_pr, pose, fx, fy, cx, cy)
137     color24 = render_mesh(mesh24_pr, pose, fx, fy, cx, cy)
138
139     r14_path = OUT_DIR / f'textured_render_n14_view{vi}.png'

```

```

140 r24_path = OUT_DIR / f'textured_render_n24_view{vi}.png'
141 Image.fromarray(color14).save(str(r14_path))
142 Image.fromarray(color24).save(str(r24_path))
143 print(f' N14: mean={color14.mean():.1f} std={color14.std():.1f}')
144 print(f' N24: mean={color24.mean():.1f} std={color24.std():.1f}')
145
146 photo_path = IMAGES14 / img_name
147 if photo_path.exists():
148     photo = np.array(Image.open(photo_path).convert('RGB').resize((RENDER_W, RENDER_H), Image.LANCZOS))
149     fig, axes = plt.subplots(1, 3, figsize=(24, 8))
150     axes[0].imshow(color14)
151     axes[0].set_title(f'N=14 Textured Mesh', fontsize=13, color='steelblue')
152     axes[0].axis('off')
153     axes[1].imshow(photo)
154     axes[1].set_title(f'Source Photo: {Path(img_name).stem}', fontsize=13, color='goldenrod')
155     axes[1].axis('off')
156     axes[2].imshow(color24)
157     axes[2].set_title(f'N=24 Textured Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
158     axes[2].axis('off')
159 else:
160     fig, axes = plt.subplots(1, 2, figsize=(16, 8))
161     axes[0].imshow(color14)
162     axes[0].set_title('N=14 Textured Mesh', fontsize=13, color='steelblue')
163     axes[0].axis('off')
164     axes[1].imshow(color24)
165     axes[1].set_title('N=24 Textured Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
166     axes[1].axis('off')
167
168 fig.suptitle(f'Textured Mesh Comparison View {vi}: {Path(img_name).stem}', fontsize=14, fontweight='bold')
169 fig.tight_layout()
170 sbs_path = OUT_DIR / f'textured_sidebyside_view{vi}.png'
171 plt.savefig(str(sbs_path), dpi=100, bbox_inches='tight')
172 plt.close()
173 sidebyside_paths.append(sbs_path)
174 print(f' Side-by-side: {sbs_path}')
175
176 print('\nGenerating 2x2 qualitative grid...')
177 fig, axes = plt.subplots(2, 2, figsize=(28, 18))
178 for i, sbs_path in enumerate(sidebyside_paths):
179     row, col = i // 2, i % 2
180     img = np.array(Image.open(sbs_path))
181     axes[row, col].imshow(img)
182     axes[row, col].set_title(f'View {i+1}: {Path(chosen_names[i]).stem}', fontsize=12)
183     axes[row, col].axis('off')
184 fig.suptitle('Textured Mesh Comparison: N=14 vs N=24 Photogrammetry (4 Viewpoints)', fontsize=16, fontweight='bold')
185 plt.tight_layout()
186 grid_path = OUT_DIR / 'textured_qualitative_grid.png'
187 plt.savefig(str(grid_path), dpi=80, bbox_inches='tight')
188 plt.close()
189 print(f'Saved: {grid_path}')
190 print('\n===== STAGE: TEXTURED QUALITATIVE COMPARISON COMPLETE =====')

```

Listing 20: qualitative_compare_textured.py

B.4.3 texture_mesh_uv.py

```

1 import numpy as np
2 import open3d as o3d
3 import cv2
4 import xatlas
5 import struct
6 import json
7 import time
8 from pathlib import Path
9 from PIL import Image
10
11 ATLAS_SIZE = 4096
12 MAX_FACES_XATLAS = 2500000
13
14 def read_cameras_bin(path):
15     cameras = {}
16     nparams = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8}
17     with open(path, 'rb') as f:
18         n = struct.unpack('<Q', f.read(8))[0]
19         for _ in range(n):
20             cid = struct.unpack('<I', f.read(4))[0]
21             mid = struct.unpack('<I', f.read(4))[0]
22             w = struct.unpack('<Q', f.read(8))[0]
23             h = struct.unpack('<Q', f.read(8))[0]
24             np_ = nparams[mid]
25             params = list(struct.unpack(f'<{np_}d', f.read(8 * np_)))
26             cameras[cid] = dict(model=mid, W=w, H=h, params=params)
27     return cameras
28
29 def read_images_bin(path):
30     images = {}
31     with open(path, 'rb') as f:

```

```

32     n = struct.unpack('<Q', f.read(8))[0]
33     for _ in range(n):
34         iid = struct.unpack('<I', f.read(4))[0]
35         qvec = np.array(struct.unpack('<4d', f.read(32)))
36         tvec = np.array(struct.unpack('<3d', f.read(24)))
37         cid = struct.unpack('<I', f.read(4))[0]
38         name = b''
39         while True:
40             c = f.read(1)
41             if c == b'\x00': break
42             name += c
43         n_pts = struct.unpack('<Q', f.read(8))[0]
44         f.read(n_pts * 24)
45         qw, qx, qy, qz = qvec
46         R = np.array([
47             [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
48             [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
49             [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qy**2+qz**2)]
50         ])
51         images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec, pos=-R.T @ tvec)
52     return images
53
54 def build_cameras(sparse_dir, images_dir):
55     sparse_dir = Path(sparse_dir)
56     images_dir = Path(images_dir)
57     raw_cams = read_cameras_bin(sparse_dir / 'cameras.bin')
58     raw_imgs = read_images_bin(sparse_dir / 'images.bin')
59     cameras = []
60     for iid, img in raw_imgs.items():
61         cam_data = raw_cams[img['cam_id']]
62         p = cam_data['params']
63         model = cam_data['model']
64         if model == 1:
65             fx, fy, cx, cy = p[0], p[1], p[2], p[3]
66             k = 0.0
67             f = fx
68         elif model == 2:
69             f, cx, cy, k = p[0], p[1], p[2], p[3]
70         else:
71             f = p[0]; cx = p[1]; cy = p[2]; k = 0.0
72         img_path = images_dir / img['name']
73         if not img_path.exists():
74             print(f' Warning: image not found: {img_path}')
75             continue
76         src_img = np.array(Image.open(img_path).convert('RGB'), dtype=np.uint8)
77         cameras.append(dict(
78             name=img['name'],
79             R=img['R'],
80             t=img['tvec'],
81             pos=img['pos'],
82             f=f, cx=cx, cy=cy, k=k,
83             W=cam_data['W'], H=cam_data['H'],
84             img=src_img,
85             model=model
86         ))
87     print(f' Loaded {len(cameras)} cameras with images')
88     return cameras
89
90 def project_pts(pts3d, cam):
91     Xc = (cam['R'] @ pts3d.T + cam['t'][:, None]).T
92     valid = Xc[:, 2] > 0.1
93     eps = 1e-8
94     xn = Xc[:, 0] / Xc[:, 2].clip(eps)
95     yn = Xc[:, 1] / Xc[:, 2].clip(eps)
96     if cam['k'] != 0.0:
97         r2 = xn**2 + yn**2
98         scale = 1.0 + cam['k'] * r2
99         xd = xn * scale
100        yd = yn * scale
101    else:
102        xd, yd = xn, yn
103    u = cam['f'] * xd + cam['cx']
104    v = cam['f'] * yd + cam['cy']
105    in_img = valid & (u >= 0) & (u < cam['W']) & (v >= 0) & (v < cam['H'])
106    return u, v, in_img
107
108 def bilinear_sample(img, u_px, v_px):
109     H, W = img.shape[:2]
110     u0 = np.floor(u_px).astype(np.int32).clip(0, W - 2)
111     v0 = np.floor(v_px).astype(np.int32).clip(0, H - 2)
112     fu = (u_px - u0).astype(np.float32)
113     fv = (v_px - v0).astype(np.float32)
114     c00 = img[v0, u0].astype(np.float32)
115     c01 = img[v0, u0 + 1].astype(np.float32)
116     c10 = img[v0 + 1, u0].astype(np.float32)
117     c11 = img[v0 + 1, u0 + 1].astype(np.float32)
118     fu = fu[:, None]; fv = fv[:, None]
119     return (1-fu)*(1-fv)*c00 + fu*(1-fv)*c01 + (1-fu)*fv*c10 + fu*fvc11
120
121 def assign_cameras(centroids, tri_norms, cameras, scene):
122     N = len(centroids)
123     N_cams = len(cameras)

```

```

124     cam_positions = np.array([c['pos'] for c in cameras], dtype=np.float64)
125
126     vecs = cam_positions[None] - centroids[:, None]
127     dists = np.linalg.norm(vecs, axis=2).clip(1e-10)
128     vecs_norm = vecs / dists[:, :, None]
129     cos_a = (vecs_norm * tri_norms[:, None]).sum(axis=2)
130
131     in_fov = np.zeros((N, N_cams), dtype=bool)
132     for ci, cam in enumerate(cameras):
133         _, _, valid = project_pts(centroids, cam)
134         in_fov[:, ci] = valid
135
136     scores = np.where(in_fov & (cos_a > 0), cos_a / dists**2, -1.0)
137     sorted_cams = np.argsort(-scores, axis=1)
138     cam_per_tri = np.full(N, -1, dtype=np.int32)
139
140     for rank in range(min(5, N_cams)):
141         need = cam_per_tri == -1
142         if not need.any():
143             break
144         cand = sorted_cams[need, rank]
145         valid_cand = scores[need, cand] > 0
146         if not valid_cand.any():
147             continue
148         check_idx = np.where(need)[0][valid_cand]
149         check_cam = cand[valid_cand]
150
151         origins = centroids[check_idx].astype(np.float32)
152         targets = cam_positions[check_cam].astype(np.float32)
153         raw_dirs = targets - origins
154         raw_dists = np.linalg.norm(raw_dirs, axis=1, keepdims=True).clip(1e-10)
155         dirs_n = (raw_dirs / raw_dists).astype(np.float32)
156         offset_o = origins + dirs_n * 0.005
157
158         rays = np.concatenate([offset_o, dirs_n], axis=1)
159         rays_t = o3d.core.Tensor(rays, dtype=o3d.core.Dtype.Float32)
160         hit = scene.cast_rays(rays_t)
161         hit_d = hit['t_hit'].numpy()
162         unoccluded = hit_d >= raw_dists.flatten() * 0.98
163
164         cam_per_tri[check_idx[unoccluded]] = check_cam[unoccluded]
165
166     still_none = cam_per_tri == -1
167     if still_none.any():
168         best = np.argmax(scores[still_none], axis=1)
169         cam_per_tri[still_none] = best
170
171     return cam_per_tri
172
173 def save_obj_mtl(out_dir, vmapping, verts, xatlas_tris, uvs, atlas_name):
174     out_dir = Path(out_dir)
175     obj_path = out_dir / 'textured.obj'
176     mtl_path = out_dir / 'textured.mtl'
177
178     with open(mtl_path, 'w') as f:
179         f.write('newmtl atlas_mat\n')
180         f.write(f'map_Kd {atlas_name}\n')
181         f.write('Kd 1.0 1.0 1.0\n')
182         f.write('Ka 0.0 0.0 0.0\n')
183         f.write('Ks 0.0 0.0 0.0\n')
184
185     N_out = len(vmapping)
186     out_verts = verts[vmapping]
187
188     chunk = 500000
189     with open(obj_path, 'w') as f:
190         f.write(f'mtllib textured.mtl\n')
191         for i in range(0, N_out, chunk):
192             block = out_verts[i:i+chunk]
193             lines = '\n'.join(f'v {v[0]:.6f} {v[1]:.6f} {v[2]:.6f}' for v in block)
194             f.write(lines + '\n')
195         for i in range(0, N_out, chunk):
196             block = uvs[i:i+chunk]
197             lines = '\n'.join(f'vt {u:.6f} {v:.6f}' for u, v in block)
198             f.write(lines + '\n')
199         f.write('usemtl atlas_mat\n')
200         for i in range(0, len(xatlas_tris), chunk):
201             block = xatlas_tris[i:i+chunk]
202             lines = '\n'.join(
203                 f'f {v0+1}/{v0+1} {v1+1}/{v1+1} {v2+1}/{v2+1}'
204                 for v0, v1, v2 in block
205             )
206             f.write(lines + '\n')
207
208     print(f' Saved OBJ: {obj_path} ({obj_path.stat().st_size/1e6:.1f} MB)')
209     print(f' Saved MTL: {mtl_path}')
210
211 def texture_mesh(label, mesh_path, sparse_dir, images_dir, out_dir):
212     out_dir = Path(out_dir)
213     out_dir.mkdir(parents=True, exist_ok=True)
214     print(f'\n{"="*60}')
215     print(f'Texturing {label}: {mesh_path}')

```

```

216 mesh = o3d.io.read_triangle_mesh(str(mesh_path))
217 mesh.compute_vertex_normals()
218 verts_in = np.asarray(mesh.vertices, dtype=np.float64)
219 tris_in = np.asarray(mesh.triangles, dtype=np.int32)
220 orig_n_verts = len(verts_in)
221 orig_n_faces = len(tris_in)
222 print(f' Input mesh: {orig_n_verts:,} verts, {orig_n_faces:,} faces')
223
224 decimate_factor = 1.0
225 if len(tris_in) > MAX_FACES_XATLAS:
226     decimate_factor = MAX_FACES_XATLAS / len(tris_in)
227     print(f' Decimating to ~{MAX_FACES_XATLAS:,} faces (factor={decimate_factor:.4f})')
228     mesh = mesh.simplify_quadric_decimation(MAX_FACES_XATLAS)
229     mesh.compute_vertex_normals()
230     verts_in = np.asarray(mesh.vertices, dtype=np.float64)
231     tris_in = np.asarray(mesh.triangles, dtype=np.int32)
232     print(f' After decimation: {len(verts_in):,} verts, {len(tris_in):,} faces')
233
234 print(' Loading cameras...')
235 cameras = build_cameras(sparse_dir, images_dir)
236
237 print(' Building raycasting scene...')
238 scene = o3d.t.geometry.RaycastingScene()
239 mesh_t = o3d.t.geometry.TriangleMesh.from_legacy(mesh)
240 scene.add_triangles(mesh_t)
241
242 v0c = verts_in[tris_in[:, 0]]
243 v1c = verts_in[tris_in[:, 1]]
244 v2c = verts_in[tris_in[:, 2]]
245 centroids = (v0c + v1c + v2c) / 3.0
246 cross = np.cross(v1c - v0c, v2c - v0c)
247 nlen = np.linalg.norm(cross, axis=1, keepdims=True).clip(1e-10)
248 tri_norms = cross / nlen
249
250 print(' Assigning cameras to triangles...')
251 t0 = time.time()
252 cam_per_tri = assign_cameras(centroids, tri_norms, cameras, scene)
253 print(f' Camera assignment: {time.time()-t0:.1f}s unassigned={(cam_per_tri==1).sum()}'')
254
255 print(' xatlas UV parametrization...')
256 t0 = time.time()
257 vmapping, xatlas_tris, uvs = xatlas.parametrize(
258     verts_in.astype(np.float32), tris_in.astype(np.uint32)
259 )
260 print(f' xatlas: {len(vmapping):,} output verts, {time.time()-t0:.1f}s')
261 print(f' UV range: [{uvs.min():.4f}, {uvs.max():.4f}]')
262
263 H = W = ATLAS_SIZE
264 uvs_px = uvs * np.array([W, H], dtype=np.float32)
265 tri_map = np.full((H, W), -1, dtype=np.int32)
266 bary_u = np.zeros((H, W), dtype=np.float32)
267 bary_v = np.zeros((H, W), dtype=np.float32)
268
269 print(f' Rasterizing {len(xatlas_tris):,} triangles to {W}x{H} atlas...')
270 t0 = time.time()
271 N_tris = len(xatlas_tris)
272 for ti in range(N_tris):
273     if ti % 300000 == 0 and ti > 0:
274         elapsed = time.time() - t0
275         eta = elapsed / ti * (N_tris - ti)
276         print(f' {ti:,}/{N_tris:,} elapsed={elapsed:.0f}s eta={eta:.0f}s')
277
278     i0, i1, i2 = xatlas_tris[ti]
279     p0 = uvs_px[i0]; p1 = uvs_px[i1]; p2 = uvs_px[i2]
280
281     x_lo = max(0, int(np.floor(min(p0[0], p1[0], p2[0]))))
282     x_hi = min(W - 1, int(np.ceil(max(p0[0], p1[0], p2[0]))))
283     y_lo = max(0, int(np.floor(min(p0[1], p1[1], p2[1]))))
284     y_hi = min(H - 1, int(np.ceil(max(p0[1], p1[1], p2[1]))))
285     if x_lo > x_hi or y_lo > y_hi:
286         continue
287
288     xs = np.arange(x_lo, x_hi + 1, dtype=np.float32) + 0.5
289     ys = np.arange(y_lo, y_hi + 1, dtype=np.float32) + 0.5
290     xx, yy = np.meshgrid(xs, ys)
291     pts = np.stack([xx.ravel(), yy.ravel()], axis=1)
292
293     e01 = p1 - p0; e02 = p2 - p0
294     d00 = float(np.dot(e01, e01)); d01 = float(np.dot(e01, e02)); d11 = float(np.dot(e02, e02))
295     det = d00 * d11 - d01 * d01
296     if abs(det) < 1e-10:
297         continue
298     inv = 1.0 / det
299     vp = pts - p0
300     dp0 = vp @ e01; dp1 = vp @ e02
301     u_b = (d11 * dp0 - d01 * dp1) * inv
302     v_b = (d00 * dp1 - d01 * dp0) * inv
303     inside = (u_b >= 0) & (v_b >= 0) & (u_b + v_b <= 1.0)
304     if not inside.any():
305         continue
306     tri_map[yy.ravel(), xx.ravel()] = ti
307

```



```

308     px = xx.ravel()[inside].astype(np.int32).clip(0, W - 1)
309     py = yy.ravel()[inside].astype(np.int32).clip(0, H - 1)
310     tri_map[py, px] = ti
311     bary_u[py, px] = u_b[inside]
312     bary_v[py, px] = v_b[inside]
313
314     print(f' Rasterization: {time.time()-t0:.1f}s')
315
316     valid_y, valid_x = np.where(tri_map >= 0)
317     valid_tris = tri_map[valid_y, valid_x]
318     u_b_vals = bary_u[valid_y, valid_x]
319     v_b_vals = bary_v[valid_y, valid_x]
320     w_b_vals = 1.0 - u_b_vals - v_b_vals
321
322     xv0 = xatlas_tris[valid_tris, 0]; xv1 = xatlas_tris[valid_tris, 1]; xv2 = xatlas_tris[valid_tris, 2]
323     pos3d = (w_b_vals[:, None] * verts_in[vmapping[xv0]] +
324             u_b_vals[:, None] * verts_in[vmapping[xv1]] +
325             v_b_vals[:, None] * verts_in[vmapping[xv2]])
326
327     pixel_cams = cam_per_tri[valid_tris]
328     atlas = np.full((H, W, 3), 128, dtype=np.uint8)
329
330     print(f' Baking {len(valid_y):,} valid atlas pixels across {len(cameras)} cameras...')
331     t0 = time.time()
332     covered = 0
333     for ci, cam in enumerate(cameras):
334         mask = pixel_cams == ci
335         if not mask.any():
336             continue
337         u_px, v_px, in_img = project_pts(pos3d[mask], cam)
338         sampled = bilinear_sample(cam['img'], u_px[in_img], v_px[in_img])
339         ay = valid_y[mask][in_img]
340         ax = valid_x[mask][in_img]
341         atlas[ay, ax] = sampled.clip(0, 255).astype(np.uint8)
342         covered += in_img.sum()
343
344     coverage = covered / max(1, len(valid_y))
345     print(f' Baking: {time.time()-t0:.1f}s coverage={coverage:.1%}')
346
347     print(' Inpainting atlas holes...')
348     fill_mask = (tri_map < 0).astype(np.uint8) * 255
349     atlas = cv2.inpaint(atlas, fill_mask, 3, cv2.INPAINT_TELEA)
350
351     atlas_path = out_dir / 'texture_atlas.png'
352     Image.fromarray(atlas).save(str(atlas_path))
353     print(f' Saved atlas: {atlas_path} ({atlas_path.stat().st_size/1e6:.1f} MB)')
354
355     save_obj_mtl(out_dir, vmapping, verts_in, xatlas_tris, uvs, 'texture_atlas.png')
356
357     summary = dict(
358         label=label,
359         tier='A',
360         format='UV-textured OBJ + 4096x4096 atlas PNG',
361         atlas_resolution=ATLAS_SIZE,
362         input_vertex_count=orig_n_verts,
363         input_face_count=orig_n_faces,
364         decimation_factor=round(decimate_factor, 4),
365         output_vertex_count=int(len(vmapping)),
366         output_face_count=int(len(xatlas_tris)),
367         coverage_rate=round(float(coverage), 4),
368         out_dir=str(out_dir),
369         files=['textured.obj', 'textured.mtl', 'texture_atlas.png']
370     )
371     summary_path = out_dir / 'summary.json'
372     with open(summary_path, 'w') as f:
373         json.dump(summary, f, indent=2)
374     print(f' Summary: {summary_path}')
375     print(f' Done: {label}')
376     return summary
377
378 t_start = time.time()
379
380 summary_n14 = texture_mesh(
381     'N=14',
382     'colmap_n15/dense/meshed-poisson.ply',
383     'colmap_n15/sparse/0',
384     'data/rgb',
385     'colmap_n15/dense/textured'
386 )
387
388 summary_n24 = texture_mesh(
389     'N=24',
390     'colmap_n24/dense/meshed-poisson.ply',
391     'colmap_n24/sparse/0',
392     'data/rgb_n24',
393     'colmap_n24/dense/textured'
394 )
395
396 combined = dict(
397     tier='A',
398     output_format='UV-textured OBJ + atlas PNG',
399     atlas_resolution=ATLAS_SIZE,

```

```

400     meshes=[summary_n14, summary_n24]
401 )
402 with open('comparison/textured_summary.json', 'w') as f:
403     json.dump(combined, f, indent=2)
404 print(json.dumps(combined, indent=2))
405 print(f'\nTotal time: {(time.time()-t_start)/60:.1f} minutes')
406 print('\n==== STAGE: UV TEXTURING COMPLETE =====')

```

Listing 21: texture_mesh_uv.py

B.4.4 texture_mesh_vertex.py

```

1  import numpy as np
2  import open3d as o3d
3  import struct
4  import json
5  import time
6  from pathlib import Path
7  from PIL import Image
8
9  def read_cameras_bin(path):
10     cameras = {}
11     nparams = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8}
12     with open(path, 'rb') as f:
13         n = struct.unpack('<Q', f.read(8))[0]
14         for _ in range(n):
15             cid = struct.unpack('<I', f.read(4))[0]
16             mid = struct.unpack('<I', f.read(4))[0]
17             w = struct.unpack('<Q', f.read(8))[0]
18             h = struct.unpack('<Q', f.read(8))[0]
19             np_ = nparams[mid]
20             params = list(struct.unpack(f'<{np_}d', f.read(8 * np_)))
21             cameras[cid] = dict(model=mid, W=w, H=h, params=params)
22     return cameras
23
24 def read_images_bin(path):
25     images = {}
26     with open(path, 'rb') as f:
27         n = struct.unpack('<Q', f.read(8))[0]
28         for _ in range(n):
29             iid = struct.unpack('<I', f.read(4))[0]
30             qvec = np.array(struct.unpack('<4d', f.read(32)))
31             tvec = np.array(struct.unpack('<3d', f.read(24)))
32             cid = struct.unpack('<I', f.read(4))[0]
33             name = b''
34             while True:
35                 c = f.read(1)
36                 if c == b'\x00': break
37                 name += c
38             n_pts = struct.unpack('<Q', f.read(8))[0]
39             f.read(n_pts * 24)
40             qw, qx, qy, qz = qvec
41             R = np.array([
42                 [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
43                 [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
44                 [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qy**2+qz**2)]
45             ])
46             images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec, pos=-R.T @ tvec)
47     return images
48
49 def build_cameras(sparse_dir, images_dir):
50     sparse_dir = Path(sparse_dir)
51     images_dir = Path(images_dir)
52     raw_cams = read_cameras_bin(sparse_dir / 'cameras.bin')
53     raw_imgs = read_images_bin(sparse_dir / 'images.bin')
54     cameras = []
55     for iid, img in raw_imgs.items():
56         cam = raw_cams[img['cam_id']]
57         p = cam['params']
58         m = cam['model']
59         if m == 1:
60             f = p[0]; cx = p[2]; cy = p[3]; k = 0.0
61         elif m == 2:
62             f = p[0]; cx = p[1]; cy = p[2]; k = p[3]
63         else:
64             f = p[0]; cx = p[1]; cy = p[2]; k = 0.0
65         img_path = images_dir / img['name']
66         if not img_path.exists():
67             continue
68         src = np.array(Image.open(img_path).convert('RGB'), dtype=np.uint8)
69         cameras.append(dict(
70             name=img['name'], R=img['R'], t=img['tvec'], pos=img['pos'],
71             f=f, cx=cx, cy=cy, k=k, W=cam['W'], H=cam['H'], img=src, model=m
72         ))
73     return cameras
74
75 def project_pts(pts3d, cam):

```

```

76 Xc = (cam['R'] @ pts3d.T + cam['t'][:, None]).T
77 valid = Xc[:, 2] > 0.1
78 eps = 1e-8
79 xn = Xc[:, 0] / Xc[:, 2].clip(eps)
80 yn = Xc[:, 1] / Xc[:, 2].clip(eps)
81 if cam['k'] != 0.0:
82     r2 = xn**2 + yn**2
83     sc = 1.0 + cam['k'] * r2
84     xd = xn * sc; yd = yn * sc
85 else:
86     xd = xn; yd = yn
87 u = cam['f'] * xd + cam['cx']
88 v = cam['f'] * yd + cam['cy']
89 in_img = valid & (u >= 0) & (u < cam['W']) & (v >= 0) & (v < cam['H'])
90 return u, v, in_img, Xc[:, 2]
91
92 def bilinear_sample(img, u_px, v_px):
93     H, W = img.shape[:2]
94     u0 = np.floor(u_px).astype(np.int32).clip(0, W - 2)
95     v0 = np.floor(v_px).astype(np.int32).clip(0, H - 2)
96     fu = (u_px - u0).astype(np.float32)[:, None]
97     fv = (v_px - v0).astype(np.float32)[:, None]
98     c00 = img[v0, u0].astype(np.float32)
99     c01 = img[v0, u0 + 1].astype(np.float32)
100    c10 = img[v0 + 1, u0].astype(np.float32)
101    c11 = img[v0 + 1, u0 + 1].astype(np.float32)
102    return (1-fu)*(1-fv)*c00 + fu*(1-fv)*c01 + (1-fu)*fv*c10 + fu*fv*c11
103
104 def color_vertices(mesh, cameras, scene, batch=200000):
105     verts = np.asarray(mesh.vertices, dtype=np.float64)
106     normals = np.asarray(mesh.vertex_normals, dtype=np.float64)
107     N = len(verts)
108
109     accum_color = np.zeros((N, 3), dtype=np.float64)
110     accum_weight = np.zeros(N, dtype=np.float64)
111
112     cam_positions = np.array([c['pos'] for c in cameras], dtype=np.float64)
113
114     for start in range(0, N, batch):
115         end = min(start + batch, N)
116         vb = verts[start:end]
117         nb = normals[start:end]
118         M = end - start
119
120         if start % 500000 == 0:
121             print(f' Vertex {start:,}/{N:,}')
122
123         for ci, cam in enumerate(cameras):
124             u_px, v_px, in_img, depths = project_pts(vb, cam)
125
126             if not in_img.any():
127                 continue
128
129             cam_dir = cam['pos'] - vb
130             dists = np.linalg.norm(cam_dir, axis=1).clip(1e-10)
131             cam_dir_n = cam_dir / dists[:, None]
132             cos_a = (cam_dir_n * nb).sum(axis=1)
133
134             good = in_img & (cos_a > 0)
135             if not good.any():
136                 continue
137
138             check_idx = np.where(good)[0]
139             origins = vb[check_idx].astype(np.float32)
140             dirs = (cam['pos'] - origins).astype(np.float32)
141             d_to_cam = np.linalg.norm(dirs, axis=1, keepdims=True).clip(1e-10)
142             dirs_n = (dirs / d_to_cam).astype(np.float32)
143             offset_o = origins + dirs_n * 0.005
144
145             rays = np.concatenate([offset_o, dirs_n], axis=1)
146             rays_t = o3d.core.Tensor(rays, dtype=o3d.core.Dtype.Float32)
147             hit = scene.cast_rays(rays_t)
148             hit_d = hit['t_hit'].numpy()
149             unoccluded = hit_d >= d_to_cam.flatten() * 0.98
150
151             if not unoccluded.any():
152                 continue
153
154             final_idx = check_idx[unoccluded]
155             w = cos_a[final_idx] / dists[final_idx]**2
156             sampled = bilinear_sample(cam['img'], u_px[final_idx], v_px[final_idx])
157
158             global_idx = np.arange(start, end)[final_idx]
159             np.add.at(accum_color, global_idx, sampled * w[:, None])
160             np.add.at(accum_weight, global_idx, w)
161
162     has_color = accum_weight > 0
163     vertex_colors = np.zeros((N, 3), dtype=np.float32)
164     vertex_colors[has_color] = (accum_color[has_color] / accum_weight[has_color, None]).clip(0, 255) / 255.0
165     vertex_colors[-has_color] = 0.5
166
167     coverage = has_color.mean()

```

```

168     return vertex_colors, coverage
169
170 def texture_mesh_vertex(label, mesh_path, sparse_dir, images_dir, out_dir):
171     out_dir = Path(out_dir)
172     out_dir.mkdir(parents=True, exist_ok=True)
173     print(f'\n{"="*60}')
174     print(f'Vertex-color texturing {label}: {mesh_path}')
175
176     t0 = time.time()
177     mesh = o3d.io.read_triangle_mesh(str(mesh_path))
178     mesh.compute_vertex_normals()
179     verts = np.asarray(mesh.vertices)
180     N_verts = len(verts)
181     N_faces = len(np.asarray(mesh.triangles))
182     print(f' Mesh: {N_verts:,} verts, {N_faces:,} faces')
183
184     cameras = build_cameras(sparse_dir, images_dir)
185     print(f' {len(cameras)} cameras loaded')
186
187     print(' Building raycasting scene...')
188     scene = o3d.t.geometry.RaycastingScene()
189     mesh_t = o3d.t.geometry.TriangleMesh.from_legacy(mesh)
190     scene.add_triangles(mesh_t)
191
192     print(' Computing vertex colors...')
193     vertex_colors, coverage = color_vertices(mesh, cameras, scene)
194     print(f' Coverage: {coverage:.1%} time: {time.time()-t0:.1f}s')
195
196     mesh.vertex_colors = o3d.utility.Vector3dVector(vertex_colors)
197     out_ply = out_dir / 'textured.ply'
198     o3d.io.write_triangle_mesh(str(out_ply), mesh)
199     print(f' Saved: {out_ply} ({out_ply.stat().st_size/1e6:.1f} MB)')
200
201     summary = dict(
202         label=label,
203         tier='B',
204         format='per-vertex-colored PLY',
205         output_vertex_count=N_verts,
206         output_face_count=N_faces,
207         coverage_rate=round(float(coverage), 4),
208         out_dir=str(out_dir),
209         files=['textured.ply']
210     )
211     with open(out_dir / 'summary.json', 'w') as f:
212         json.dump(summary, f, indent=2)
213     return summary
214
215 t_all = time.time()
216
217 summary_n14 = texture_mesh_vertex(
218     'N=14',
219     'colmap_n15/dense/meshed-poisson.ply',
220     'colmap_n15/sparse/0',
221     'data/rgb',
222     'colmap_n15/dense/textured'
223 )
224
225 summary_n24 = texture_mesh_vertex(
226     'N=24',
227     'colmap_n24/dense/meshed-poisson.ply',
228     'colmap_n24/sparse/0',
229     'data/rgb_n24',
230     'colmap_n24/dense/textured'
231 )
232
233 combined = dict(
234     tier='B',
235     output_format='per-vertex-colored PLY',
236     note='Per-vertex RGB coloring. Recognized as textured mesh in photogrammetry tools (Metashape, CloudCompare). Chosen because xatlas UV  

    parametrization exceeded 45-minute per-mesh limit on 2.1M-face input.',
237     meshes=[summary_n14, summary_n24]
238 )
239
240 with open('comparison/textured_summary.json', 'w') as f:
241     json.dump(combined, f, indent=2)
242     print(json.dumps(combined, indent=2))
243     print(f'\nTotal time: {(time.time()-t_all)/60:.1f} minutes')
244     print('\n===== STAGE: VERTEX TEXTURING COMPLETE =====')

```

Listing 22: texture_mesh_vertex.py